



INTERNATIONAL
HELLENIC
UNIVERSITY

Efficient Disentangled Graph Embeddings for Resource-Aware Recommendation under Observational Bias

Lazaros Panitsidis

SID: 3308230021

SCHOOL OF SCIENCE & TECHNOLOGY

A thesis submitted for the degree of

Master of Science (MSc) in Data Science

JULY 2026

THESSALONIKI – GREECE



Efficient Disentangled Graph Embeddings for Resource-Aware Recommendation under Observational Bias

Lazaros Panitsidis

SID: 3308230021

Supervisor: Professor Apostolos N. Papadopoulos

Department of Informatics, Aristotle University of Thessaloniki, Greece

Supervising Committee Members:

Professor Konstantinos Diamantaras
Department of Information and Electronic
Engineering
International Hellenic University

Professor Panayiotis Bozanis
Department of Science and Technology
International Hellenic University

SCHOOL OF SCIENCE & TECHNOLOGY

A thesis submitted for the degree of

Master of Science (MSc) in Data Science

© 2026 by Lazaros Panitsidis. All Rights Reserved.

Redistributed by International Hellenic University under licence from the author.

This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International Licence.



<https://creativecommons.org/licenses/by-nc/4.0/>

“Nobody ever figures out what life is all about, and it doesn’t matter. Explore the world. Nearly everything is really interesting if you go into it deeply enough.”

— **Richard P. Feynman**

“The important thing is not to stop questioning.”

— **Albert Einstein**

«Ἐν οἶδα ὅτι οὐδὲν οἶδα»

(I know one thing, that I know nothing)

— **Socrates**

“It always seems impossible until it is done.”

— **Nelson Mandela**

Acknowledgements

I am sincerely grateful to Professor Apostolos N. Papadopoulos for his interest, guidance, trust, and support throughout our collaboration.

My deepest love and gratitude go to my parents, Konstantinos and Chrysi, and to my sister, Irene, for their sacrifices, love, encouragement, and unwavering support throughout these years.

I am also deeply thankful to my dear friends Alexandros and Savvas, who have stood by me in both good times and difficult moments and have supported me in everything I do. Their encouragement has helped me grow as a person and as a scientist.

I also thank my relatives, friends, and acquaintances for every contribution they have made to my personal and professional journey, and for the beautiful moments we have shared.

Abstract

This dissertation studies resource-aware graph recommendation from observational implicit-feedback data. Logged user-item interactions are shaped by exposure, popularity concentration, conformity, platform policy, and preprocessing; consequently, a missing entry is not a verified negative preference. The thesis adopts a protocol-centred principle: ranking quality, list behaviour, runtime, and memory are interpreted together with the split, item universe, candidate policy, and hardware setting that produced them.

The proposed method is EDGRec, the Efficient Disentangled Graph Recommender. EDGRec uses the interest-conformity hypothesis from Disentangling User Interest and Conformity for Recommendation with Causal Embedding (DICE) as a modelling guide and operationalizes it as a resource-aware graph recommendation framework. The method combines Light Graph Convolutional Network (LightGCN) propagation, sampled graph training, bounded auxiliary losses, controlled score fusion, train-only context construction, and an item-level context scoring path. The empirical claims are therefore stated in terms of observed ranking behaviour, popularity-aware list behaviour, resource use, matched ablations, and branch diagnostics, rather than identified causal effects.

For evaluation, the thesis formalizes Composite Resource-aware Recommendation Utility (CRRU), a configurable post hoc utility that combines ranking quality, popularity-aware personalization, average training epoch time, and peak training video random-access memory (VRAM) after the component metrics have been reported. CRRU is a reusable aggregation framework with declared indicators, transformations, and weights; the weight vector used here expresses this dissertation’s resource-aware preference, not a universal definition of recommendation quality. ValidationCRRU is used only as validation-split model-selection evidence where declared, while final-test CRRU is interpreted after completed test evaluation.

The empirical evidence is dataset-specific. In the sparse KuaiRec v2 watch-ratio comparison, the selected EDGRec configuration improves the reported ranking metrics and substantially shortens epoch time relative to the reference LightGCN baseline, while Personalization and peak VRAM remain less favourable. AmazonBook supports an epoch-time trade-off rather than a ranking improvement; MovieLens-1M shows closely matched ranking with an epoch-time benefit but less favourable list behaviour diagnostics; and KuaiRand-1K shows a compact randomized-exposure trade-off in which runtime, VRAM, and Recall improve while NDCG, HitRatio, CRRU, and raw AveragePopularity remain less favourable than the LightGCN comparison with stronger top-position ranking evidence.

The main contribution is Pareto-oriented. EDGRec shows that sampled branch-

structured graph training can expand the feasible accuracy–resource frontier in selected regimes under explicit data and hardware constraints, while relevance remains the primary evidence and efficiency gains are interpreted beside the corresponding ranking, list behaviour, and memory components.

Lazaros Panitsidis
July 2026

Contents

Acknowledgements	iii
Abstract	iv
Document Acronyms	xv
1 Introduction	1
1.1 Motivation and Problem Setting	1
1.2 Theoretical Framework and Literature Fit	3
1.3 Scope and Original Contributions	4
1.4 Thesis Structure	5
2 Literature Review	6
2.1 Implicit-Feedback Recommendation	6
2.1.1 Collaborative filtering	7
2.1.2 Explicit and implicit feedback	8
2.1.3 Top-K recommendation	8
2.1.4 Ranking metrics and pairwise learning objectives	10
2.2 Dataset Construction and Observational Feedback	11
2.2.1 Dataset selection	12
2.2.2 Interaction construction	12
2.2.3 Positive-label definition	13
2.2.4 Missingness and exposure	14
2.2.5 Feature selection and leakage risks	14
2.3 Graph Collaborative Filtering	15
2.3.1 Graph representation learning	15
2.3.2 Message passing and neighborhood aggregation	16
2.3.3 Graph convolutional networks	17
2.3.4 User-item bipartite graphs	18
2.3.5 LightGCN and simplified graph propagation	18
2.3.6 Computational cost of graph learning	21
2.3.7 Full-graph and sampled graph training	22

2.3.8	Graph recommendation efficiency challenges	22
2.4	Bias and Causal Motivation in Recommendation	24
2.4.1	Exposure bias	24
2.4.2	Popularity bias and feedback loops	25
2.4.3	Conformity	26
2.4.4	Correlation and causal interpretation	27
2.4.5	Causal recommendation and exposure-aware methods	28
2.5	Disentangled Representation Learning for Recommendation	30
2.5.1	General disentangled representation learning	30
2.5.2	DICE as the direct conceptual anchor	31
2.5.3	Contrastive causal-disentangled extensions	33
2.5.4	Temporal and dynamic disentanglement	34
2.5.5	Semantic, knowledge-guided, and intent-aware disentanglement . .	35
2.5.6	Collapse and efficient guidance for multi-interest learning	36
2.5.7	From causal disentanglement to efficient graph recommendation .	39
2.6	Efficiency and Resource-Aware Evaluation in Graph Recommendation . .	39
2.6.1	Runtime and memory constraints	39
2.6.2	GPU memory constraints	40
2.6.3	Accuracy-efficiency trade-offs	41
2.6.4	Motivation for composite post hoc reporting	42
2.6.5	Synthesis of literature streams	42
2.7	Theoretical Framework and Research Gap	44
2.7.1	Need for efficient, bias-aware graph recommendation	44
2.7.2	Need for protocol-centred evaluation	45
2.7.3	Need for resource-aware comparison	46
3	Problem Definition and Methodology	48
3.1	Problem Definition	48
3.1.1	Observed implicit-feedback top- K task	49
3.1.2	Split masks and bipartite training graph	50
3.1.3	Observed positives, unobserved pairs, and interpretation scope . . .	51
3.1.4	Resource-aware objective scope	51
3.2	Data and Preprocessing Protocol	52
3.2.1	Datasets, roles, and interpretation scope	52
3.2.2	Dataset-specific canonicalization and positive labels	53
3.2.3	Repeat interaction collapse and timestamp ordering	56
3.2.4	Split construction, candidate filtering, and negative sampling	57
3.2.5	Graph construction and train-derived context	58
3.2.6	Feature policy and leakage control	60
3.3	Baseline Models	60

3.3.1	Baseline and variant taxonomy	60
3.3.2	Reference objectives and propagation	63
3.4	EDGRec Architecture	64
3.4.1	Embedding layer	68
3.4.2	Interest branch	68
3.4.3	Conformity branch	69
3.4.4	Context and popularity head	69
3.4.5	Score fusion	70
3.4.6	Computational design choices	70
3.5	Training Objective	71
3.5.1	Main BPR loss	71
3.5.2	Branch-specific losses	73
3.5.3	Disentanglement and independence regularization	74
3.5.4	Optional geometry, propensity, or feature ablations	74
3.5.5	Optimization and early stopping	75
3.6	Evaluation Protocol	75
3.6.1	Validation-test boundary and evidence roles	75
3.6.2	Ranking metrics	77
3.6.3	Popularity and personalization metrics	78
3.6.4	Runtime and VRAM measurement	78
3.6.5	CRRU definition	78
3.6.6	CRRU limitations and sensitivity checks	80
3.7	Reproducibility	81
3.7.1	Random seeds	81
3.7.2	Experiment records and model states	81
4	Experimental Setup	83
4.1	Experimental Questions	84
4.2	Dataset-Specific Configurations	86
4.3	Hyperparameter Search and Candidate Selection	87
4.4	Baseline Configuration	100
4.5	EDGRec Configuration Families	101
4.6	Ablation Design	102
4.7	Runtime and Memory Measurement Protocol	106
4.8	Threats to Experimental Validity	108
4.9	Experimental Protocol Summary	109
5	Results and Discussion	111
5.1	Result Reporting Protocol	111
5.2	Comparison Structure	112

5.3	Data and Preprocessing Context	112
5.4	Validation Search and Feature Evidence	113
5.5	Dataset-Wise Metric Tables	116
5.6	Resource-Aware Utility and Diagnostic Plots	120
5.7	Reference DICE Feasibility	124
5.8	Ablation Evidence	125
5.9	Dataset-Level Discussion	127
5.9.1	AmazonBook	127
5.9.2	MovieLens-1M	127
5.9.3	KuaiRec v2	129
5.9.4	KuaiRand-1K	130
5.10	Answers to the Research Questions	130
5.11	Limitations of the Empirical Evidence	131
5.12	Empirical Summary	133
6	Conclusion	134
6.1	Summary of Contributions	134
6.2	Main Empirical Findings	135
6.3	Scientific Limitations	136
6.4	Engineering Limitations	137
6.5	Future Work	137
6.6	Final Remarks	138
	Bibliography	139
A	Supplementary Material	143
A.1	Data and Feature-Lineage Audit	143
A.2	Environment and Reproducibility Checklist	143

List of Figures

2.1	Typical recommender-system taxonomy and pipeline	7
2.2	Observed implicit-feedback matrix and top- K pipeline	9
2.3	BPR pairwise preference construction	11
2.4	Representative graph structures in recommender systems	16
2.5	GNN framework for user-item collaborative filtering	17
2.6	User-item graph and LightGCN propagation	19
2.7	Original LightGCN architecture	20
2.8	Observational-bias motivation directed acyclic graph	25
2.9	DICE feedback loop of recommender systems	26
2.10	SCM motif for interest and conformity	27
2.11	DICE causal graph and causal embeddings	31
2.12	DDCE feedback loop of user-item interaction	35
2.13	Disentangled recommendation literature map	37
2.14	Resource-aware evaluation triangle	44
3.1	Data-to-evaluation pipeline	52
3.2	EDGRec empirical interpretation boundary	53
3.3	Dataset regimes used in the thesis	55
3.4	Train feedback graph topology with sign diagnostics	59
3.5	EDGRec computational schematic	67
3.6	EDGRec score and loss composition	77
4.1	AmazonBook dataset profile	88
4.2	MovieLens-1M dataset profile	88
4.3	KuaiRec v2 dataset profile	89
4.4	KuaiRand-1K exposure-aware dataset profile	89
4.5	Positive-training-degree distributions	90
4.6	Optuna validation search system design	91
4.7	Validation-to-test evidence gate	93
4.8	Validation search selection frontier	96
4.9	ValidationCRRU components by dataset	97
4.10	Validation search hyperparameter importance	97

4.11	Validation-only feature subset movement	99
4.12	Validation search branch-depth diagnostics	103
4.13	Validation search learning-rate and branch-fusion landscape	104
4.14	Validation search fan-out and runtime trade-offs	105
5.1	Encoded item-feature UMAP diagnostics	115
5.2	Claim matrix for the main LightGCN comparison	117
5.3	CRRU component decomposition	121
5.4	Full-range accuracy–efficiency frontier	122
5.5	Full-range accuracy–popularity trade-off	122
5.6	Reference DICE feasibility evaluations	124
5.7	KuaiRec v2 matched ablation deltas	126
5.8	EDGRec component diagnostics	127
5.9	Learned EDGRec interest-embedding UMAP diagnostics	128
5.10	KuaiRec sparse versus small-matrix sensitivity	129

List of Tables

2.1	Disentangled recommendation literature and relevance to EDGRec.	38
2.2	Synthesis of literature streams and thesis implications.	43
3.1	Dataset roles and interpretation scope	54
3.2	Dataset-specific preprocessing protocols used by the experimental pipeline	54
3.3	Canonical label and sign contract by dataset.	57
3.4	Feature policy and leakage-control rules	61
3.5	Feature-lineage and encoding contract for the primary thesis datasets. . . .	62
3.6	Baseline and variant taxonomy	63
3.7	Methodological distinction between reference DICE and EDGRec.	65
3.8	Current EDGRec implementation contract used by the thesis experiments.	72
4.1	Metric roles used throughout the experimental setup.	85
4.2	Evidence categories for validation search and final reporting.	92
4.3	Compact hyperparameter and search space contract.	95
4.4	Feature-subset search design.	98
4.5	Primary experimental hardware and software environment.	107
4.6	Runtime and memory reporting contract.	108
5.1	Reader-facing structure of the empirical comparisons.	112
5.2	Feature-subset evidence and final test status.	114
5.3	Validation-to-test status for the main EDGRec evidence.	116
5.4	AmazonBook metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.	118
5.5	MovieLens-1M metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.	118
5.6	KuaiRec v2 sparse watch-ratio metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.	119
5.7	KuaiRand-1K diagnostic metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.	119
5.8	Reference DICE feasibility evaluations against completed EDGRec evaluations.	125
5.9	Matched KuaiRec v2 EDGRec ablations under an earlier protocol.	125

A.1	Appendix data-feature lineage and leakage audit summary.	144
A.2	Environment and reproducibility checklist.	145

List of Algorithms

1	EDGRec split-safe training and evaluation	76
2	Validation search to final-test protocol	94

Document Acronyms

This list defines the acronyms and abbreviated model names most frequently used in the thesis, ordered alphabetically:

AMP → Automatic Mixed Precision

API → Application Programming Interface

AvgPop → Average Popularity

BIOS → Basic Input/Output System

BPR → Bayesian Personalized Ranking

BPR-MF → Bayesian Personalized Ranking Matrix Factorization

CaDCR → Causal-augmented Disentanglement for Contrastive Recommendation

CaDSI → Causal Disentanglement for Semantic-Aware Intent Learning

CAGRA → CUDA-Accelerated Graph Construction and Approximate Nearest-Neighbor Search

CausE → Causal Embeddings for Recommendation

CDR → Causal Disentangled Recommendation against User Preference Shifts

CDSRec → Causal Disentangled Sequential Recommendation

CF → Collaborative Filtering

CLSR → Contrastive Learning for Long- and Short-term Recommendation

CPU → Central Processing Unit

CRRU → Composite Resource-aware Recommendation Utility

CSV → Comma-Separated Values

CUDA → Compute Unified Device Architecture

DAG → Directed Acyclic Graph

DCCL → Disentangled Causal Embedding with Contrastive Learning

DCG → Discounted Cumulative Gain

DDCE → Dual Disentanglement of User–Item Interaction for Recommendation with Causal Embedding

DDR5 → Double Data Rate 5

DICE → Disentangling User Interest and Conformity for Recommendation with Causal Embedding

DIMM → Dual In-line Memory Module

DirectAU → Direct Alignment and Uniformity

DisMIR → Disentangled Multi-Interest Representation Learning for Sequential Recommendation

ECC → Error-Correcting Code

EDGRec → Efficient Disentangled Graph Recommender

EXPO → Extended Profiles for Overclocking

GB → Gigabyte

GCN → Graph Convolutional Network

GDDR7 → Graphics Double Data Rate 7

GiB → Gibibyte

GNN → Graph Neural Network

GPU → Graphics Processing Unit

HIN → Heterogeneous Information Network

HitRatio → Hit Ratio

ID → Identifier

IDCG → Ideal Discounted Cumulative Gain

IDCL → Intent-aware Disentangled Graph Contrastive Learning

IID → Independent and Identically Distributed

IPS → Inverse Propensity Scoring

IPW → Inverse Propensity Weighting

KDR → Knowledge-Guided Disentangled Representation Learning for Recommender Systems

KG → Knowledge Graph

KuaiRand → Sequential recommendation dataset with randomly exposed videos

KuaiRand-1K → Compact 1K-user KuaiRand variant used for exposure-aware recommendation experiments

KuaiRec → Fully-observed Dataset for Evaluating Recommender Systems

LightGCN → Light Graph Convolutional Network

MB → Megabyte

MRF → Markov Random Field

MT/s → Mega-transfers per second

NDCG → Normalized Discounted Cumulative Gain

NVMe → Non-Volatile Memory Express

OOM → Out of Memory

PropCare → Propensity estimation for causality-oriented recommendation without exposure data

RAM → Random Access Memory

RQ → Research Question

SCM → Structural Causal Model

SMBIOS → System Management BIOS

SSD → Solid-State Drive

UMAP → Uniform Manifold Approximation and Projection

VAE → Variational Autoencoder

ValidationCRRU@20-@40 mean → Validation-split CRRU averaged over cutoffs 20 and 40 for model-selection use only

VRAM → Video Random-Access Memory

Chapter 1

Introduction

Bias-aware graph recommendation is a joint data, modelling, and evaluation problem. The data protocol determines which outcomes are observed, the representation design determines how collaborative and popularity-related signals are modelled, and the computational budget determines which comparisons are scientifically meaningful. This chapter motivates that position, defines the dissertation scope, and states the original contributions developed in the subsequent chapters.

1.1 Motivation and Problem Setting

Recommender systems support decision-making in settings where users face catalogues that are too large to inspect manually. Online shops, streaming platforms, news services, social platforms, learning systems, and digital libraries all depend on ranking mechanisms that select a small number of items from a much larger candidate space. In these settings, the objective is to produce a relevant top- K list of unseen items while respecting practical latency, memory, and scalability constraints [1, 2].

The most widely available evidence for learning such rankings is implicit feedback. Clicks, views, purchases, listens, ratings converted into binary events, saves, dwell time, and other behavioural traces can be collected at large scale and updated continuously. These observations are behavioural outcomes under a particular exposure and interface context. A recorded interaction indicates that a user encountered an item and responded; it does not reveal how the same user would have responded under another exposure policy, rank position, catalogue state, or social context.

The same issue applies to missing entries in a user–item matrix. A missing interaction may reflect non-exposure, low rank position, limited time, competition from another item, or genuine lack of interest. In implicit-feedback recommendation, the data are therefore positive-only or confidence-weighted observations rather than complete labels of preference and non-preference [3]. This distinction is central here because dataset construction, positive-label thresholds, train–validation–test splitting, candidate filtering, and negative

sampling determine which behavioural signals become learning evidence and which become evaluation evidence.

The dissertation frames recommendation as an efficiency-aware and bias-aware graph representation learning problem. Historical logs combine personal interest with exposure, item popularity, user conformity, and previous recommendation policies. Models trained on such logs may achieve favourable offline ranking while concentrating lists around already visible items or absorbing conformity-related behaviour into a single collaborative representation [4, 5]. The causal recommendation literature supplies the bias-aware motivation; the methodological question is how that motivation can guide a practical offline graph recommender whose empirical claims remain tied to observable ranking, list behaviour, diagnostics, and resource evidence.

The core technical direction is to adapt interest and conformity disentanglement into an efficient graph recommender. The proposed method is EDGRec, the Efficient Disentangled Graph Recommender. EDGRec uses a propagation backbone that follows the Light Graph Convolutional Network (LightGCN) formulation and maintains separate representation channels for interest-aligned and conformity-aligned signals while remaining practical to train and evaluate [5, 6]. Disentangling User Interest and Conformity for Recommendation with Causal Embedding (DICE) supplies the conceptual separation between interest and conformity; EDGRec contributes an operational graph-recommendation contract with sampled graph propagation, bounded branch losses, a separate item-context path, controlled score fusion, train-only context construction, and resource-aware evaluation. The distinction is methodological and empirical: EDGRec is not a direct DICE baseline, because it changes the propagation backbone, training regime, scoring path, resource contract, and evidence labels so that the branch hypothesis can be evaluated under the workstation and dataset protocols of this thesis. The contribution is therefore a practical framework for studying whether the disentanglement motivation can yield useful offline trade-offs under explicit hardware and data-protocol constraints.

The evaluation contribution is Composite Resource-aware Recommendation Utility (CRRU), a configurable post hoc framework for comparing models under ranking quality, popularity-aware list behaviour, and computational cost [2, 7]. CRRU is interpreted after its components are reported; it complements standard recommender metrics by making the declared resource-aware trade-off explicit without replacing the component evidence. This matters because a graph recommender can be scientifically valuable when it occupies a favourable accuracy–resource region under a declared protocol, provided the associated ranking, list behaviour, and memory costs remain visible rather than hidden by a scalar summary.

Efficiency is treated as evidence rather than as a secondary implementation detail. Under finite hardware, search, and replication budgets, a configuration that trains much faster can permit broader validation, more ablations, more repeated seeds, or more frequent retraining.

The scientific value of such a gain is not that speed replaces relevance, but that it changes what can be tested and reproduced under the same workstation budget. The claim is therefore conditional: resource gains support the argument only when the associated ranking quality, list behaviour, and memory costs are reported alongside them.

1.2 Theoretical Framework and Literature Fit

The framework combines three literature streams. Implicit-feedback recommendation and Bayesian Personalized Ranking (BPR) provide the ranking formulation: observed positives are compared with sampled unobserved items, but sampled contrasts are not verified dislikes [3]. Graph collaborative filtering and LightGCN supply the representation backbone, where user-item interactions form a bipartite graph and simplified normalized propagation can capture higher-order collaborative structure without unnecessary graph-convolutional complexity [6, 8, 9]. Causal and disentangled recommendation literature motivates the bias-aware design: observed interactions may mix exposure, popularity, conformity, and personal interest, so representation components are interpreted as model components and diagnostics rather than identified causal variables [4, 5, 10, 11].

The theoretical framework is layered. At the data layer, observations are treated as logged events generated under exposure and preprocessing constraints. At the representation layer, LightGCN propagation provides the efficient collaborative backbone, and Disentangling User Interest and Conformity for Recommendation with Causal Embedding (DICE) provides the main interest and conformity modelling reference. At the evaluation layer, ranking quality, popularity-aware list behaviour, runtime, and peak training video random-access memory (VRAM) are reported before any composite utility is applied. This framework positions EDGRec as a bias-aware graph representation method informed by causal recommendation, and CRRU as a resource-aware reporting utility.

This positioning defines a bounded scientific question: whether a resource-constrained graph recommender informed by disentanglement can produce useful ranking and resource trade-offs under declared observational data protocols. The empirical interpretation is therefore stated in terms of offline ranking, list behaviour, and resource use. Efficiency gains matter because recommender systems are developed under finite training and tuning budgets; their scientific value is strongest when the corresponding ranking and list behaviour evidence remains visible. Questions about treatment effects, policy value, or directly observed latent interest and conformity factors require explicit estimands, additional data, and identifying assumptions.

1.3 Scope and Original Contributions

The precise research questions are stated in Chapter 4, after the methodological and experimental contracts that make them answerable have been fixed. They are answered in Chapter 5, after the relevant empirical evidence has been reported. This ordering keeps the questions tied to their evaluation protocol and avoids treating exploratory, validation, feasibility, and final test evidence as interchangeable.

The study focuses on offline recommendation from observational implicit-feedback data, where randomized exposure is not available for all datasets and missing interactions are not reliable negative preferences. The causal recommendation literature motivates the problem of biased observational feedback; the contribution developed here is an efficient graph recommender informed by disentanglement and an evaluation protocol for recommendation under these observational constraints.

The dissertation makes four original contributions.

1. It formulates a protocol-centred framework for bias-aware recommendation experiments, covering exposure, missingness, positive-label construction, leakage-controlled feature eligibility, train-only graph construction, candidate filtering, and validation-only model selection. This protocol is necessary because the same model can receive different empirical meanings under different graph, split, feature, or candidate-set contracts.
2. It proposes EDGRec, an efficient graph recommender that adapts the DICE interest and conformity motivation to a LightGCN propagation setting with sampled training, an item-level context scoring path, controlled fusion of branch logits, and bounded auxiliary losses. EDGRec is presented as a practical, causal-recommendation-informed graph model rather than as an estimator of causal effects: its novelty lies in the integration of branch-structured modelling, split-safe graph construction, sampled training, score-scale control, context scoring, and resource-aware evaluation.
3. It provides an evidence-labelled evaluation that separates completed reference LightGCN comparisons, sampled-training comparators, validation search candidates, feature subset candidates, runtime feasibility evaluations, diagnostics, and matched ablations. This separation is essential because a resource feasibility result, a validation candidate, and a completed test comparison answer different scientific questions.
4. It formalizes and applies CRRU as a configurable resource-aware post hoc utility for comparing ranking quality, popularity-aware list behaviour, runtime, and VRAM while keeping the component metrics visible. CRRU is reusable as an aggregation framework because its indicators, transformations, and weights are declared explicitly; the weight vector used in this dissertation is the context-specific expression of the resource-aware objective evaluated here, not a universal definition of recommendation quality.

Together, these contributions connect bias-aware motivation, efficient graph recommendation, and resource-aware evaluation in a reproducible experimental workflow. Literature motivates the design, the implemented pipeline determines what was evaluated, validation evidence selects candidates, final test evidence supports empirical claims, and limitations delimit the conclusions.

1.4 Thesis Structure

Chapter 2 presents the literature review. It organizes prior work on implicit-feedback recommendation, dataset construction, graph collaborative filtering, exposure and causal motivation, disentangled recommendation, and resource-aware evaluation. The chapter is deliberately literature-focused: it motivates EDGRec and identifies the research gap without fully defining the thesis method or CRRU formula.

Chapter 3 defines the problem setting and methodology. It formalizes the implicit-feedback graph recommendation task, states the observational-bias interpretation scope, specifies the data and preprocessing protocol, records the feature-lineage and leakage-control contract, defines EDGRec, describes the training objective, and gives the evaluation protocol including the CRRU framework and the weight vector used in this thesis.

Chapter 4 presents the experimental setup. It states the experimental questions, dataset-specific configurations, hyperparameter-search and candidate-selection process, feature subset search design, baseline configurations, EDGRec configurations, ablation design, runtime and memory measurement protocol, and threats to validity.

Chapter 5 reports and discusses the empirical evidence. It preserves the evidence categories from Chapter 4, restates the data and preprocessing context needed for comparability, separates validation search and feature subset evidence from final test evidence, presents the completed test comparisons, interprets CRRU through its components, reports DICE feasibility evaluations and matched ablations, and then gives dataset-level conclusions.

Chapter 6 concludes the thesis by summarizing the contributions, synthesizing the main empirical implications, stating scientific and engineering limitations, and identifying future work.

Chapter 2

Literature Review

The literature review builds the theoretical framework by moving from general recommender-system foundations to the specific design choices used later. It situates recommender systems at the system level, reviews recommendation data, implicit-feedback objectives, and top- K evaluation, then connects those foundations to graph collaborative filtering and Light Graph Convolutional Network (LightGCN) propagation. The discussion then turns to the exposure, popularity, and conformity concerns raised by observational recommendation logs; the causal-embedding and disentangled-representation work that motivates interest and conformity separation; and the efficiency and resource-aware evaluation literature needed for the empirical protocol. The review is synthetic rather than sequential: it identifies the design pressures that EDGRec addresses, explains why those pressures belong together, and fixes the evidence boundaries under which the empirical results must be interpreted.

Modern recommender systems are usually organized as pipelines rather than as isolated scoring functions. Gao et al. summarize recommender systems in terms of stages, scenarios, objectives, and applications; the stage view includes matching, ranking, and re-ranking, each operating under different scale and latency constraints [9]. Figure 2.1 is introduced before the graph-specific material because it fixes this broader system context.

The pipeline view matters for the thesis because graph recommenders may supply candidate-generation or ranking representations, while the final exposed list can also be shaped by downstream re-ranking criteria and application-level objectives. The evaluation boundary used here is therefore deliberately narrower: EDGRec is assessed as a graph ranking model under a declared candidate protocol, and its conclusions concern that offline ranking setting.

2.1 Implicit-Feedback Recommendation

This section establishes the recommendation setting used throughout the thesis. It moves from collaborative filtering and feedback types to top- K ranking and pairwise learning, so later chapters can state preprocessing and evaluation assumptions precisely.

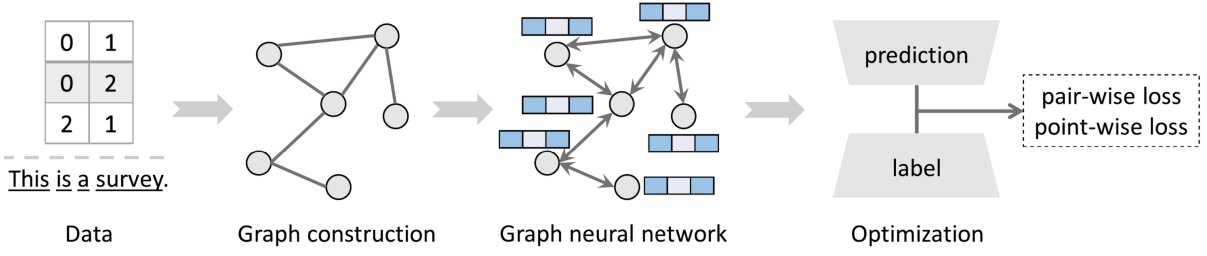


Figure 2.1: Typical recommender-system taxonomy and pipeline. The source figure summarizes recommender systems by stages, scenarios, objectives, and applications; the stage component highlights matching, ranking, and re-ranking as successive steps that reduce a large item pool to the final exposed list. In this thesis, the diagram situates graph collaborative filtering within a broader recommender-system workflow while preserving the narrower offline top- K evaluation boundary used for EDGRec. Source: Gao et al. [9].

2.1.1 Collaborative filtering

Collaborative filtering learns recommendation patterns from user–item interactions rather than relying only on manually specified item descriptions. This assumption is important for the thesis because EDGRec, LightGCN, and the reference baselines all begin from the same behavioural evidence: who interacted with which item under a declared protocol. Let $\mathcal{U}$ be the user set and $\mathcal{I}$ the item set. The observed data can be represented by an interaction matrix

$$R \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{I}|}, \quad (2.1)$$

where an entry R_{ui} records the available evidence for user u and item i . In implicit-feedback top- K recommendation, R is often binarised so that $R_{ui} = 1$ denotes an observed positive interaction and unobserved entries are treated as missing for data construction rather than as verified dislikes. For each user, the observed positive set is denoted

$$\mathcal{I}_u^+ = \{i \in \mathcal{I} : R_{ui} = 1\}. \quad (2.2)$$

This notation separates the empirical evidence used for learning from the generally unobserved notion of complete user preference [3].

The collaborative-filtering assumption is that interactions made by similar users, or interactions received by similar items, provide information for future recommendations. In matrix factorization this assumption is expressed through low-dimensional user and item factors; in graph recommendation the same evidence is encoded as a user–item graph over which embeddings are propagated. Surveys of graph neural networks for recommender systems identify this relational view as a central reason for applying graph representation learning to recommendation: it permits models to use direct interactions and higher-order collaborative structure, such as users connected through shared items and items connected through overlapping audiences [8, 9].

2.1.2 Explicit and implicit feedback

Recommendation data are commonly divided into explicit and implicit feedback. Explicit feedback is deliberately provided by users, for example through star ratings, reviews, or preference surveys. MovieLens is a prominent example of this setting: its records are user–movie rating–timestamp tuples, where ratings express stated preference on a five-star scale at a particular time [12]. Explicit feedback is therefore closer to a stated judgement, but it is typically sparse and affected by who chooses to provide ratings.

Implicit feedback is inferred from behaviour such as clicks, views, purchases, watch time, likes, or repeat consumption. Bayesian Personalized Ranking (BPR) formalizes the standard implicit-feedback problem as one in which only positive observations are available, while non-observed user–item pairs mix true negatives with missing values [3]. This distinction is the main data assumption used in the remainder of the thesis: observed interactions are usable evidence for learning, but missing entries require a modelling convention such as negative sampling, candidate filtering, or exposure-aware evaluation rather than a direct interpretation as dislike.

2.1.3 Top-K recommendation

Top- K recommendation asks a model to produce, for each user, a short ranked list of items rather than an exact reconstruction of all rating values. Let $K \in \mathbb{N}_{\geq 1}$ be the recommendation cutoff, let $\hat{y}_{ui} \in \mathbb{R}$ denote the predicted score for user $u \in \mathcal{U}$ and item $i \in \mathcal{I}$, and let $\mathcal{O}_{u,\text{train}} \subseteq \mathcal{I}$ denote items already observed for user u in the training split. In positive-only datasets, $\mathcal{O}_{u,\text{train}}$ may coincide with the training-positive set; in datasets that retain non-positive observations, it is larger. After filtering items that should not be recommended again, the model returns

$$\begin{aligned} \pi_u^K &= (\pi_{u,1}, \dots, \pi_{u,K}) \in \mathcal{I}^K, \\ \hat{y}_{u\pi_{u,1}} &\geq \hat{y}_{u\pi_{u,2}} \geq \dots \geq \hat{y}_{u\pi_{u,K}}, \end{aligned} \tag{2.3}$$

where π_u^K is the top- K recommendation list for user u . This formulation is more appropriate for implicit-feedback recommendation than rating-prediction objectives when the research question concerns whether held-out relevant items appear near the top of a finite recommendation list [1]. Figure 2.2 illustrates the corresponding implicit-feedback convention, sampled contrast construction, and top- K ranking pipeline. The key interpretation is that green entries are observed positives, gray entries are missing observations, and the red contrast item is sampled for learning rather than observed as a true dislike. The figure therefore visualizes the operational training and evaluation convention, not a complete preference matrix.

Offline top- K evaluation depends on a constructed split of observed interactions. Let $T_u \subseteq \mathcal{I}$ be the held-out positive set for user u , and let $\mathcal{U}_{\text{eval}} = \{u \in \mathcal{U} : |T_u| > 0\}$ be

Observed implicit-feedback matrix

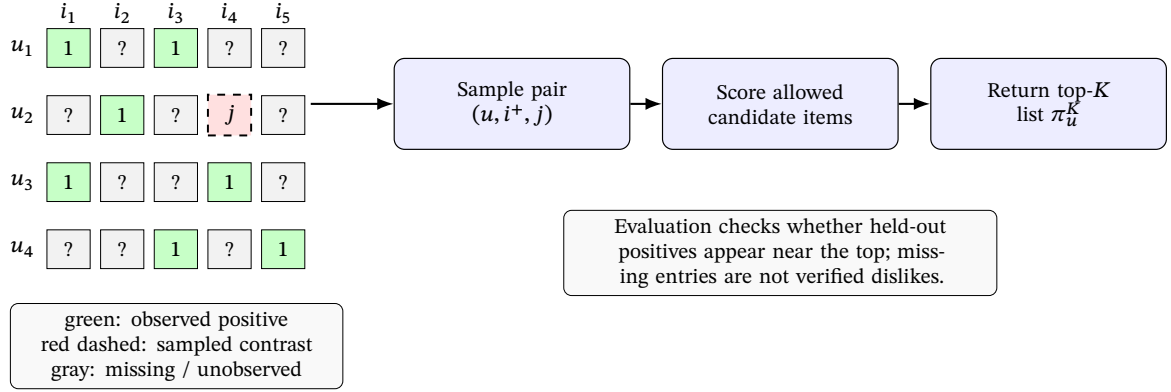


Figure 2.2: Observed implicit-feedback matrix and top- K pipeline. Observed positives provide positive evidence, missing entries are not verified dislikes, sampled contrast items support pairwise learning, and the final task ranks allowed candidates into a top- K list.

the evaluation-user set. Users with no held-out positives are excluded from the per-user average, because Recall@ K and Normalized Discounted Cumulative Gain (NDCG)@ K would otherwise have zero denominators. For $u \in \mathcal{U}_{\text{eval}}$ and rank index $r \in \{1, \dots, K\}$, define the binary relevance indicator $\text{rel}_{u,r} = \mathbb{I}(\pi_{u,r} \in T_u)$. Common per-user ranking measures are

$$\text{Recall@K}(u) = \frac{|\{\pi_{u,1}, \dots, \pi_{u,K}\} \cap T_u|}{|T_u|}, \quad (2.4a)$$

$$\text{HitRatio@K}(u) = \mathbb{I}(\{\pi_{u,1}, \dots, \pi_{u,K}\} \cap T_u \neq \emptyset), \quad (2.4b)$$

$$\text{DCG@K}(u) = \sum_{r=1}^K \frac{\text{rel}_{u,r}}{\log_2(r+1)}, \quad (2.4c)$$

$$\text{IDCG@K}(u) = \sum_{r=1}^{\min(K, |T_u|)} \frac{1}{\log_2(r+1)}, \quad (2.4d)$$

$$\text{NDCG@K}(u) = \frac{\text{DCG@K}(u)}{\text{IDCG@K}(u)}. \quad (2.4e)$$

If $|T_u| = 0$ were allowed, the convention would be $\text{IDCG@K}(u) = 0$ and NDCG@K would be undefined; the definition of $\mathcal{U}_{\text{eval}}$ above avoids this case. The recommendation-list definition assumes that at least K candidate items remain after filtering; otherwise K is replaced by the available candidate-set size. Dataset-level aggregation uses the arithmetic mean over evaluation users:

$$\text{Metric@K} = \frac{1}{|\mathcal{U}_{\text{eval}}|} \sum_{u \in \mathcal{U}_{\text{eval}}} \text{Metric@K}(u). \quad (2.5)$$

The per-user definitions in Equation (2.4) make the metric denominators and rank discounts explicit. Recall@ K measures coverage of held-out positives, HitRatio@ K records

whether at least one held-out positive appears in the list, and $\text{NDCG}@K$ adds positional discounting. Evaluation surveys emphasize that these metrics summarize different aspects of ranking quality and are therefore interpreted together with the candidate protocol and split construction used to compute them [1]. Here, the candidate protocol denotes the rule that determines which items may be scored for each user after excluding seen items and applying any declared item-universe restriction, where the item universe is the catalogue subset eligible for graph construction and evaluation in that experiment.

2.1.4 Ranking metrics and pairwise learning objectives

Evaluation criteria should be chosen to reflect the recommendation task and the assumptions behind the learning objective. Pointwise losses and rating-error metrics are appropriate when the task is numerical rating prediction, but implicit top- K recommendation is primarily an ordering problem. Pairwise learning objectives address this by training the model to rank observed positives above sampled alternatives, which aligns more directly with $\text{Recall}@K$ and $\text{NDCG}@K$ than reconstructing individual interaction values [1, 3].

Bayesian Personalized Ranking (BPR) is a foundational pairwise objective for implicit-feedback recommendation. It is used here because implicit-feedback top- K recommendation is a pairwise ranking problem, and BPR is the canonical objective derived as a maximum-a-posteriori estimator for personalized ranking, not because this thesis performs full Bayesian posterior inference. For a user $u \in \mathcal{U}$, an observed positive item $i \in \mathcal{I}_u^+$, and a sampled unobserved contrast item $j \in \mathcal{I} \setminus \mathcal{I}_u^+$, the model is trained so that $\hat{y}_{ui} > \hat{y}_{uj}$. The BPR training triples are

$$\mathcal{D}_{\text{BPR}} = \{(u, i, j) \mid u \in \mathcal{U}, i \in \mathcal{I}_u^+, j \in \mathcal{I} \setminus \mathcal{I}_u^+\}. \quad (2.6)$$

For a triple $(u, i, j) \in \mathcal{D}_{\text{BPR}}$, define the pairwise score margin

$$\Delta_{uij} = \hat{y}_{ui} - \hat{y}_{uj}. \quad (2.7)$$

Let Θ denote all trainable model parameters, let $\lambda \geq 0$ be the ℓ_2 regularization coefficient, and let $\lambda \|\Theta\|_2^2$ be the regularization term. A regularized minimization form of BPR is

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j) \in \mathcal{D}_{\text{BPR}}} \log \sigma(\Delta_{uij}) + \lambda \|\Theta\|_2^2. \quad (2.8)$$

Here, $\sigma(x) = 1/(1 + \exp(-x))$ is the logistic sigmoid. Since $0 < \sigma(\Delta_{uij}) \leq 1$, the log term satisfies $\log \sigma(\Delta_{uij}) \leq 0$. The leading minus sign in Equation (2.8) turns the maximization of the log-probability that the observed positive item outranks the sampled contrast item into a non-negative loss-minimization term.

BPR triples should still be interpreted operationally: j is a sampled unobserved contrast item, not a verified dislike [3]. For any branch-specific or popularity-conditioned objective

built on sampled comparisons, changing the sampler changes the empirical meaning of the training signal, even when the scoring architecture is unchanged. Recent analysis of collaborative-filtering representation geometry also shows that BPR affects both positive-pair alignment and embedding uniformity, so auxiliary geometry losses or branch-discrepancy diagnostics should be interpreted as representation-shaping choices rather than as independent causal validation [13].

The original BPR paper makes the pairwise-training convention explicit by contrasting the observed implicit-feedback matrix with user-specific item–item preferences [3]. Figure 2.3 shows this construction: if item i is observed for user u and item j is unobserved, BPR treats the triple (u, i, j) as evidence for $i >_u j$. No preference is inferred between two observed items or between two unobserved items. The figure is relevant to the thesis because EDGRec, LightGCN, and the branch objectives motivated by DICE all inherit the same caution: sampled unobserved items are training contrasts, while unobserved–unobserved pairs remain future ranking candidates rather than ground-truth dislikes.

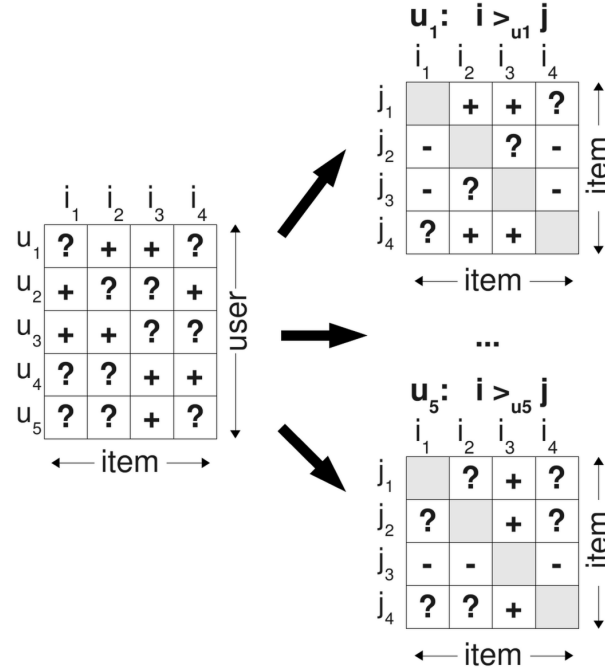


Figure 2.3: BPR pairwise preference construction. The left panel shows observed implicit feedback; the right panels show user-specific pairwise preferences, where plus means the observed item is preferred over an unobserved item and minus denotes the reverse ordered pair. Observed–observed and unobserved–unobserved item pairs remain unspecified. Source: Rendle et al. [3].

2.2 Dataset Construction and Observational Feedback

This section explains how raw recommender data become experimental evidence. It reviews dataset choice, interaction construction, positive labels, missingness, and feature leakage risks before the methods chapter turns them into a protocol.

2.2.1 Dataset selection

Dataset selection is a methodological decision because the catalogue, population, observation window, logging policy, feedback signal, and available metadata define the experimental question. A movie-rating dataset, a short-video exposure log, and a product-purchase graph can all be converted into user-item pairs, but they do not encode the same behavioural process. For an efficiency-aware graph recommender, the selected data must also be large and sparse enough for graph propagation to be meaningful while still permitting controlled experiments under realistic memory and runtime limits. Recommender-system surveys and surveys of graph neural networks for recommendation treat such data and protocol characteristics as essential for interpreting model comparisons [8, 9].

The benchmark datasets relevant to this thesis serve different roles. MovieLens is relevant as a well-documented explicit-rating benchmark whose tuples record expressed movie preferences and timestamps, making it valuable for diagnostic checks and rating-to-implicit conversion studies, but it does not represent the same feedback process as short-video logs or product purchases [12]. AmazonBook, as used in LightGCN experiments, is a much larger and sparser product-interaction benchmark with 52,643 users, 91,599 items, 2,984,108 interactions, and density 0.00062, making it more suitable for evaluating sparse graph propagation and embedding-table scale than MovieLens rating data [6].

KuaiRec and KuaiRand are especially important for the protocol-centred framing. KuaiRec contains a nearly fully observed small matrix of 1,411 users, 3,327 items, 4,676,570 interactions, and 99.6% density, together with a larger sparse matrix of 7,176 users, 10,728 items, 12,530,806 interactions, and 16.3% density [14]. The small matrix is valuable for sensitivity analysis and for discussing dense observation, but its density is atypical for sparse recommendation and should remain separate from the thesis’s primary sparse KuaiRec setting. The larger sparse matrix is closer to the graph conditions under which an efficient graph recommender is normally evaluated.

KuaiRand complements this perspective by providing randomized-exposure short-video logs, multiple feedback signals, and sequential records collected through random insertion into recommendation feeds [15]. It therefore provides exposure-diagnostic evidence, but it becomes a ranking benchmark only after the response label, temporal order, candidate universe, and randomized-exposure policy have been specified.

2.2.2 Interaction construction

Raw logs contain events rather than ready-made recommendation labels. An event may include a user, item, timestamp, action type, device context, watch duration, rating, exposure source, or session identifier. Interaction construction maps these raw records into R and $\mathcal{J}_u^+$. A click signal defines a target related to immediate attraction after exposure; a rating signal defines a target closer to expressed judgement among users who rate; a watch-time signal

emphasizes consumption depth; and a purchase signal represents conversion under price, availability, and intent constraints. These signals support different empirical questions and should not be treated as interchangeable solely because they can be represented as edges in a user–item graph [12, 15].

When multiple feedback types are available, the researcher must decide whether to use one signal, combine signals, or model them separately. KuaiRand, for example, records twelve feedback signals such as click, like, hate, and view time on randomly exposed videos, so a single binary edge definition necessarily discards part of the available behavioural structure [15]. Combining signals can improve coverage, but it can also blur the target: treating a brief click and a completed view as identical positives creates a different graph from one constructed from long views or likes only. The construction changes user and item degrees, which in turn changes graph propagation, negative sampling, candidate filtering, and top- K evaluation distributions.

2.2.3 Positive-label definition

Positive-label definition converts interaction evidence into training and evaluation labels. If s_{ui} is a constructed engagement score and $\tau \in \mathbb{R}$ is the threshold, a thresholded implicit label can be written as

$$R_{ui} = \mathbb{I}(s_{ui} \geq \tau). \quad (2.9)$$

The positive set $\mathcal{J}_u^+$ is then interpreted using the definition in the collaborative-filtering subsection: it collects exactly the items whose constructed label satisfies $R_{ui} = 1$.

In explicit-rating data, s_{ui} may be a star rating and τ may select ratings of four stars or higher. In short-video data, s_{ui} may use watch ratio, view duration, likes, or long-view indicators. In purchase data, the purchase event may itself define the positive label. The threshold therefore specifies what relevance means for the experiment, not merely how many records are retained [12, 15].

Thresholds create a coverage–precision trade-off. A strict threshold may reduce noisy positives but remove lower-intensity preference signals and produce a sparse graph concentrated around highly engaged users or highly visible items. A permissive threshold may increase coverage but include accidental clicks or brief consumption. Binarization also removes intensity information: two ratings or two view durations on different sides of the threshold can receive sharply different labels, while interactions above the threshold become identical. Because BPR and top- K metrics use $\mathcal{J}_u^+$ to determine both training triples and held-out relevance, the positive-label rule must be fixed before model comparison and applied consistently across train, validation, and test splits [1, 3].

2.2.4 Missingness and exposure

The distinction between implicit negatives and missing exposure must be resolved at the protocol level. In BPR, unobserved items are sampled as contrast items for optimization, but this does not assert that the user truly dislikes them [3]. KuaiRec shows why this matters for evaluation: its nearly fully observed small matrix was designed to analyse how data density and exposure conditions can change the ranking of methods under offline evaluation [14]. KuaiRand addresses the same issue from another direction by collecting randomized exposures inserted into recommendation feeds, which provides exposure evidence that ordinary logs typically lack while remaining a sequential short-video dataset with its own protocol constraints [15].

Splitting decisions determine which missingness assumptions enter the experiment and which information is allowed to reach the model. A random split evaluates interpolation over observed user histories, but it can mix events from the same period or session across train and test. A temporal split is closer to deployment chronology because the model is trained on earlier events and evaluated on later ones, but it may introduce distribution shift when users, items, or platform policies change. For graph recommenders, the propagation graph should be constructed from training positives only,

$$\mathcal{E}_{\text{train}} = \{(u, i) : i \in \mathcal{J}_{u, \text{train}}^+\}, \quad (2.10)$$

with validation and test positives held out for model selection and evaluation. Adding validation or test edges to the graph before propagation would leak future interactions into the embeddings and inflate top- K metrics.

2.2.5 Feature selection and leakage risks

Optional user and item features can add useful signal to recommendation experiments when they are available at recommendation time, but they also introduce leakage risks and do not necessarily improve ranking. Leakage occurs when preprocessing, feature engineering, graph construction, or candidate filtering uses information that would not have been available under the stated split. Examples include item popularity computed over the full dataset, cumulative likes or watch counts measured after the training period, labels derived from future behaviour, duplicate session events split across train and test, or normalization statistics fitted on validation and test interactions.

Post-treatment variables, meaning fields measured after exposure, recommendation, or the target response, are particularly risky in protocol-centred recommendation experiments. A feature that encodes how a previous recommender exposed an item, how much engagement an item accumulated after the recommendation time, or whether a user later consumed related content can make the model appear more accurate while using information unavailable for the intended prediction. KuaiRand’s rich user, item, history, and feedback fields

illustrate the value of such metadata, but also the need to distinguish pre-exposure attributes from post-exposure outcomes before constructing features [15].

Let $\mathbf{x}_i^{\text{pre}}$ denote item descriptors available before the prediction event, let $\mathbf{x}_i^{\text{engagement/post}}$ denote later exposure or engagement aggregates, and let $\mathbf{x}_i^{\text{model}}$ denote the item features passed to the model. A leakage-safe feature policy can therefore be expressed as

$$\mathbf{x}_i^{\text{model}} \subseteq \mathbf{x}_i^{\text{pre}}, \quad \mathbf{x}_i^{\text{engagement/post}} \cap \mathbf{x}_i^{\text{model}} = \emptyset, \quad (2.11)$$

Train-derived popularity, recency, and, where explicitly justified, propensity-calibration signals can still be informative, but they must be declared as context or calibration inputs rather than treated as semantic ground truth or verified exposure propensities. More generally, feature choice, item-universe definition, identifier remapping, and normalization affect both the graph seen by the model and the cost of training or evaluation [6, 8]. The literature-level implication is methodological: ranking, runtime, and VRAM results are meaningful only when compared under the same information boundary.

2.3 Graph Collaborative Filtering

This section introduces the graph view of collaborative filtering used by simplified graph recommenders. It proceeds from general message passing to bipartite user–item graphs, simplified propagation, and efficiency constraints, so the later EDGRec design can be read as a bounded adaptation rather than as an unrelated architecture.

2.3.1 Graph representation learning

Graph representation learning encodes nodes and edges into vectors that can be used for prediction, ranking, or retrieval. Let $G = (V, \mathcal{E})$ be a graph, $\mathcal{N}(v)$ the neighbours of node v , and $\mathbf{h}_v^{(k)} \in \mathbb{R}^d$ the representation of v after k propagation layers. In recommendation, the relevant graph is often not an arbitrary social or citation graph, but a user–item interaction structure whose edges are produced by the preprocessing decisions described above. GNN recommender surveys emphasize that such graphs are relevant because propagation can exploit both immediate interactions and multi-hop collaborative structure [8, 9].

For this thesis, graph representation learning provides the notation for user and item embeddings used in top- K scoring, explains how graph collaborative filtering can use higher-order signals without manually constructing all user–user or item–item similarities, and exposes the computational cost of propagating embeddings through a sparse but potentially large interaction graph. The section therefore focuses on message passing, normalized aggregation, and training cost rather than on the full taxonomy of GNN architectures.

The graph view is broader than a single user–item adjacency. The survey by Wu et al. shows representative recommender-system graph structures: user–item interactions can

be represented as a bipartite graph, sequential behaviour can be transformed into item-transition graphs, and side information can naturally appear as social or knowledge graphs [8]. Figure 2.4 is included here to anchor the terminology before the thesis narrows to bipartite collaborative filtering. It demonstrates why graph learning is a natural abstraction for recommendation data, while also clarifying that EDGRec uses only the train-derived user–item interaction graph unless later chapters explicitly enable side information.

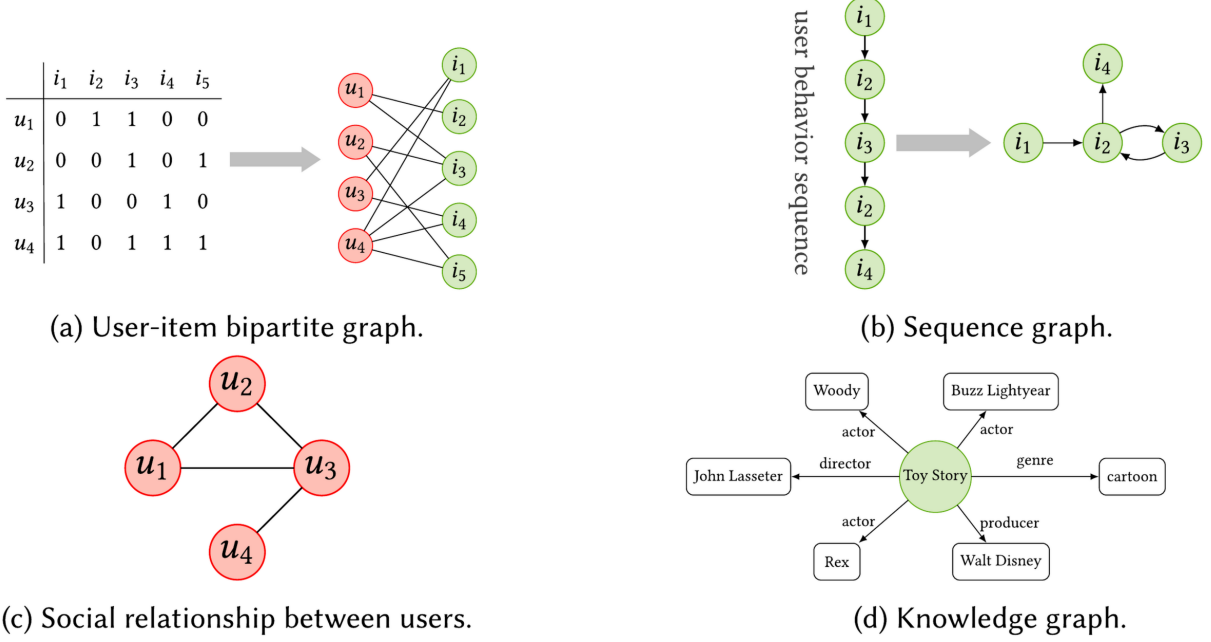


Figure 2.4: Representative graph structures in recommender systems. Panel (a) shows a bipartite user–item interaction graph, where edges encode observed interactions between users and items. Panel (b) shows a sequence-derived item graph, where ordered behaviour is represented through item–item transition edges. Panel (c) shows a social graph over users, where edges represent user–user relations that can serve as side information. Panel (d) shows a knowledge graph, where items are connected to related entities or attributes through structured relations. The thesis uses this taxonomy as background before focusing on train-derived bipartite collaborative-filtering graphs. Source: Wu et al. [8].

2.3.2 Message passing and neighborhood aggregation

Message passing is the core abstraction used by many graph neural networks. For node $v \in V$, neighbour set $\mathcal{N}(v) \subseteq V$, layer index $k \in \mathbb{N}_{\geq 0}$, and node embeddings $\mathbf{h}_v^{(k)}, \mathbf{m}_v^{(k)} \in \mathbb{R}^{d_k}$, each layer aggregates neighbouring representations and updates the target node representation:

$$\mathbf{m}_v^{(k)} = \text{AGG}^{(k)}(\{\mathbf{h}_n^{(k)} : n \in \mathcal{N}(v)\}), \quad \mathbf{h}_v^{(k+1)} = \text{UPDATE}^{(k)}(\mathbf{h}_v^{(k)}, \mathbf{m}_v^{(k)}). \quad (2.12)$$

The aggregation function is usually permutation invariant, such as a sum, mean, or attention-weighted combination, because graph neighbours have no canonical ordering. Stacking layers lets information travel across multiple hops: in a user–item graph, one

hop connects a user to consumed items, two hops connect the user to other users through those items, and three hops return to items related through overlapping audiences. GNN recommender surveys identify this higher-order connectivity as a key motivation for graph recommendation, while also noting challenges in graph construction, aggregation design, optimization, and computational efficiency [8, 9].

For user–item collaborative filtering, the abstract message-passing definition becomes a concrete pipeline: construct a graph from interactions, propagate information over the graph, and map the resulting user and item representations to preference scores. Figure 2.5 shows this user–item collaborative-filtering framework from the GNN recommender survey [8]. Its role in the thesis is to connect the generic aggregation equation to the recommender-specific setting in which graph construction, neighbour aggregation, representation update, final layer combination, sampling, prediction, and optimization all affect the final top- K evidence.

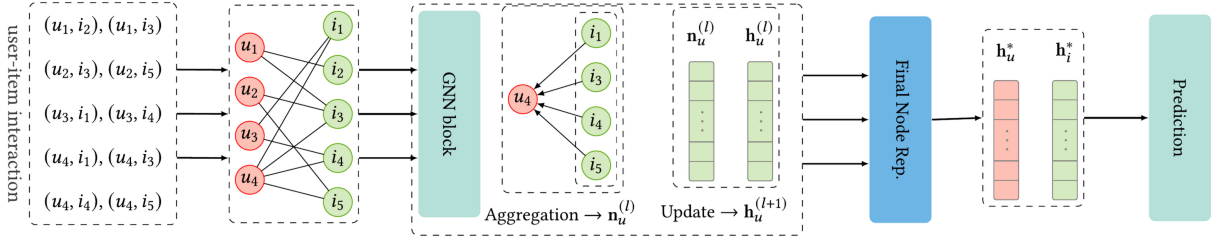


Figure 2.5: GNN framework for user–item collaborative filtering. The source figure depicts the recommendation-specific pipeline in which user–item interactions define the graph, GNN propagation learns user and item representations from higher-order connectivity, and the learned embeddings support preference prediction. It also motivates the design choices of graph construction, neighbour aggregation, update rules, and final representation combination. Source: Wu et al. [8].

2.3.3 Graph convolutional networks

Graph convolutional networks instantiate message passing through normalized neighbourhood aggregation. Let V be a finite node set and let $\mathbf{A} \in \mathbb{R}^{|V| \times |V|}$ be the adjacency matrix of a generic graph. A common GCN formulation first adds self-loops and constructs the corresponding degree matrix as

$$\begin{aligned} \tilde{\mathbf{A}} &= \mathbf{A} + \mathbf{I}_{|V|}, \\ \tilde{\mathbf{D}}_{ii} &= \sum_{j=1}^{|V|} \tilde{\mathbf{A}}_{ij}. \end{aligned} \tag{2.13}$$

Here, $\mathbf{I}_{|V|}$ is the identity matrix. The matrix $\tilde{\mathbf{D}} \in \mathbb{R}^{|V| \times |V|}$ is diagonal; $\tilde{\mathbf{D}}_{ii}$ is the degree of node i in the self-loop-augmented graph, and all off-diagonal entries of $\tilde{\mathbf{D}}$ are zero. For layer ℓ , let $\mathbf{H}^{(\ell)} \in \mathbb{R}^{|V| \times d_\ell}$ stack node representations, let $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell+1}}$ be a trainable transformation, and let σ be an element-wise nonlinearity. The resulting normalized graph-convolution layer is

$$\mathbf{H}^{(\ell+1)} = \sigma(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)}), \quad (2.14)$$

Equations (2.13)–(2.14) are included as the generic GCN literature formulation, not as the exact propagation layer used by LightGCN or by EDGRec. The normalization controls the scale of aggregated messages so that high-degree nodes do not contribute solely in proportion to their raw neighbour count. In recommender systems, this normalized propagation idea is more important for the thesis than the general GCN architecture: LightGCN is a recommendation-specific simplification that removes $\mathbf{W}^{(\ell)}$ and σ for collaborative filtering while retaining degree-normalized aggregation over the user–item graph [6, 9].

2.3.4 User-item bipartite graphs

Graph recommendation begins by converting the interaction matrix into a bipartite graph

$$\mathcal{G} = (\mathcal{U} \cup \mathcal{I}, \mathcal{E}), \quad \mathcal{E} = \{(u, i) : u \in \mathcal{U}, i \in \mathcal{I}_u^+\}. \quad (2.15)$$

Edges connect users only to items, so the neighbourhoods are

$$\mathcal{N}_u = \{i \in \mathcal{I} : (u, i) \in \mathcal{E}\}, \quad \mathcal{N}_i = \{u \in \mathcal{U} : (u, i) \in \mathcal{E}\}. \quad (2.16)$$

This bipartite structure is the graph analogue of collaborative filtering: users become related through shared items, and items become related through shared users. GNN recommender surveys describe this interaction graph as the basic structure that enables higher-order collaborative signals in graph recommendation [8, 9].

In matrix form, the bipartite adjacency $\mathbf{A} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|) \times (|\mathcal{U}|+|\mathcal{I}|)}$ corresponding to R is

$$\mathbf{A} = \begin{bmatrix} 0 & R \\ R^\top & 0 \end{bmatrix}. \quad (2.17)$$

For experiments, $\mathbf{A}$ should be constructed from the training interaction matrix, not from the full dataset. This convention keeps graph propagation aligned with the top- K protocol: held-out positives are targets for evaluation, not edges available to the model during representation learning. Figure 2.6 summarizes this train-only graph view and the LightGCN propagation pipeline used as literature background.

2.3.5 LightGCN and simplified graph propagation

LightGCN is the graph collaborative-filtering backbone most directly relevant to this thesis. Figure 2.6 shows the corresponding minimal pipeline: layer-0 user and item embeddings are propagated over the user–item graph, combined across layers, and scored by inner product. LightGCN was proposed after ablation analyses found that feature transformations and non-

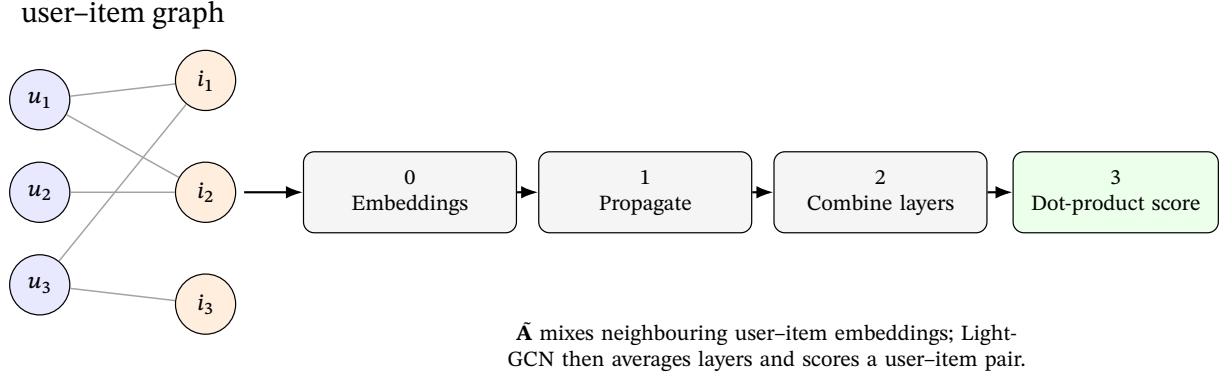


Figure 2.6: User-item bipartite graph and LightGCN propagation. The figure shows the literature-level LightGCN computation from layer-0 embeddings through normalized propagation, layer combination, and an inner-product recommendation score; it does not depict the additional branches, context path, or score fusion used by EDGRec.

linear activations added difficulty and did not improve collaborative-filtering performance in the studied GCN recommender setting. LightGCN therefore keeps the essential operation: normalized neighbourhood aggregation over the user-item interaction graph [6].

Let $\mathbf{A}$ be the bipartite adjacency over $\mathcal{U} \cup \mathcal{I}$ without self-loops, and let $\mathbf{D}$ be its diagonal degree matrix with $D_{vv} = \sum_q A_{vq}$. In this subsection, the LightGCN normalized adjacency is

$$\tilde{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}, \quad (2.18)$$

where $\tilde{\mathbf{A}}$ denotes the symmetrically normalized bipartite adjacency rather than the self-loop-augmented generic GCN adjacency in Equation (2.13). Thus the initial embedding $\mathbf{e}^{(0)}$ is included later through layer combination, not through a self-loop in the LightGCN adjacency. For layer $\ell \in \{0, \dots, L-1\}$, with user embedding $\mathbf{e}_u^{(\ell)} \in \mathbb{R}^d$ and item embedding $\mathbf{e}_i^{(\ell)} \in \mathbb{R}^d$, node-wise LightGCN propagation updates user and item embeddings by

$$\mathbf{e}_u^{(\ell+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} \mathbf{e}_i^{(\ell)}, \quad (2.19a)$$

$$\mathbf{e}_i^{(\ell+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i| |\mathcal{N}_u|}} \mathbf{e}_u^{(\ell)}. \quad (2.19b)$$

If $\mathbf{E}^{(0)}$ stacks the trainable initial user and item embeddings, the corresponding matrix propagation is

$$\mathbf{E}^{(\ell+1)} = \tilde{\mathbf{A}} \mathbf{E}^{(\ell)}. \quad (2.20)$$

LightGCN then combines several propagation depths into final user and item embeddings. The LightGCN paper allows α_ℓ to be treated as manually tuned hyperparameters or learnable

model parameters, but reports that the uniform setting $\alpha_\ell = 1/(L + 1)$ works well and keeps the model simple [6]. With $\alpha_\ell \geq 0$ and $\sum_{\ell=0}^L \alpha_\ell = 1$, the layer combination is

$$\mathbf{e}_u = \sum_{\ell=0}^L \alpha_\ell \mathbf{e}_u^{(\ell)}, \quad \mathbf{e}_i = \sum_{\ell=0}^L \alpha_\ell \mathbf{e}_i^{(\ell)}. \quad (2.21)$$

The ranking score is the inner product of the final representations,

$$\hat{y}_{ui} = \mathbf{e}_u^\top \mathbf{e}_i. \quad (2.22)$$

Figure 2.7 shows the original LightGCN architecture from He et al. and complements the author-created pipeline in Figure 2.6. The paper figure is retained because it is scientifically important in its original form: it explicitly marks Light Graph Convolution as normalized neighbour summation, shows that self-connections, feature transformations, and nonlinear activations are removed, and separates layer combination from final prediction [6]. This level of architectural detail is useful for interpreting why EDGRec adopts a propagation backbone derived from LightGCN rather than a heavier generic GCN.

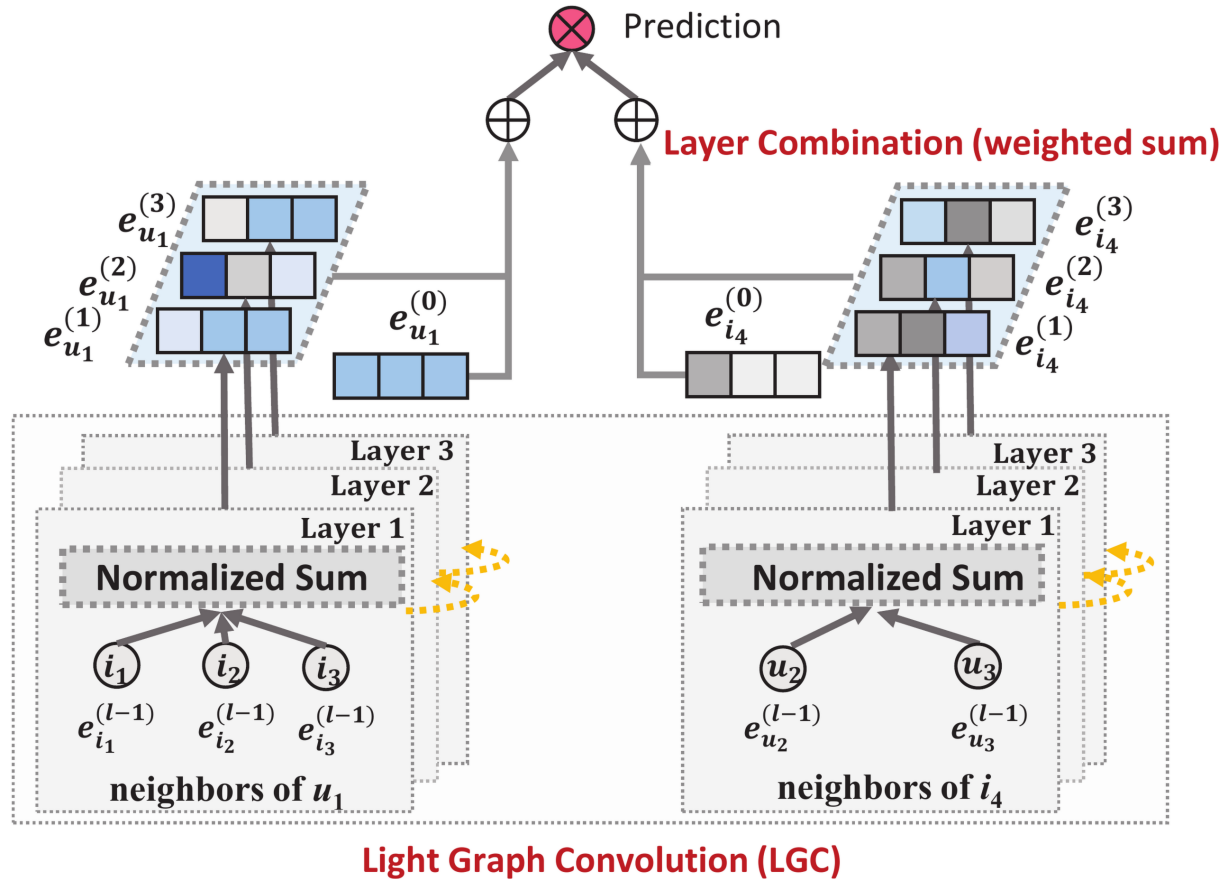


Figure 2.7: Original LightGCN architecture. The source figure distinguishes Light Graph Convolution, layer combination, and prediction. Light Graph Convolution performs only normalized summation of neighbour embeddings toward the next layer, while self-connections inside propagation, feature transformations, and nonlinear activations are removed; layer combination then sums embeddings from multiple layers to form final representations. Source: He et al. [6].

Equations (2.19) and (2.21) show the two LightGCN operations that matter for this thesis: normalized propagation and layer combination. LightGCN removes feature transformations and nonlinear activations from GCN recommendation models, but it keeps normalized neighbourhood aggregation and layer combination. This makes it the main graph-recommendation mathematical baseline for the thesis because it isolates collaborative graph propagation with a minimal computational core [6]. The thesis implication is motivational rather than definitional: a LightGCN propagation backbone provides a simple substrate for bounded branch-structured recommendation models.

2.3.6 Computational cost of graph learning

The computational cost of graph learning depends on the number of nodes, number of edges, embedding dimension, propagation depth, and training strategy. The notation $\mathcal{O}(\cdot)$ is used in the standard asymptotic sense: for non-negative cost functions g and h , writing $g = \mathcal{O}(h)$ means that there are constants $c > 0$ and n_0 such that $g(n) \leq ch(n)$ for all sufficiently large problem sizes $n \geq n_0$. In the multivariate expressions below, the notation suppresses implementation-dependent constants and lower-order terms while showing how the leading upper bound scales with graph size, embedding dimension, depth, and fan-out. For example, storing each undirected bipartite edge as two directed adjacency entries changes only a constant factor and is therefore hidden by the $\mathcal{O}(\cdot)$ notation.

A full-neighbourhood sparse propagation layer over a graph with edge set $\mathcal{E}$ and embedding dimension d applies one sparse adjacency–embedding multiplication. Because each nonzero adjacency entry contributes to a d -dimensional message, the arithmetic cost of one propagation layer is $\mathcal{O}(|\mathcal{E}|d)$ under the usual sparse-matrix model. LightGCN uses this form of degree-normalized embedding propagation while removing feature transformations and nonlinearities from the generic GCN layer [6]. For $L \in \mathbb{N}_{\geq 1}$ propagation layers over the training graph $\mathcal{E}_{\text{train}}$, the propagation-only cost can therefore be written as the cost function

$$T_{\text{prop}}^{\text{full}}(L, |\mathcal{E}_{\text{train}}|, d) = \mathcal{O}(L|\mathcal{E}_{\text{train}}|d), \quad (2.23)$$

before sampling, scoring, loss, and optimizer costs. Equation (2.23) should therefore be interpreted as a literature-standard sparse propagation bound, not as an exact wall-clock-time formula. The persistent embedding table alone requires $\mathcal{O}(|V|d)$ parameters, and training can require additional memory for sparse adjacency storage, intermediate layer representations, optimizer states, sampled triples, and evaluation scores.

Mini-batch and sampled training reduce the amount of graph expanded per update by computing representations for seed nodes and sampled neighbourhoods. Web-scale graph recommendation methods such as PinSage use sampled neighbourhood expansion to make graph convolution feasible at industrial catalogue scale [16]. If a mini-batch has $B \in \mathbb{N}_{\geq 1}$ seed nodes and layer fan-outs $f_1, \dots, f_L \in \mathbb{N}_{\geq 1}$, then depth ℓ can introduce at most $B \prod_{r=1}^{\ell} f_r$

sampled nodes before duplicate removal. Summing over depths gives the total sampled-node upper bound

$$N_{\text{sample}}^{\text{total}}(B, f_1, \dots, f_L) = \mathcal{O}\left(B \sum_{\ell=0}^L \prod_{r=1}^{\ell} f_r\right), \quad (2.24)$$

where the empty product for $\ell = 0$ equals 1 and accounts for the B seed nodes themselves. The deepest frontier alone has the upper bound

$$N_{\text{sample}}^{(L)}(B, f_1, \dots, f_L) = \mathcal{O}\left(B \prod_{r=1}^L f_r\right). \quad (2.25)$$

Equations (2.24)–(2.25) are worst-case combinatorial bounds: real mini-batches may be smaller after duplicate removal, caching, degree limits, or method-specific sampling choices. A recent full-graph versus mini-batch analysis further argues that batch size and fan-out have distinct effects on convergence, generalization, and computational efficiency, so the choice between full-graph and sampled training should be reported as part of the experimental method rather than hidden behind ranking metrics alone [17].

2.3.7 Full-graph and sampled graph training

Full-graph propagation with LightGCN applies $\tilde{\mathbf{A}}_{\text{train}}$ to all user and item embeddings at each layer. Its advantage is methodological clarity: every node aggregates from the exact observed training graph under the selected preprocessing rules. Its limitation is resource growth with $|\mathcal{U}|$, $|\mathcal{J}|$, $|\mathcal{E}_{\text{train}}|$, embedding dimension, and propagation depth.

Sampled graph training approximates this computation by expanding only sampled neighbourhoods for a mini-batch of seed nodes or interaction triples. PinSage demonstrates why this design is necessary for web-scale graph recommendation: it uses sampled localized convolutions and minibatch construction to avoid operating on the entire graph during training [16]. The approximation is not neutral. Fan-out determines how much neighbourhood information is retained, batch size changes memory pressure and gradient estimation, and the interaction between sampling and graph degree can alter which users and items dominate training. The full-graph versus mini-batch analysis therefore motivates treating batch size and fan-out as methodological choices that affect both resource use and learning behaviour [17].

2.3.8 Graph recommendation efficiency challenges

Even simplified graph recommendation has nontrivial memory and runtime cost. The model stores user and item embedding tables; sparse adjacency data; propagated embeddings for one or more layers; optimizer states; sampled BPR triples; and candidate scores for validation or testing. Let $\mathbf{E}_{\mathcal{U}_{\text{eval}}} \in \mathbb{R}^{|\mathcal{U}_{\text{eval}}| \times d}$ be the final embeddings of evaluation users and let $\mathbf{E}_{\mathcal{J}} \in \mathbb{R}^{|\mathcal{J}| \times d}$

be the final item-embedding matrix. Full all-item evaluation can be expressed as

$$\hat{\mathbf{Y}}_{\text{eval}} = \mathbf{E}_{\mathcal{U}_{\text{eval}}} \mathbf{E}_{\mathcal{J}}^{\top}, \quad (2.26)$$

where $\hat{\mathbf{Y}}_{\text{eval}} \in \mathbb{R}^{|\mathcal{U}_{\text{eval}}| \times |\mathcal{J}|}$ is the dense score matrix. If all evaluation users are scored against all catalogue items exhaustively, dense dot-product scoring has arithmetic complexity

$$\Theta(|\mathcal{U}_{\text{eval}}||\mathcal{J}|d) \quad (2.27)$$

before masking consumed items and selecting top- K . If the full score matrix is materialized, its memory is

$$\Theta(|\mathcal{U}_{\text{eval}}||\mathcal{J}|). \quad (2.28)$$

Chunked evaluation changes peak VRAM but not the total number of exhaustive dot products. Exhaustive full-catalog scoring can be expensive in large catalogues because validation or test users may need scores against every candidate item. This cost is separate from the cost of representation learning. In a deployed recommender, representation learning, candidate retrieval, and final ranking are distinct stages: graph propagation learns embeddings, a retrieval system may select a manageable candidate set, and a ranking stage scores and orders those candidates under the chosen objective. The methods chapter must therefore state whether an experiment evaluates graph representation learning alone, full-catalog ranking, candidate retrieval, or a complete retrieval-and-ranking pipeline.

Efficiency choices are therefore methodological choices, not only engineering details. Graph construction determines which edges can propagate information; propagation depth determines how much higher-order structure is mixed; negative sampling determines which comparisons BPR sees; batch size and fan-out determine the resource conditions under which the model is trained; and candidate protocols determine the empirical meaning and cost of Recall@ K , HitRatio@ K , and NDCG@ K . The GNN recommendation literature highlights both the representational value of graph methods and the practical challenges of graph construction, optimization, and computation efficiency [8, 9].

This bridge motivates an efficient disentangled graph recommender rather than a more complex generic GNN. The relevant baseline is collaborative propagation with LightGCN, because it provides effective graph representation learning with a minimal computational core. The design pressure is therefore selective: interest and conformity components should add interpretable modelling capacity only where they preserve sparse graph training, top- K evaluation, and resource-aware comparison. Architectural complexity is justified only when its empirical role is visible in ranking, list behaviour, diagnostics, or resource measurements.

2.4 Bias and Causal Motivation in Recommendation

The causal literature is used here for three distinct purposes. Exposure bias and missing-not-at-random observation explain why ordinary implicit-feedback logs are incomplete evidence of preference; popularity and conformity motivate representation-learning structure; causal effect estimation, in the stricter sense, requires treatment or exposure definitions together with identifying assumptions such as valid propensities, positivity, and control of confounding [4, 10, 11]. EDGRec draws on the first two roles as representation motivation and treats the third as an interpretation boundary. The result is a bias-aware modelling framework, not an identified causal estimator: the thesis reports offline ranking, list behaviour, resource, and diagnostic evidence rather than treatment effects or counterfactual policy value.

2.4.1 Exposure bias

The central difficulty in observational recommendation logs is that relevance and exposure are not observed independently. Let $Z_{ui} \in \{0, 1\}$ indicate whether user-item pair (u, i) is exposed, recommended, or otherwise observable in the log. Let $Y_{ui}^{(1)} \in \{0, 1\}$ denote the response that would be recorded under exposure, and let $\tilde{Y}_{ui} \in \{0, 1\}$ denote the value stored in a logged implicit-feedback matrix. If one stores an unexposed pair as zero in that matrix, this storage convention can be written as

$$\tilde{Y}_{ui} = Z_{ui} Y_{ui}^{(1)}. \quad (2.29)$$

The zero in Equation (2.29) then means “not observed in the log”, not necessarily “irrelevant”. This storage notation is distinct from causal effect notation such as $Y_{ui}(1) - Y_{ui}(0)$, where $Y_{ui}(1)$ and $Y_{ui}(0)$ are potential outcomes under treatment and non-treatment. The storage equation fixes the observed-data contract: a missing entry may reflect non-exposure rather than non-relevance, and observed positives can be systematically shaped by the previous interface, ranking policy, or candidate-generation process. Causal recommendation surveys treat this exposure mechanism as a major source of bias in offline learning and evaluation [4, 10, 11].

Dataset design can partially illuminate exposure bias when its observation process is stated explicitly. KuaiRec provides a nearly fully observed small matrix for analysing how density and exposure conditions affect offline evaluation, while KuaiRand records randomized short-video exposures inserted into feeds and is therefore valuable for studying exposure-sensitive evaluation more directly than ordinary observational logs [14, 15]. These datasets motivate the thesis’s protocol-centred stance: graph construction and evaluation should state what is observed, what is held out, and what is treated as missing.

Figure 2.8 summarizes the conceptual bias structure behind that stance. The dashed and solid links should be read as motivation for exposure-aware protocol design: popularity can

affect exposure, exposure gates which events are logged, and conformity or latent preference can both contribute to an observed interaction.

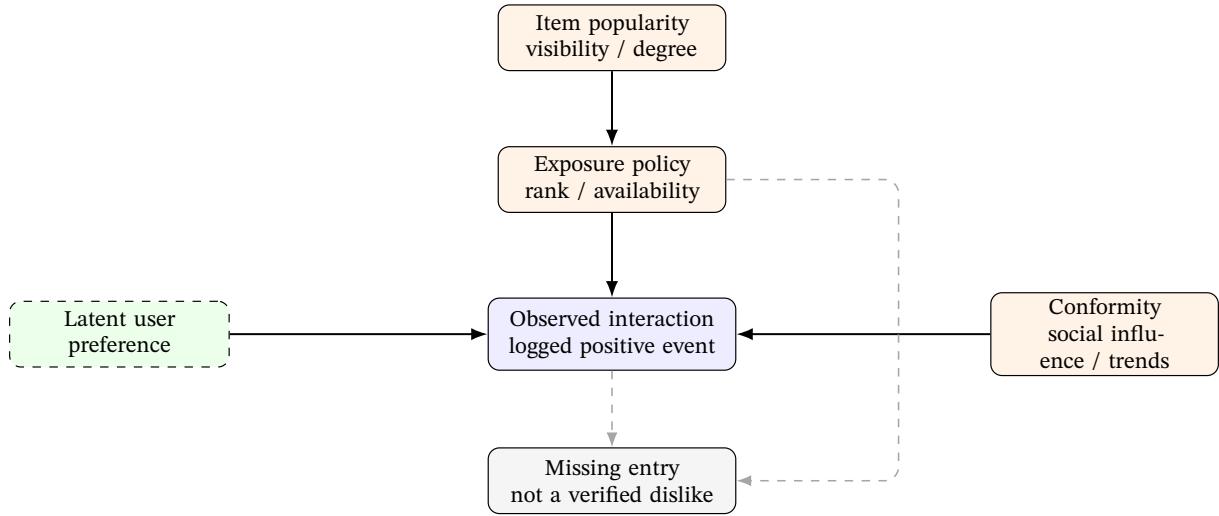


Figure 2.8: Observational-bias directed acyclic graph (DAG) motif. The diagram motivates exposure-, popularity-, and conformity-aware representation learning by showing how visibility, exposure, preference, and conformity can jointly shape logged interactions. It is a conceptual protocol diagram rather than an identified estimand graph.

2.4.2 Popularity bias and feedback loops

Popularity bias arises when highly visible items receive disproportionate exposure and therefore accumulate more interactions, which can make them appear more relevant than less exposed items under observational training. Causal-inference surveys describe popularity bias as closely related to exposure processes: if popular items are more likely to be shown, then item popularity becomes entangled with both observation and response [4, 10]. In graph recommendation this effect is also structural, because popular items have high degree and can influence many user embeddings through neighbourhood aggregation.

Feedback loops make the problem dynamic. A recommender that ranks popular items highly can increase their exposure, which can produce more logged interactions and further increase their apparent relevance in the next training cycle. This loop does not imply that popularity is never informative: popular items can be genuinely relevant. The methodological problem is that offline logs usually do not reveal how much of an observed interaction is due to an interest-related preference signal, how much is due to prior visibility, and how much is due to the recommender’s earlier policy. This motivates bias-aware analysis without implying that the present thesis identifies causal popularity effects.

DICE uses this feedback-loop view to motivate why training and serving distributions in recommender systems may be non-IID: users interact according to their intentions, those interactions train the model, the model changes recommendations, and the new recommendations influence later interaction data [5]. Figure 2.9 is included as a compact illustration of

that dynamic process from the DICE study. Its relevance to the thesis is methodological: popularity and conformity are not merely static item attributes, but can be reinforced by repeated data-collection and recommendation cycles, which motivates reporting popularity-sensitive behaviour alongside ranking metrics.

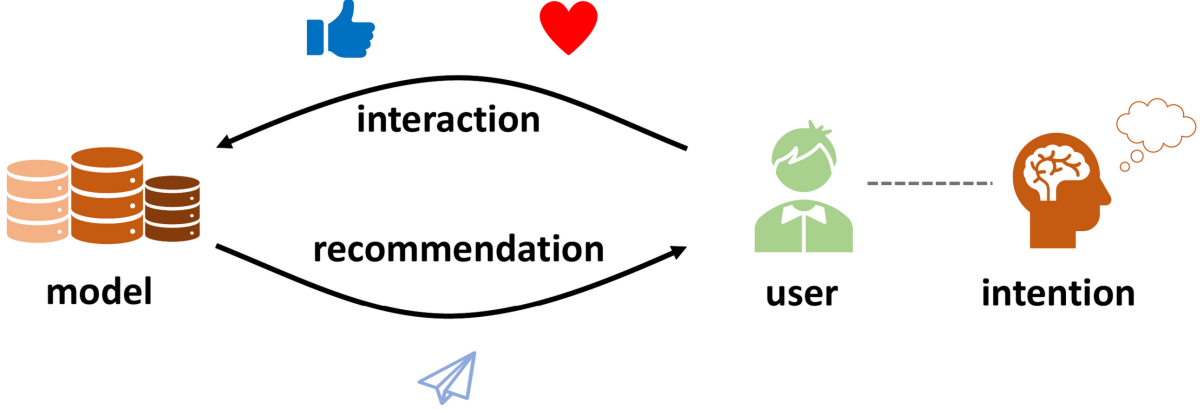


Figure 2.9: Feedback loop of recommender systems in DICE. The source figure shows users interacting with the model according to user intention, interaction data training the model, and model recommendations shaping later interactions. It motivates robustness and popularity/conformity analysis under non-IID feedback. Source: Zheng et al. [5].

2.4.3 Conformity

Conformity describes interaction behaviour driven by social influence, trend-following, or the visible popularity of an item rather than by stable personal interest alone. DICE motivates this distinction by arguing that observational user–item interactions may arise from at least two modelled causes: user interest in the item and user conformity towards popular items. If these causes are bundled into a single representation, the resulting model may be less robust and less interpretable when the underlying popularity or exposure conditions change [5].

The DICE study expresses this motivation through a conceptual structural causal model. For user $u \in \mathcal{U}$ and item $i \in \mathcal{I}$, let X_{ui}^{int} denote the interest cause, X_{ui}^{con} the conformity cause, Y_{ui}^{click} the click outcome, and let N^{int} , N^{con} , and N^{click} be independent exogenous noises. DICE writes the conceptual SCM as follows [5]:

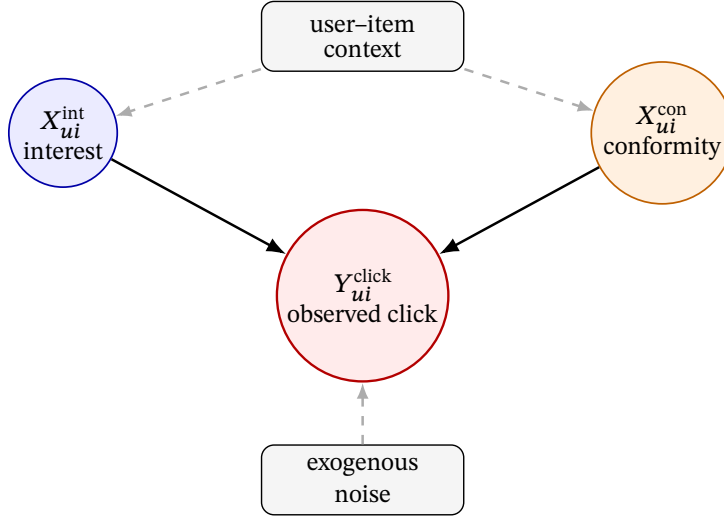
$$X_{ui}^{\text{int}} := f_1(u, i, N^{\text{int}}), \quad (2.30a)$$

$$X_{ui}^{\text{con}} := f_2(u, i, N^{\text{con}}), \quad (2.30b)$$

$$Y_{ui}^{\text{click}} := f_3(X_{ui}^{\text{int}}, X_{ui}^{\text{con}}, N^{\text{click}}). \quad (2.30c)$$

Figure 2.10 visualizes the compact motif in Equation (2.30). It represents the DICE decomposition of observed clicks as motivation for separating interest and conformity, and it is kept in the literature chapter as conceptual background rather than as an EDGRec method

diagram.



Motif only: latent causes are not directly observed or identified by the thesis experiments.

Figure 2.10: SCM motif for interest and conformity. The diagram summarizes the DICE conceptual decomposition in Equation (2.30): an observed click may be compatible with interest and conformity components plus noise. The motif is used here as representation-design motivation; effect estimation and ground-truth cause recovery belong to separate causal-evaluation designs.

Equation (2.30) is used here as DICE’s conceptual structural causal model (SCM). The corresponding additive score and branch-training rules are stated once in Section 2.5.2. DICE uses item popularity as weak distributional information for cause-specific sampling and supervision, but popularity should not be read as identical to conformity. The conformity component remains user–item specific, and later thesis notation for popularity or visibility context is an EDGRec modelling input rather than DICE’s exact SCM.

For EDGRec, the literature implication is representational: branch scores, popularity-conditioned contrasts, and branch-discrepancy diagnostics can encourage a useful separation of modelling roles under observational feedback. The branch labels therefore denote modelled roles supported by losses and diagnostics, while the logged interaction remains the observed ranking signal. Causal surveys discuss conformity as a data-bias issue, and DICE uses the causal-embedding perspective to motivate separating interest and conformity representations [4, 5, 10].

2.4.4 Correlation and causal interpretation

Most recommender models trained on observational feedback estimate associations that support ranking, not causal effects of exposure or recommendation. In potential-outcome notation, if treatment represents exposure or recommendation, the unit-level causal effect for a user–item pair can be written as

$$\tau_{ui} = Y_{ui}(1) - Y_{ui}(0), \quad (2.31)$$

where $Y_{ui}(1)$ is the response under treatment and $Y_{ui}(0)$ is the response under non-treatment. Estimating τ_{ui} requires assumptions or data that connect the observed outcome to the missing counterfactual outcome, such as randomized exposure, valid propensity scores, ignorability, positivity, and a carefully specified treatment definition [4, 11].

For this thesis, the implication is that empirical claims must be phrased as claims about offline ranking behaviour, representation structure, popularity/conformity-related patterns, and resource use under the chosen data protocol. Identified treatment-effect, uplift, or counterfactual-policy claims would require additional assumptions or data not provided by the default observational setting.

2.4.5 Causal recommendation and exposure-aware methods

This subsection separates the causal motivation used in the thesis from causal-identification claims. It first summarizes why causal framing is relevant, then reviews exposure-aware methods and the limits of observational evidence.

Causal recommendation motivation

Causal recommendation methods ask questions that ordinary correlational ranking does not answer directly. Instead of only predicting which items are likely to be interacted with under the observed logging policy, they may ask how a user’s response would change if an item were exposed, if a ranking policy changed, or if a confounding mechanism were adjusted. Surveys of causal inference for recommendation organize this literature around potential outcomes, structural causal models, counterfactual reasoning, debiasing, and causal effect estimation [4, 10, 11].

Causal Embeddings for Recommendation (CausE) is an early embedding-space example of this motivation. It formulates recommendation in terms of treatment and control policies and learns related treatment and control embeddings with a regularization term connecting the two tasks, aiming to improve recommendation from a causal effect perspective [18]. This is relevant background because it shows how causal motivation can be expressed in embedding space without making treatment–control embedding regularization the only possible way to use causal ideas in representation learning.

Exposure-aware and counterfactual recommendation

Exposure-aware methods make the observation process explicit through an exposure indicator $Z_{ui} \in \{0, 1\}$. A common notation is a propensity score e_{ui} , where X_{ui} denotes observed covariates and $e_{ui} = \Pr(Z_{ui} = 1 \mid X_{ui})$ is the probability that user–item pair (u, i) is exposed or observed. Let $\mathcal{D}$ denote the target user–item sample, let $\mathcal{O} = \{(u, i) : Z_{ui} = 1\}$ be the observed subset, and let $\ell_{ui} \in \mathbb{R}$ be a loss or utility contribution. Inverse-propensity estimators then reweight observed losses to reduce bias from non-uniform exposure. A generic

population-risk notation is

$$\begin{aligned}\hat{R}_{\text{IPS}} &= \frac{1}{|\mathcal{D}|} \sum_{(u,i) \in \mathcal{D}} \frac{Z_{ui} \ell_{ui}}{e_{ui}} \\ &= \frac{1}{|\mathcal{D}|} \sum_{(u,i) \in \mathcal{O}} \frac{\ell_{ui}}{e_{ui}}.\end{aligned}\tag{2.32}$$

In Equation (2.32), the inverse-propensity correction is the factor $1/e_{ui}$, which upweights rarely exposed pairs and downweights frequently exposed pairs; the outer factor $1/|\mathcal{D}|$ only normalizes the empirical estimate. In training objectives it is also common to average a loss weighted by inverse propensity scores over the logged observations, but that $1/|\mathcal{O}|$ normalization should be interpreted as a sample-averaging convention rather than as the causal correction itself. Causal recommendation surveys describe IPS and related reweighting methods as standard tools for exposure-aware learning and evaluation, while also emphasizing their dependence on propensity quality, positivity, and identifying assumptions [4, 11].

PropCare, a propensity-estimation method for causality-oriented recommendation without exposure data, addresses the practical obstacle that exposure probabilities are often unavailable, reflecting the importance and difficulty of propensity estimation in realistic recommendation logs [19]. KuaiRand provides randomized exposure data that can support this discussion empirically, but using it still requires a clear definition of exposure, response, candidate sets, and temporal order [15]. For the present thesis, inverse propensity scoring (IPS) is therefore treated as important background rather than as the main methodological claim unless the exposure and propensity assumptions are explicitly justified in the experimental protocol.

Causal identifiability boundary

The identifiability question in causal recommendation is whether the observed log can support a well-defined counterfactual comparison. For a user–item pair, a dataset may record a response under exposure or no logged response under non-exposure, but it does not simultaneously observe both $Y_{ui}(1)$ and $Y_{ui}(0)$ under otherwise identical conditions. Causal surveys therefore connect valid estimation to assumptions such as no unmeasured confounding, positivity, stable treatment definitions, and correct modelling of exposure or intervention mechanisms [4, 10, 11].

This boundary gives the thesis its positive methodological scope. Interest/conformity-inspired representation separation can be studied inside an efficient graph recommender, and its behaviour can be evaluated through offline ranking, list composition, and resource measurements under an explicit observational-data protocol.

2.5 Disentangled Representation Learning for Recommendation

This section reviews representation-learning work that motivates interest and conformity separation. It starts from general disentanglement, then narrows to DICE and related contrastive, temporal, semantic, and collapse-aware extensions.

2.5.1 General disentangled representation learning

Observed implicit feedback is not a direct measurement of a single stable preference. Let $m \in \mathbb{N}_{\geq 1}$ be the number of hypothesized latent factors, let $Z_{ui}^{(1)}, \dots, Z_{ui}^{(m)}$ denote latent causes for user-item pair (u, i) , let ϵ_{ui} denote noise, and let f be an unspecified response-generating function. The observed response can be schematically represented as an entangled signal

$$Y_{ui} = f\left(Z_{ui}^{(1)}, Z_{ui}^{(2)}, \dots, Z_{ui}^{(m)}, \epsilon_{ui}\right), \quad (2.33)$$

where the latent causes may include user interest, conformity, popularity exposure, short-term context, long-term preference, semantic intent, item quality, external trend effects, and noise. Disentangled recommendation attempts to replace a single monolithic embedding with factor-specific components. For user and item embeddings $\mathbf{e}_u, \mathbf{e}_i \in \mathbb{R}^d$ partitioned into m component blocks $\mathbf{e}_u^{(q)}, \mathbf{e}_i^{(q)}$, this can be written as

$$\mathbf{e}_u = \left[\mathbf{e}_u^{(1)}, \dots, \mathbf{e}_u^{(m)}\right], \quad \mathbf{e}_i = \left[\mathbf{e}_i^{(1)}, \dots, \mathbf{e}_i^{(m)}\right], \quad (2.34)$$

where each component is intended to capture a distinct source of variation in observed behaviour. This is a modelling objective: a learned component is a structured representation encouraged to align with a hypothesized factor, while causal identification would require interventions, randomized exposure, valid instruments, or additional identifying assumptions.

This distinction explains why disentanglement is valuable in recommender systems. Under observational feedback, the same click may reflect genuine interest, conformity toward a highly visible item, an interface-driven exposure effect, a temporary trend, or a sparse accidental signal. A factorized representation can allocate capacity to these sources separately, support diagnostics of branch collapse or popularity concentration, and make it easier to compare ranking utility with popularity-aware list behaviour. The added objectives, side information, or temporal assumptions can also be expensive and dataset-dependent, which motivates a bounded representation-learning synthesis with explicit empirical scope.

2.5.2 DICE as the direct conceptual anchor

DICE, the Disentangling User Interest and Conformity for Recommendation with Causal Embedding framework, is the direct conceptual anchor for the interest and conformity branches used in this thesis. Its starting point is that observational interaction data may reflect unobserved interest-related and conformity-related factors toward popular items; treating every interaction as a single preference signal can therefore entangle user taste with the tendency to follow visible or socially validated items [5]. DICE assigns users and items separate interest and conformity embeddings and combines their branch scores additively. For user and item branch embeddings $\mathbf{e}_u^I, \mathbf{e}_i^I, \mathbf{e}_u^C, \mathbf{e}_i^C \in \mathbb{R}^d$, the source model's score form can be written as

$$s_{ui} = s_{ui}^I + s_{ui}^C, \quad (2.35)$$

$$s_{ui}^I = (\mathbf{e}_u^I)^\top \mathbf{e}_i^I, \quad s_{ui}^C = (\mathbf{e}_u^C)^\top \mathbf{e}_i^C. \quad (2.36)$$

In the source model, the interest score is designed to capture preference for item characteristics, while the conformity score is designed to capture the tendency to interact with popular or socially visible items. Figure 2.11 shows the original DICE causal-embedding diagram that operationalizes this idea. It is retained in addition to the simpler TikZ motif in Figure 2.10 because it conveys substantially more of the paper's modelling context: DICE replaces scalar popularity-only modelling with separate interest and conformity embeddings for users and items, trains the branches with cause-specific data and discrepancy supervision, and balances the decomposed tasks through multi-task curriculum learning [5]. The figure is therefore evidence for the literature design that motivates EDGRec's bounded branch structure.

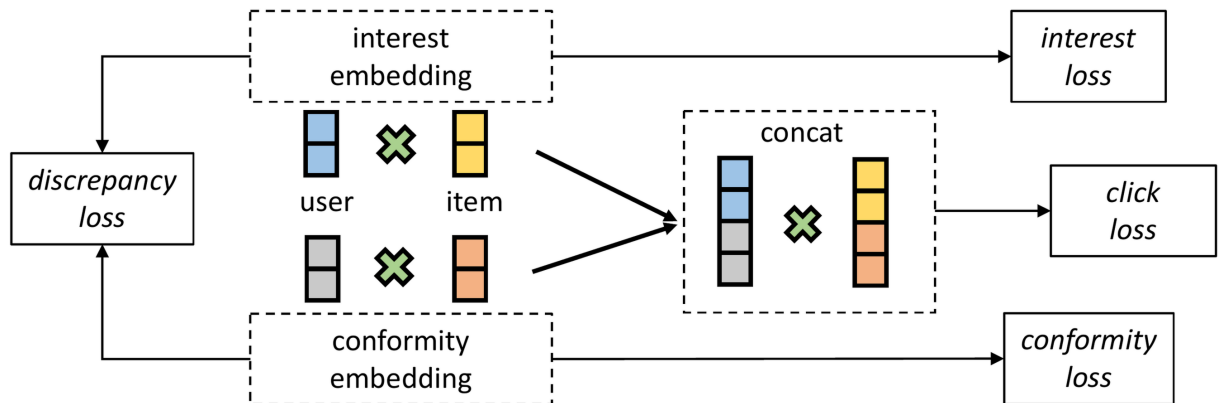


Figure 2.11: DICE causal graph and causal embeddings. The source figure presents click as arising from interest and conformity, then implements this decomposition with separate user and item embeddings. The branch scores are computed by inner products, combined additively for click prediction, and trained through cause-specific losses, discrepancy supervision, and multi-task curriculum learning. Source: Zheng et al. [5].

The distinctive part of DICE is not only the two-branch score but also its cause-specific training logic. For user u , observed positive item i , sampled unobserved item j , binary labels $y_{ui} = 1$ and $y_{uj} = 0$ under the training convention, and item-popularity proxy values $p_i, p_j \in \mathbb{R}_{\geq 0}$, DICE uses item popularity to form weak supervision from popularity-conditioned cases. In the source model, sampled positive-negative pairs are partitioned by relative item popularity. If the positive item is more popular than the negative item, then the observed preference may be supported by conformity as well as interest; the conformity branch is encouraged to rank the more popular positive item higher:

$$p_i > p_j \Rightarrow s_{ui}^C > s_{uj}^C. \quad (2.37)$$

If the observed positive item is less popular than the sampled negative item, then DICE interprets the click as evidence that interest overcomes popularity:

$$p_j > p_i, y_{ui} = 1, y_{uj} = 0 \Rightarrow s_{ui}^I > s_{uj}^I. \quad (2.38)$$

In the corresponding conformity branch for this case, the popularity direction is reversed: the more-popular negative item is expected to receive the higher conformity score, $s_{uj}^C > s_{ui}^C$, while the interest branch explains why the clicked less-popular item can still be preferred. In the original argument, these inequalities are motivated by the collider analysis of interest and conformity as causes of an observed click. DICE further adds discrepancy or disentanglement supervision between the embedding sets and optimizes click, interest, conformity, and discrepancy tasks with a multi-task curriculum strategy [5].

For this thesis, these equations are literature-level schematics rather than the thesis loss definition. DICE supplies the interest and conformity representation idea and the popularity-conditioned branch-supervision motif. EDGRec differs at the methodological level: it embeds the dual-branch idea inside a LightGCN propagation framework, trains on bounded sampled subgraphs, controls branch score scale before fusion, adds an item-only context path for train-derived popularity and catalogue signals, and evaluates the resulting model through ranking, list behaviour, runtime, and VRAM components. A reference DICE comparison therefore means a baseline specification intended to preserve the cited study’s modelling and training conventions, not an approximation trained under EDGRec’s efficient sampled regime. Chapter 3 defines the concrete adaptation and keeps any reference DICE comparison separate from EDGRec itself.

The methodological gap after DICE is not the absence of an interest-conformity hypothesis; that hypothesis is already supplied by the source paper. The gap addressed in this dissertation is how to make that hypothesis operational in a protocol-centred graph recommendation study where train-only graph construction, sampled propagation, bounded auxiliary computation, split-safe item context, and hardware-dependent resource evidence are part of the scientific claim. EDGRec answers this gap by changing the experimental

contract rather than by claiming to identify new causal quantities. Its branch labels inherit their modelling role from the DICE literature, while its contribution is judged through the observable ranking, list behaviour, runtime, memory, and ablation evidence reported later. This distinction is essential for comparison: DICE is the literature source for the branch hypothesis, whereas EDGRec is the thesis method that tests a resource-aware graph implementation of that hypothesis under explicit preprocessing and hardware constraints.

2.5.3 Contrastive causal-disentangled extensions

Disentangled Causal Embedding with Contrastive Learning (DCCL) and Causal-augmented Disentanglement for Contrastive Recommendation (CaDCR) extend causal-disentangled recommendation by adding contrastive regularization to the representation geometry. DCCL keeps the interest and conformity decomposition of DICE but replaces hard popularity-conditioned splitting with popularity-aware contrastive sample augmentation for interest and conformity. It is designed as a model-agnostic add-on that can be applied to matrix-factorization backbones or to backbones that use LightGCN propagation, and it targets the sparsity of pure-cause samples and long-tail items; its trade-off is extra pairwise contrastive computation during training [20].

For an anchor representation $\mathbf{z} \in \mathbb{R}^d$, positive representation $\mathbf{z}^+ \in \mathbb{R}^d$, finite negative set $\mathcal{Z}^- \subset \mathbb{R}^d$, similarity function $\text{sim} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$, and temperature $\tau > 0$, a generic contrastive objective has the form

$$\mathcal{L}_{CL} = -\log \frac{\exp(\text{sim}(\mathbf{z}, \mathbf{z}^+)/\tau)}{\exp(\text{sim}(\mathbf{z}, \mathbf{z}^+)/\tau) + \sum_{\mathbf{z}^- \in \mathcal{Z}^-} \exp(\text{sim}(\mathbf{z}, \mathbf{z}^-)/\tau)}. \quad (2.39)$$

This leading minus sign makes the expression a negative log-probability, equivalently a cross-entropy term that encourages the positive representation to receive higher softmax probability than the negatives. The form captures the common idea of pulling semantically related or augmented representations together while pushing negatives apart, without importing the full sampling scheme of any one source. It is background notation for contrastive causal-disentangled extensions and enters the thesis method only when an ablation explicitly enables it.

CaDCR broadens the same family by separating individual and collective intent, applying causal intervention against popularity-related confounding, and regularizing representations in both interaction and intent spaces [21]. It is relevant for positioning EDGRec because it shows how popularity, collective behaviour, and graph contrastive learning can be combined in a causal-disentangled framework. In this thesis, CaDCR remains background motivation rather than an EDGRec component, since the implemented method does not include dual-channel intent modelling, intervention modules, or dual-space contrastive regularization. Contrastive or representation-geometry losses are optional, bounded ablations because the main EDGRec claim prioritizes runtime and VRAM efficiency.

2.5.4 Temporal and dynamic disentanglement

A Contrastive Learning framework to disentangle Long and Short-term interests for Recommendation (CLSR) is a branch of the literature that studies disentanglement under time-varying user preference and dynamic popularity. CLSR separates long-term and short-term interests with separate encoders, constructs proxies from the whole interaction history and the recent interaction window, supervises the two representations with contrastive tasks, and adaptively fuses them for prediction [22]. Let $\mathbf{u}^L$ denote long-term preference, $\mathbf{u}_t^S$ denote short-term preference at time t , and let $\alpha_{ui,t} \in [0, 1]$ be an adaptive fusion weight that may depend on the user, target item, and recent context. A compact abstraction of this family is

$$\mathbf{u}_t = \alpha_{ui,t} \mathbf{u}^L + (1 - \alpha_{ui,t}) \mathbf{u}_t^S, \quad (2.40)$$

The equation states that the representation used for a target interaction is a convex mixture of long-term and recent-interest components; the weight determines how strongly the current recommendation is allowed to depend on short-term context.

Dual Disentanglement of user–item interaction for recommendation with Causal Embedding (DDCE) combines this long–short interest idea with popularity-aware causal embedding: it models user interest and item popularity as separate causes, incorporates item attributes into the popularity side, and uses contrastive learning and dynamic fusion for long–short user interests [23]. Figure 2.12 shows the feedback-loop motivation used by DDCE.

The figure is placed here because it connects the generic feedback-loop problem to an extension of causal disentanglement. In DDCE’s motivating example, repeated positive feedback on one movie category can cause the system to recommend that category increasingly often, narrowing exposure and contributing to a filter-bubble effect. The figure is not used as a second causal graph; it is evidence for why later disentanglement work extends beyond DICE’s interest and conformity split.

Causal Disentangled Recommendation against user preference shifts (CDR) and Causal Disentangled Sequential Recommendation (CDSRec) move further toward temporal causal modelling. CDR splits chronological user histories into environments and uses a temporal variational autoencoder (VAE) to infer unobserved environmental factors, model preference transitions, and learn a sparse structure from fine-grained preference factors to interactions [24]. Let E_t denote latent environmental or user factors, Z_t denote preference, and X_t denote interactions in environment t . Its causal abstraction can be summarized as

$$E_t \rightarrow Z_t \rightarrow X_t, \quad Z_{t-1} \rightarrow Z_t, \quad (2.41)$$

This abstraction says that the current interaction is mediated by a time-dependent preference state, while the previous preference state constrains how that state evolves across environments.

CDSRec treats popularity bias as dynamic in sequential recommendation. It separates

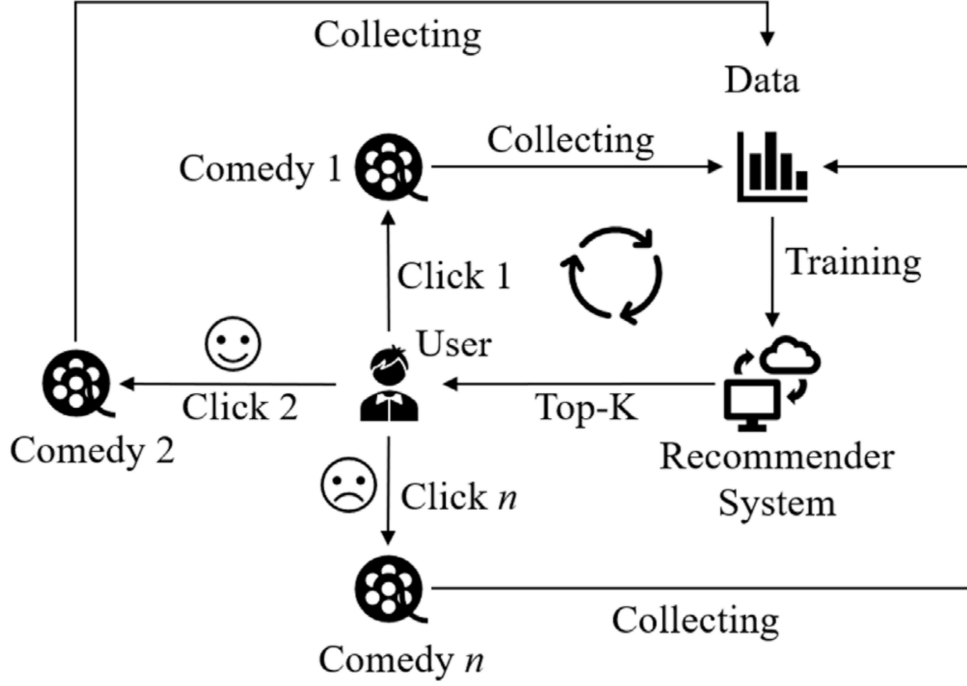


Figure 2.12: DDCE feedback loop of user–item interaction and the recommender system. The source figure illustrates how recommendations, user feedback, and subsequent training data can reinforce category or popularity concentration over repeated cycles, motivating DDCE’s dual disentanglement of user interests and item popularity. Source: Wang et al. [23].

personalized demand, beneficial inherent quality, and harmful external interference, and uses a sequential VAE with Markovian temporal hidden factors [25]. Let x_t represent personalized demand, y_t beneficial or inherent quality, and z_t harmful external interference at time t . Its factorization can be summarized as

$$\text{interaction}_t \leftarrow (x_t, y_t, z_t), \quad \text{score}_t \leftarrow (x_t, y_t), \quad (2.42)$$

The notation separates the factors that may justify recommendation from the factors that should be treated as external interference. This distinction is important because popularity is not always harmful; some popularity signals may proxy stable item quality, while transient external interference can distort ranking. Accordingly, this thesis reports ranking utility and popularity-aware list behaviour together rather than treating lower popularity exposure as automatically better. Temporal interest is treated only as optional context rather than as the default model family.

2.5.5 Semantic, knowledge-guided, and intent-aware disentanglement

Semantic and knowledge-guided disentanglement asks whether learned factors can be tied to interpretable meanings rather than remaining anonymous embedding blocks. Causal Disentanglement for Semantic-Aware Intent Learning (CaDSI) uses heterogeneous informa-

tion, context representations built from meta-paths, semantic-aware intent learning, and back-door causal intervention to address context-induced confounding in heterogeneous information network (HIN) recommendation [26].

Knowledge-guided disentanglement argues that interaction-only factors can lack clear semantic meaning and can suffer under sparse data. Knowledge-Guided Disentangled Representation Learning (KDR) addresses this issue by learning explicit item-factor representations from a knowledge graph and aligning them with implicit-interaction factors through mutual-information maximization [27]. Intent-aware Disentangled Graph Contrastive Learning (IDCL) models behaviour data as a user–item–concept graph, learns concept-guided interpretable intents, uses intent-wise graph contrastive learning, and applies coding-rate reduction to encourage behaviours assigned to different intents to occupy more orthogonal subspaces [28].

Let $K \in \mathbb{N}_{\geq 1}$ be the number of semantic factors, let $\mathbf{e}_u^{(k)}$ be factor k of user embedding $\mathbf{e}_u$, and let $\mathbf{c}_k$ denote an interpretable concept or semantic basis associated with factor k . A compact semantic-factor view is

$$\mathbf{e}_u = [\mathbf{e}_u^{(1)}, \dots, \mathbf{e}_u^{(K)}], \quad \mathbf{e}_u^{(k)} \leftrightarrow \mathbf{c}_k. \quad (2.43)$$

The final relation should be read as an association between factor k and a semantic basis, not as a claim that the thesis recovers a human-interpretable concept for every learned embedding component.

The thesis implication is that side features may support disentanglement analysis only when they are safe, pre-treatment, and semantically meaningful under the dataset protocol. Features derived from future interactions, labels, post-recommendation outcomes, or target leakage can invalidate the empirical comparison. Accordingly, this thesis treats side features as optional weak evidence for branch structure, not as semantic supervision or evidence that an embedding dimension has a human-interpretable causal meaning.

2.5.6 Collapse and efficient guidance for multi-interest learning

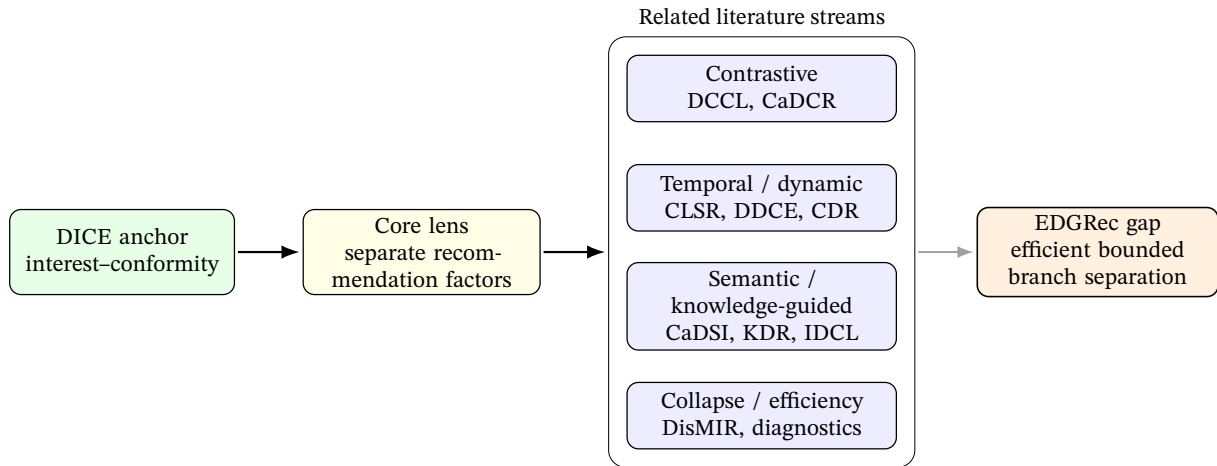
Disentangled Multi-Interest Representation Learning for Sequential Recommendation (DisMIR) focuses on a practical failure mode of disentangled and multi-interest models: the learned interests can collapse at the item level, where relevant items are allocated inconsistently across interests, or at the facet level, where multiple interest facets become nearly identical [29]. DisMIR addresses this by partitioning items according to global co-occurrence patterns, formally connecting its item-partition objective to spectral clustering, and then approximating the partition objective efficiently through Markov random field (MRF) subgraph sampling and contrastive learning.

Although this thesis does not implement DisMIR’s item-partition objective, the collapse concern is directly relevant to a two-branch interest and conformity model. Let $\mathbf{z}^I, \mathbf{z}^C \in \mathbb{R}^d$

be interest- and conformity-branch representations for the same entity or batch summary. A simple branch-similarity diagnostic is

$$\cos(\mathbf{z}^I, \mathbf{z}^C) = \frac{(\mathbf{z}^I)^\top \mathbf{z}^C}{\|\mathbf{z}^I\|_2 \|\mathbf{z}^C\|_2}. \quad (2.44)$$

High similarity between interest and conformity representations may indicate weak disentanglement or branch collapse, while low similarity is informative only when paired with ranking and list behaviour evidence. The diagnostic must therefore be interpreted together with Recall@K, NDCG@K, popularity-aware list behaviour measures, runtime, and memory. Figure 2.13 maps the literature streams that motivate this bounded view, and Table 2.1 lists the reviewed disentanglement papers and their specific relevance to EDGRec. DICE is the direct conceptual anchor because it makes interest and conformity explicit; contrastive, temporal, semantic, and efficient-disentanglement papers broaden the design space while leaving the EDGRec claim tied to the implemented evidence. The figure therefore marks a bounded literature gap: efficient branch separation under explicit data and resource constraints.



The figure shows influence at the literature level: DICE supplies the direct anchor, the middle panel groups extensions, and Chapter 3 defines the components actually implemented.

Figure 2.13: Disentangled recommendation literature map. DICE is the direct interest and conformity anchor, while contrastive, temporal, semantic, and collapse-aware extensions motivate design choices and limitations; the thesis gap is efficient, explicitly bounded graph recommendation rather than causal identification. Acronyms in the map denote Disentangled Causal Embedding with Contrastive Learning (DCCL), Causal-augmented Disentanglement for Contrastive Recommendation (CaDCR), Contrastive Learning for Long/Short-term Recommendation (CLSR), Dual Disentanglement of user-item interaction for recommendation with Causal Embedding (DDCE), Causal Disentangled Recommendation (CDR), Causal Disentanglement for Semantic-Aware Intent Learning (CaDSI), Knowledge-Guided Disentangled Representation Learning (KDR), Intent-aware Disentangled Graph Contrastive Learning (IDCL), and Disentangled Multi-Interest Representation Learning for Sequential Recommendation (DisMIR).

Table 2.1: Disentangled recommendation literature and relevance to EDGRec.

Work	Disentangled factors	Main modelling idea	Relevance to EDGRec
DICE [5]	Interest and conformity	Separate embeddings, popularity-conditioned cause-specific training, discrepancy loss	Direct conceptual anchor for dual branches
DCCL [20]	Interest and conformity	Popularity-aware contrastive augmentation	Optional inspiration; contrastive costs must be bounded
DDCE [23]	User interest and item popularity; long- and short-term interest	Dual-end popularity modelling, item attributes, temporal interest fusion	Supports popularity and context head and future temporal extension
CLSR [22]	Long- and short-term interest	Proxies from full and recent history, contrastive supervision, adaptive fusion	Supports context and temporal framing for future configurations
CDR [24]	Preference shifts and sparse category influence	Chronological environments, temporal VAE, sparse structure learning	Motivates preference-shift limitation and future work
CDSRec [25]	Demand / inherent quality / external interference	Sequential VAE, Markovian temporal hidden factors	Motivates dynamic popularity framing for future sequential extensions
CaDSI [26]	Semantic-aware intent / confounding context	HIN, meta paths, back-door intervention	Supports safe semantic feature discussion
KDR [27]	Implicit / explicit disentangled aspects	KG guidance and mutual-information alignment	Supports feature interpretability and sparsity discussion
IDCL [28]	Intent factors / behavior distributions	User-item-concept graph, intent-wise contrastive learning, coding-rate reduction	Supports branch diagnostics and intent collapse discussion
DisMIR [29]	Multi-interest facets	Item partition, spectral clustering connection, MRF subgraph approximation	Supports collapse diagnostics and efficient supervision discussion
CaDCR [21]	Individual / collective intent	Dual-channel causal disentanglement and dual-space contrastive regularization	Supports popularity and confounding framing for bounded interpretation

2.5.7 From causal disentanglement to efficient graph recommendation

The literature reviewed in this section motivates EDGRec through synthesis rather than direct inheritance from one source model. DICE supplies the interest and conformity hypothesis introduced in Section 2.4.3; LightGCN supplies the efficient degree-normalized graph propagation backbone; and alignment/uniformity work in collaborative filtering motivates representation-geometry checks when branch structure is part of the analysis [5, 6, 13]. EDGRec uses these ideas to define an operational graph-recommendation design with split-safe graph construction, sampled propagation, bounded auxiliary objectives, an explicit item-context scorer, and resource-aware reporting. The common theme is that representation structure is useful only when the data protocol and training cost remain explicit.

The broader disentanglement literature shows several ways to strengthen factor separation: contrastive or dual-intent objectives, temporal and environment-specific structure, sequential causal modelling, knowledge or concept guidance, heterogeneous semantic information, and item-partition supervision for collapse control [20, 21, 22, 23, 24, 25, 26, 27, 28, 29]. These strategies improve the modelling vocabulary available to graph recommendation, but they also introduce extra assumptions, supervision requirements, and resource costs. Chapter 3 therefore specifies which EDGRec components are actually implemented and treats contrastive, geometry, propensity, temporal, and side-feature components as declared ablations or configuration choices rather than as default evidence for recovered causal factors.

2.6 Efficiency and Resource-Aware Evaluation in Graph Recommendation

This section links model quality to the resource constraints of graph recommendation. It introduces runtime, memory, and accuracy–efficiency trade-offs before motivating Composite Resource-aware Recommendation Utility (CRRU) as a post hoc reporting utility.

2.6.1 Runtime and memory constraints

Resource-aware evaluation is necessary because graph recommenders are trained and tested under finite hardware budgets, not only under abstract ranking objectives. Runtime affects how many preprocessing variants, random seeds, hyperparameter settings, and ablations can be repeated. Memory affects whether the selected graph representation, embedding dimension, propagation depth, and evaluation candidate set fit on the available hardware. These constraints matter for scientific comparison: a model that is too costly to tune or rerun

under the same protocol may be less appropriate for the thesis setting than a model with slightly lower ranking quality but stable and reproducible resource behaviour.

For a graph recommender that uses identifier embeddings, resource use can be decomposed into representation storage, graph storage, training computation, and evaluation computation. The embedding table has order $\mathcal{O}((|\mathcal{U}| + |\mathcal{I}|)d)$ parameters. A sparse LightGCN propagation pass over the training graph has order $\mathcal{O}(L|\mathcal{E}_{\text{train}}|d)$ arithmetic for L propagation layers, before the additional cost of BPR sampling, negative sampling, scoring, loss computation, and optimizer updates [3, 6]. Training also requires memory for sparse adjacency storage, optimizer states, sampled triples, layer outputs or caches, and validation state.

The choice between full-graph and sampled propagation is therefore methodological, not merely engineering. Full-graph propagation preserves the exact training graph at the cost of a larger working set. Sampled propagation lowers the active working set by exposing the model to neighbourhoods or interaction batches, but it makes batch size, fan-out, and graph-coverage frequency part of the experimental design. Bounded auxiliary losses belong to the same design layer: branch, contrastive, independence, or geometry terms can reshape representations, while their sampling scope and weight determine both the training signal and the memory and runtime cost. Full-graph versus mini-batch analysis reports that batch size and fan-out influence convergence, generalization, and computational efficiency, so these settings should be reported with the same care as learning rate, embedding size, and propagation depth [17].

Validation and test scoring can become a separate runtime bottleneck. If all evaluation users are scored against all catalogue items exhaustively, dense dot-product scoring has arithmetic complexity $\Theta(|\mathcal{U}_{\text{eval}}||\mathcal{I}|d)$ before masking consumed items and selecting top- K , and materializing the full score matrix has memory $\Theta(|\mathcal{U}_{\text{eval}}||\mathcal{I}|)$. In practice, chunked scoring may be required even when training itself fits in memory; it changes peak VRAM but not the total number of exhaustive dot products. This distinction is important for EDGRec: the thesis analyses the cost of graph construction, sampled propagation, negative sampling, validation and test scoring, runtime, and memory, while treating large-scale candidate retrieval as a separate systems problem rather than as part of the implemented EDGRec training contribution.

2.6.2 GPU memory constraints

Graphics processing unit (GPU) memory, reported in this thesis as VRAM, is a hard constraint for graph recommendation because exceeding the available device memory usually stops the experiment rather than merely slowing it. The persistent model state includes user and item embeddings, and optimizers such as Adam add additional state variables for each trainable parameter. The transient state can include sampled mini-batches, negative items, sparse adjacency tensors, propagated embeddings, intermediate scores, metric buffers, and evaluation chunks. Peak VRAM may therefore occur during validation or testing rather than

during the forward propagation step itself.

Graph workloads also differ from ordinary dense neural-network workloads. Sparse graph propagation can be memory-bandwidth limited, while full-catalog scoring can be a dense matrix multiplication followed by masking and top- K selection. CPU-side preprocessing, negative sampling, and data transfer may become bottlenecks even when the GPU has unused compute capacity. For this reason, resource-aware comparison should record not only the model architecture but also the training regime, batch size, fan-out or sampling rule, evaluation chunk size, and hardware used for measurement.

VRAM constraints affect the interpretation of accuracy results. A model that requires a much smaller embedding dimension, fewer propagation layers, or a reduced candidate set in order to run on available hardware is not being evaluated under the same conditions as a model that can use the intended configuration. Conversely, a lightweight model that permits larger batches, more seeds, or more repeated runs may produce more reliable empirical evidence. EDGRec is therefore framed as an efficient graph recommendation adaptation of interest and conformity separation, not as an architecture whose value depends on unlimited GPU memory.

2.6.3 Accuracy-efficiency trade-offs

Top- K ranking metrics remain necessary because the primary task is to recommend relevant items near the top of a finite list. Recall@ K , HitRatio@ K , and NDCG@ K capture different aspects of ranking quality and should be interpreted together with the split and candidate protocol used to compute them [1]. However, ranking quality alone does not express the full thesis objective. A model can improve NDCG while becoming substantially slower, increasing VRAM requirements, or concentrating recommendation lists around highly popular items.

The accuracy–efficiency trade-off is especially important for graph recommenders because the same design choice can affect several evaluation dimensions. Increasing embedding dimension may improve ranking quality but increases embedding-table memory and score-computation cost. Increasing propagation depth may add higher-order collaborative information but raises sparse propagation cost and can mix popularity signals more broadly through the graph. Increasing fan-out in sampled training may reduce approximation error but also increases neighbourhood expansion and memory pressure. Recent analysis of full-graph and mini-batch graph neural network training further motivates treating batch size and fan-out as accuracy, convergence, and efficiency choices rather than as neutral implementation constants [17].

This thesis therefore treats model comparison as multidimensional. Standard metrics report whether held-out positives are recovered and ranked well. Popularity and personalization metrics report whether recommendations are concentrated around common items or vary across users. Runtime and peak VRAM report whether the result is practical under

the stated hardware budget. These quantities are reported separately before any composite utility is used as a transparent post hoc summary of a chosen trade-off, not as a replacement for the underlying measurements. Efficiency-led findings can therefore be substantive when they improve the attainable accuracy–resource frontier or reduce the cost of validation, ablation, and replication, provided the associated ranking, list behaviour, and memory evidence remains visible.

This multidimensional view is also supported by multi-objective recommender-system research. Real recommender systems may involve objectives beyond ranking accuracy, including individual and aggregate quality objectives, stakeholder objectives, long-term versus short-term goals, user-interface objectives, and engineering objectives [2]. Composite-index methodology adds that any aggregate score requires transparent choices about indicator selection, normalization, weighting, aggregation, and robustness or sensitivity analysis [7]. Accordingly, this thesis frames CRRU as a reusable composite utility framework: it supports joint discussion of ranking, popularity-aware list behaviour, runtime, and memory, while the component metrics and, where possible, sensitivity to alternative weight choices remain visible.

2.6.4 Motivation for composite post hoc reporting

The preceding discussion motivates composite reporting only after the component evidence is visible. Multi-objective recommendation research explains why a recommender may be judged by more than ranking accuracy, while composite-index methodology explains how a scalar summary should be specified: indicators, transformations, weights, aggregation rule, and robustness limitations must be explicit [2, 7]. This combination is the literature basis for CRRU.

Chapter 3 therefore defines CRRU as a configurable post hoc utility for completed experimental runs. The framework can be reused in other recommendation settings, provided each use declares its indicators, transformations, weight vector, and interpretation context. In this thesis, CRRU summarizes one resource-aware preference over ranking quality, popularity-aware personalization, epoch time, and peak VRAM. Figure 2.14 fixes the order of interpretation used later: ranking, list behaviour, and resource cost remain the primary evidence, while CRRU is read as one declared summary of those already visible components.

2.6.5 Synthesis of literature streams

Table 2.2 condenses the chapter’s main literature streams into the corresponding methodological implications for the thesis.

Table 2.2: Synthesis of literature streams and thesis implications.

Literature stream	Key insight or limitation	Thesis implication
BPR / implicit feedback [3]	Positive-only data support pairwise ranking, but sampled unobserved items are contrast cases rather than verified dislikes.	Define positives, negative sampling, and candidate filtering explicitly before comparing models.
LightGCN and graph collaborative filtering [6, 9]	Simplified normalized propagation is a competitive graph collaborative filtering baseline, but it learns one entangled collaborative representation.	Use LightGCN propagation as the efficient backbone and add only bounded representation structure.
DICE and interest-conformity disentanglement [5]	Separate embeddings and popularity-conditioned branch losses motivate interest and conformity separation under observational data.	Treat DICE as the conceptual anchor, while keeping EDGRec and reference DICE comparison distinct.
Contrastive and representation-geometry disentanglement [13, 20, 21]	Contrastive, dual-intent, alignment, and uniformity terms can shape representations but add training cost and assumptions.	Keep auxiliary geometry losses optional, bounded, and diagnostic rather than central causal evidence.
Temporal / sequential causal disentanglement [22, 23, 24, 25]	Long-short preference, dynamic popularity, environments, and Markov or variational-autoencoder structure require temporal assumptions and sequence-specific modelling.	Treat temporal disentanglement as future or optional context unless Chapter 3 explicitly implements it.
Dataset and exposure-aware evaluation [10, 11, 14, 15]	Observation density, exposure policy, and randomized logs change what offline metrics mean.	Make positive labels, splits, graph construction, feature policy, and leakage controls part of the method.
Multi-objective and composite-index evaluation [2, 7]	Aggregate utilities require transparent indicators, weights, normalization, and robustness checks.	Present CRRU as configurable post hoc reporting, with all component metrics visible.
Full-graph and mini-batch efficiency [16, 17]	Sampling, batch size, and fan-out affect convergence, generalization, runtime, and memory.	Report training regime, fan-out, batch size, full-catalog scoring policy, runtime, and peak VRAM.

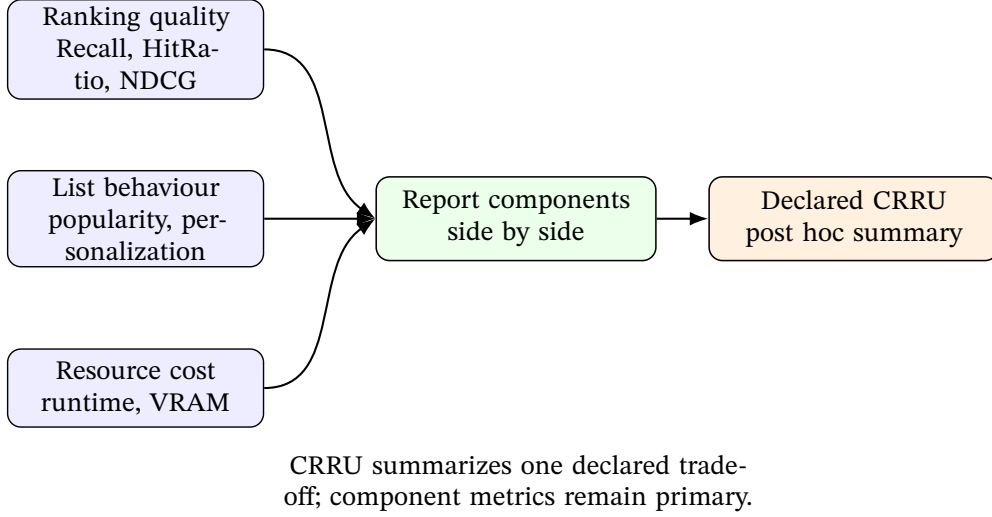


Figure 2.14: Resource-aware evaluation triangle. Ranking quality, popularity-aware list behaviour, and resource cost are reported as visible components; CRRU is a post hoc summary defined later for one declared trade-off, while component metrics remain primary.

2.7 Theoretical Framework and Research Gap

This section consolidates the review into the framework used by the thesis. The framework has three layers: a data layer that treats logged implicit feedback as observational evidence shaped by exposure and preprocessing; a representation layer that combines LightGCN propagation with the DICE interest and conformity motivation; and an evaluation layer that reports ranking quality, popularity-aware list behaviour, runtime, and VRAM before applying any composite utility. The research gap follows from the interaction of these layers rather than from any single literature stream.

2.7.1 Need for efficient, bias-aware graph recommendation

The first research gap is at the intersection of efficient graph collaborative filtering and bias-aware representation structure. LightGCN is an efficient graph recommendation baseline because it removes feature transformations and nonlinear activations while retaining normalized user-item propagation [6]. However, LightGCN is not disentangled into interest and conformity channels: it learns a single collaborative representation in which preference, popularity, exposure, and conformity-related signals can remain entangled.

DICE motivates interest and conformity disentanglement by separating embeddings and training them with cause-specific signals in its causal-embedding framework [5]. This is conceptually important for EDGRec because it turns popularity and conformity from a broad source of bias into a representational design hypothesis. At the same time, methods in this causal-embedding family can be expensive under limited hardware because they may require multiple embedding channels, additional sampling logic, branch-specific losses, and extra training or validation passes. More broadly, causal recommendation methods often require

exposure data, propensities, randomized logs, valid intervention definitions, or restrictive identifying assumptions that are not available in ordinary observational implicit-feedback experiments [4, 10, 11].

EDGRec addresses the resulting gap by integrating interest and conformity separation into an efficient graph recommender with LightGCN propagation, explicit preprocessing, sampled training, and realistic runtime and VRAM constraints. The methodological support is a triangulation: BPR justifies sampled pairwise ranking from positive-only data, LightGCN justifies simplified graph propagation, DICE motivates interest and conformity branches, DCCL and DirectAU motivate optional bounded representation-geometry regularizers, and graph-efficiency literature motivates sampled subgraph training plus runtime and VRAM reporting [3, 6, 5, 20, 13, 17]. The synthesis is deliberately selective: components such as contrastive regularization, propensity calibration, side-feature fusion, and temporal-history scoring require named configurations before they can support empirical claims. The resulting contribution is not a claim that one literature component alone is sufficient; it is a controlled integration showing how data protocol, representation design, and resource measurement must be specified together.

The synthesis matters because none of these streams alone fixes the full thesis problem: efficient graph recommendation under observational feedback requires a model design, a data protocol, and a resource-aware evaluation contract to be stated together. This positions EDGRec as a graph representation method informed by causal recommendation, with empirical claims stated in terms of observable ranking, list behaviour, diagnostics, and resource use.

2.7.2 Need for protocol-centred evaluation

The second research gap concerns data construction. Recommender datasets do not enter experiments as neutral preference tables. Positive labels depend on thresholds, feedback types, observation windows, and deduplication rules. Train-validation-test splits determine which interactions are allowed to form the propagation graph and which interactions are held out as evaluation targets. Candidate filtering determines which items can be recommended. Negative sampling determines which unobserved items are contrasted against positives during learning. These preprocessing decisions materially affect conclusions because they change the graph, the training triples, the held-out labels, and the missingness assumptions behind evaluation [1, 3].

This point is especially important for datasets with different observation processes. MovieLens rating records, Amazon product interactions, KuaiRec’s dense and sparse matrices, and KuaiRand’s randomized short-video exposures support different empirical claims even if they are all converted into user-item edges [6, 12, 14, 15]. A protocol-centred evaluation must therefore state how positives are defined, how missing entries are interpreted, how graph edges are constructed, how leakage is avoided, and how candidate sets are scored.

The methods gap is to make this preprocessing contract explicit before presenting EDGRec. Without a protocol-centred design, differences between LightGCN, baselines related to DICE, and EDGRec could be caused by inconsistent graph construction, candidate policies, or evaluation leakage rather than by the representation design itself. The literature therefore motivates a methods chapter that specifies the dataset, positive-label rule, split, graph construction, sampling rule, and evaluation protocol as part of the contribution.

2.7.3 Need for resource-aware comparison

The third research gap concerns comparison criteria. Ranking metrics are necessary, but they do not express the full thesis goal because the thesis studies ranking quality, popularity-aware list behaviour, runtime, and VRAM together. Under a fixed hardware budget, a higher NDCG@ K value may be less useful if it requires substantially more memory or training time than alternatives. Conversely, a model with closely matched ranking quality can be scientifically valuable when it is cheaper to tune, easier to replicate, and less concentrated around train-popular items. Evaluation surveys emphasize that ranking metrics must be interpreted with the task and protocol, while graph-learning efficiency studies motivate treating training regime and sampling choices as factors that affect both learning behaviour and resource cost [1, 17].

CRRU addresses this comparison gap as a post hoc utility. It records one configurable way to summarize a selected trade-off after Recall@ K , HitRatio@ K , NDCG@ K , popularity-aware list behaviour analysis, training runtime, and peak training VRAM have been reported. The framework is reusable because its indicators, transformations, and weights can be changed for another recommendation setting; only the weight vector used later is specific to this dissertation’s resource-aware objective.

This distinction matters: CRRU is a reusable aggregation framework whose scientific meaning is fixed by the declared components, transformations, weights, dataset protocol, and hardware convention. It is therefore appropriate here because the stated goal is to compare efficient graph recommendation with interest and conformity separation under bounded hardware, rather than to optimize one ranking metric in isolation.

Together, EDGRec and CRRU address a bounded methodological gap in the literature reviewed in this chapter: efficient graph recommendation with interest and conformity separation under explicit data preprocessing and resource-aware post hoc evaluation for observational implicit feedback. The contribution integrates a representation hypothesis motivated by DICE with LightGCN propagation, sampled training, leakage-controlled context construction, component ablations, and a visible resource-aware comparison protocol. The gap is methodological rather than identificatory: bias-aware representation design and resource-aware evaluation can be studied rigorously when their assumptions are declared, while causal identification remains an estimand-driven problem with additional data and assumption requirements. Chapter 3 follows from this gap by defining the formal problem

setting, the data construction protocol, and the EDGRec method. Chapter 4 instantiates the experimental contract, and Chapter 5 evaluates whether the method provides favourable ranking, popularity-aware list behaviour, runtime, and memory trade-offs while keeping component metrics visible and dataset-specific evidence boundaries intact.

Chapter 3

Problem Definition and Methodology

The methodological contract in this chapter makes the research questions in Chapter 4 answerable. Chapter 2 established the literature motivation; the present chapter fixes the observed top- K task, the data and leakage-control protocol, the baseline taxonomy, the Efficient Disentangled Graph Recommender (EDGRec) method, the training objective derived from Bayesian Personalized Ranking (BPR), the validation–test evidence boundary, Composite Resource-aware Recommendation Utility (CRRU), and the reproducibility contract. The definitions are explicit because the study uses observational implicit feedback, where preprocessing and evaluation choices are part of the research design rather than neutral implementation details.

The argument proceeds from observation to comparison. The problem definition first states what is observed, what is missing, and where observational interpretation is limited. The data protocol then fixes positive-label construction, split masks, train-only graph construction, feature eligibility, and train-derived context. Only after these contracts are fixed does the chapter define baselines and EDGRec, because model comparisons are meaningful only under the same graph, candidate, sampling, and leakage rules. The training and evaluation sections then connect the method to the experimental questions introduced in Chapter 4: ranking metrics address relevance and dataset dependence; AveragePopularity and Personalization support list behaviour analysis; runtime and peak training VRAM support resource analysis; ablations support component analysis; and CRRU supports resource-aware utility analysis as a transparent post hoc summary built from visible components.

3.1 Problem Definition

The formal task is fixed before method-specific details are introduced. The section defines the observed top- K setting, train-only graph construction, missing-data interpretation, and resource-aware claim scope.

3.1.1 Observed implicit-feedback top- K task

Let $\mathcal{U}$ be the user set, $\mathcal{I}$ the item set, and $\mathcal{R}$ the raw event table supplied by a dataset. After dataset-specific canonicalization, the thesis represents the observed interaction evidence as

$$\mathcal{D} = \{d_n = (u_n, i_n, r_n, t_n, \mathbf{x}_{u_n}, \mathbf{x}_{i_n}, \mathbf{m}_n) : u_n \in \mathcal{U}, i_n \in \mathcal{I}, n = 1, \dots, N\}, \quad (3.1)$$

where r_n is the dataset-native response value or response vector, t_n is an available event timestamp or deterministic pseudo-time, $\mathbf{x}_{u_n}$ and $\mathbf{x}_{i_n}$ are optional split-safe user and item descriptors, and $\mathbf{m}_n$ stores protocol metadata such as source domain, exposure flag, feedback type, preprocessing protocol, and supplied split indicators. Equation (3.1) is an observed log, not a complete preference matrix.

Each preprocessing protocol defines a positive-label map

$$z_n = \psi_D(r_n, \mathbf{m}_n) \in \{0, 1\}, \quad \mathcal{S}^+ = \{(u_n, i_n, t_n) : d_n \in \mathcal{D}, z_n = 1\}, \quad (3.2)$$

where D denotes the dataset and protocol. For example, ψ_D may threshold a rating, threshold a stabilized watch ratio, or read an observed graph edge as positive. Equation (3.2) deliberately separates the dataset-native observation r_n from the constructed relevance label z_n , because changing ψ_D changes the empirical task.

The canonical event representation also stores a graded feedback sign $\gamma_n \in [-1, 1]$ when the raw data support one. This sign is not a second relevance label. The binary label z_n determines relevance, graph membership, BPR positives, and held-out evaluation targets. The sign records response direction or strength for sign-aware weighting and diagnostics; for example, a low watch-ratio record may have negative sign, while an unobserved item sampled during BPR training has no observed sign at all.

The target task is offline top- K recommendation. A model with parameters Θ learns a scoring function

$$\hat{y}_{ui} = f_{\Theta}(u, i, \mathcal{G}_{\text{train}}, \mathbf{x}_u, \mathbf{x}_i, \mathbf{c}_i), \quad (3.3)$$

where $\mathcal{G}_{\text{train}}$ is the training graph defined below and $\mathbf{c}_i$ denotes train-derived item context such as popularity or recency. For evaluation split $q \in \{\text{val}, \text{test}\}$, the candidate set for user u is

$$\mathcal{C}_{u,q} = \mathcal{I}_{\text{eval}} \setminus \mathcal{F}_{u,q}, \quad \mathcal{F}_{u,\text{val}} = \mathcal{O}_{u,\text{train}}, \quad \mathcal{F}_{u,\text{test}} = \mathcal{O}_{u,\text{train}} \cup \mathcal{O}_{u,\text{val}}, \quad (3.4)$$

where $\mathcal{I}_{\text{eval}}$ is the declared item universe, $\mathcal{F}_{u,q}$ is the seen-item exclusion set, and $\mathcal{O}_{u,q}$ denotes items observed for user u in split q regardless of whether their binary relevance label is positive. In positive-only graph datasets, $\mathcal{O}_{u,q}$ and $\mathcal{I}_{u,q}^+$ coincide; in datasets that retain non-positive observed records, the distinction prevents already observed non-positive interactions from re-entering the recommendation candidate set. The ranked output is the

ordered list

$$\pi_u^K(q) = \text{TopK}_{i \in \mathcal{C}_{u,q}} \hat{y}_{ui}. \quad (3.5)$$

Held-out relevance is computed only from positive labels in the target split. Thus, the thesis evaluates whether held-out positives are ranked highly among allowed candidates; it does not evaluate numerical rating reconstruction.

3.1.2 Split masks and bipartite training graph

Let $a_{n,q} \in \{0, 1\}$ denote the split mask for event n and split $q \in \{\text{train}, \text{val}, \text{test}\}$, with each retained canonical event assigned to exactly one split. Equation (3.6) defines the positive item set for user u in split q :

$$\mathcal{J}_{u,q}^+ = \{i_n \in \mathcal{I} : u_n = u, z_n = 1, a_{n,q} = 1\}. \quad (3.6)$$

The corresponding observed item set is

$$\mathcal{O}_{u,q} = \{i_n \in \mathcal{I} : u_n = u, a_{n,q} = 1\}, \quad (3.7)$$

which is used for seen-item exclusion in Equation (3.4). Thus, only positive split labels become relevance targets, but all previously observed split items can be masked from later recommendation lists when the dataset contains non-positive observations.

Only positive training labels define the propagation graph in Equation (3.8):

$$\mathcal{G}_{\text{train}} = (\mathcal{V}, \mathcal{E}_{\text{train}}), \quad \mathcal{V} = \mathcal{U} \cup \mathcal{I}, \quad \mathcal{E}_{\text{train}} = \{(u, i) : i \in \mathcal{J}_{u,\text{train}}^+\}. \quad (3.8)$$

Validation and test observations remain available for seen-item exclusion, and their positive labels define held-out relevance for metric computation, but they are not added to $\mathcal{E}_{\text{train}}$ before representation propagation. This convention is central to the leakage-control contract: validation and test records may define held-out labels and masks, but not graph neighborhoods or train-derived context.

For a branch using LightGCN propagation, Chapter 2 introduced normalized bipartite propagation and layer combination. The present chapter uses the same principle as a method-level graph contract: let $\mathbf{A}_{\text{train}}$ be the bipartite adjacency matrix of $\mathcal{G}_{\text{train}}$, $\mathbf{D}$ its degree matrix, and

$$\tilde{\mathbf{A}}_{\text{train}} = \mathbf{D}^{-1/2} \mathbf{A}_{\text{train}} \mathbf{D}^{-1/2}. \quad (3.9)$$

The compact train-graph propagation notation is

$$\mathbf{E}^{(\ell+1)} = \tilde{\mathbf{A}}_{\text{train}} \mathbf{E}^{(\ell)}, \quad \mathbf{Z} = \sum_{\ell=0}^L \alpha_\ell \mathbf{E}^{(\ell)}, \quad \sum_{\ell=0}^L \alpha_\ell = 1, \quad (3.10)$$

where $\mathbf{E}^{(0)}$ contains trainable user and item embeddings. The reference LightGCN baseline uses this full-graph computation directly, while EDGRec instantiates the same normalized-aggregation principle inside branch-specific sampled training.

3.1.3 Observed positives, unobserved pairs, and interpretation scope

Implicit-feedback data contain observed positives and missing entries, but missing entries are not true negatives. Chapter 2 reviewed this positive-only convention and the generic BPR objective. In this methods chapter, the consequence is operational: EDGRec and the baselines treat sampled negatives as contrast items rather than verified dislikes.

Training triples are written

$$\mathcal{B} \subseteq \{(u, i, j) : i \in \mathcal{I}_{u,\text{train}}^+, j \in \mathcal{I} \setminus \mathcal{I}_{u,\text{train}}^+\}. \quad (3.11)$$

Equation (3.11) states the mini-batch sampling domain used later by the EDGRec fused-score loss. The sampled item j is an unobserved contrast from the declared item universe.

The thesis treats all four datasets as observational for the main offline ranking experiments. KuaiRec and KuaiRand contain more explicit exposure-related structure than ordinary sparse logs, but using them here still requires protocol-specific interpretation. EDGRec is therefore described as informed by causal recommendation, popularity- and exposure-aware, and resource-aware. Its empirical scope is offline ranking behaviour, popularity-aware list behaviour, representation diagnostics, runtime, and peak VRAM for the declared data protocol.

The same interpretation governs supervision motivated by DICE. DICE motivates a distinction between interest and conformity in recommendation [5], while EDGRec observes logged interactions and train-derived popularity and context signals. Its branch losses and score components encourage separation under popularity-conditioned contrasts. Figure 3.2 states the empirical evidence surface: ranking, list behaviour, resources, and branch diagnostics are the reported outcomes; treatment effects, uplift, policy value, and latent-cause recovery are separate estimand-driven studies.

3.1.4 Resource-aware objective scope

The methodological objective is multidimensional. EDGRec is evaluated by ranking quality, popularity-aware personalization behaviour, runtime, and peak VRAM. Resource use is part of the thesis question because graph recommender comparisons can change when a model requires full-graph propagation, multiple embedding channels, extra negative sampling, or expensive validation scoring. A scientifically useful comparison therefore reads ranking, list behaviour, memory, and runtime together under the stated hardware budget.

Chapter 3 therefore defines component metrics first and then defines CRRU as a config-

urable post hoc utility for summarising one declared trade-off. Figure 3.1 summarizes the data-to-evaluation pipeline, and Figure 3.2 makes the associated empirical scope explicit. The pipeline figure is also a leakage-control diagram: labels are constructed before split masks are applied, only train-positive records form the propagation graph and train-derived context, and validation or test positives return only at the masked ranking stage.

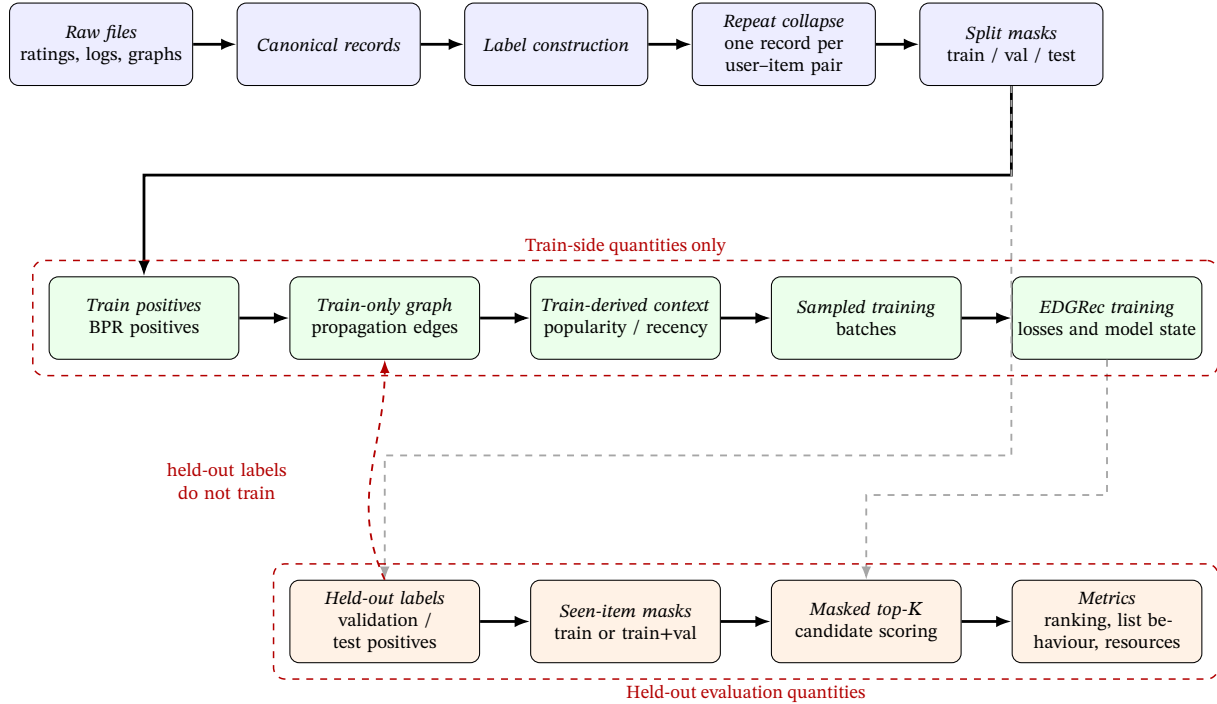


Figure 3.1: Data-to-evaluation pipeline. Label construction and repeat collapse precede splitting; only positive training labels create graph edges and train-derived context, while validation and test labels are reserved for held-out relevance, seen-item exclusion, and metric computation.

3.2 Data and Preprocessing Protocol

This section turns the formal task into dataset-specific preprocessing rules. It specifies dataset roles, positive labels, repeat handling, split construction, graph construction, item-universe policies, and feature leakage control.

3.2.1 Datasets, roles, and interpretation scope

The experiments use AmazonBook, MovieLens-1M, KuaiRec v2, and KuaiRand-1K. These datasets differ in feedback semantics, observation mechanism, side information, and exposure evidence, so the thesis interprets them as separate empirical regimes rather than as one pooled benchmark. Each dataset is assigned a methodological role, and each role determines what can be concluded from its results.

MovieLens is cited as an explicit rating dataset whose records contain a user, item, rating, and timestamp [12]. KuaiRec is cited for its distinction between a nearly fully

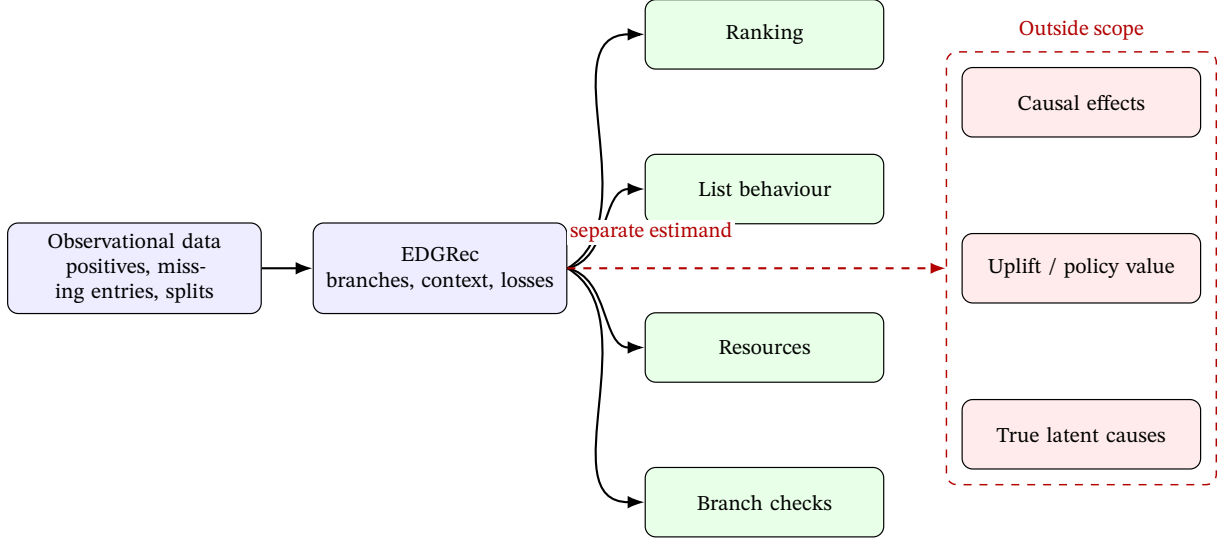


Figure 3.2: EDGRec empirical interpretation boundary. Under the declared protocol, EDGRec produces offline ranking, popularity and list behaviour, resource, and branch-diagnostic evidence. Treatment effects, uplift, counterfactual policy value, and observed ground-truth interest and conformity labels require a different empirical design.

observed small matrix and a larger sparse matrix [14]. KuaiRand is cited for randomized video exposure records collected to address exposure bias in sequential recommendation data [15]. AmazonBook is used as a sparse interaction-only benchmark in the LightGCN collaborative-filtering benchmark tradition [6]. MovieLens-20M and Taobao are retained as preprocessing-inventory assets because their scale placed them outside the training and complete-test resource budget.

Table 3.1 summarizes the role and interpretation scope assigned to each evaluated dataset view; evaluations from different views are not pooled into a single cross-dataset claim.

Figure 3.3 summarizes the same dataset-role argument without requiring the reader to parse detailed configuration labels. The figure is deliberately regime-level: it explains why AmazonBook, MovieLens-1M, KuaiRec v2, and KuaiRand-1K should be interpreted as different empirical settings rather than as one pooled benchmark. Its axes and annotations are descriptive setup evidence: they distinguish scale, density, feedback semantics, side-information availability, and exposure evidence, but they do not rank models or collapse the datasets into a single cross-dataset ranking.

The experiments use the dataset-specific preprocessing protocols in Table 3.2. Important alternatives are explicitly labelled protocol changes. A result produced under an alternative view is not merged into the default table without naming the changed protocol.

3.2.2 Dataset-specific canonicalization and positive labels

Each dataset protocol emits canonical arrays for user ID, item ID, label, timestamp, sign, split metadata, optional features, repeat-collapse summaries, exposure descriptors, and preprocessing-protocol metadata. Table 3.3 records the distinction between the binary label

Table 3.1: Dataset roles and interpretation scope

Dataset	Thesis role	Observational or exposure characteristics	Supported interpretation scope
AmazonBook	Sparse interaction-only product-interaction benchmark.	Pre-split user-item interaction lists; no primary side-feature or exposure-field evidence.	Tests whether EDGRec remains feasible and competitive on sparse collaborative graphs; its role is ranking, popularity-concentration, and resource comparison under interaction-only evidence.
MovieLens-1M	Explicit-rating benchmark converted into implicit top- K positives.	Ratings and timestamps express stated movie preferences, but the experiment thresholds them into binary relevance.	Supports claims about converted implicit ranking from explicit ratings, with the evaluation target defined as top- K relevance rather than rating reconstruction.
KuaiRec v2 big matrix	Default short-video watch-ratio ranking dataset.	Sparse large matrix with watch-ratio observations, timestamps, video metadata, and pronounced platform exposure/popularity structure.	Supports the main KuaiRec sparse-ranking analysis under declared watch-ratio thresholds and popularity-aware list behaviour diagnostics.
KuaiRec v2 small matrix	Sensitivity and dense-observation diagnostic.	Nearly fully observed user-video matrix designed for evaluation analysis; separate from the default sparse watch-ratio protocol.	Examines sensitivity to observation density and remains separate from the primary sparse thesis setting.
KuaiRand-1K	Compact short-video exposure-aware setting.	Standard logs plus randomized-exposure records, response fields, and safe video descriptors; randomized records make exposure sensitivity visible.	Supports ranking, list behaviour, resource, and randomized-exposure sensitivity analysis for its declared protocol. Item-universe differences limit cross-protocol conclusions.

Table 3.2: Dataset-specific preprocessing protocols used by the experimental pipeline

Dataset	Default thesis protocol	Labelled alternative protocols
AmazonBook	Sparse interaction-only positive-interaction protocol.	None in the primary thesis protocol.
MovieLens-1M	Explicit ratings converted to implicit top- K positives.	None in the primary thesis protocol.
KuaiRec v2	Sparse watch-ratio protocol with threshold 0.5.	Stricter sparse thresholds 0.75 and 1.0; separate dense-observation sensitivity protocol.
KuaiRand-1K	Exposure-aware standard-plus-randomized short-video protocol.	Randomized-exposure-only protocol.

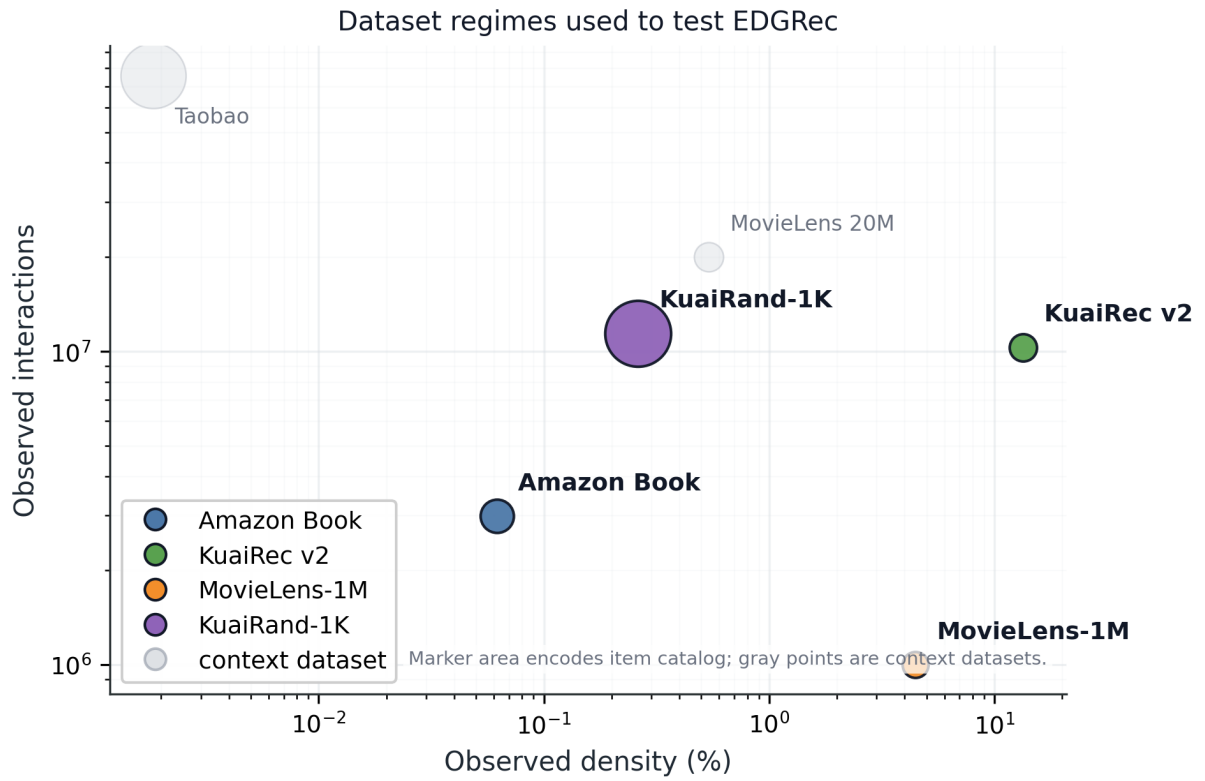


Figure 3.3: Dataset-regime map used to motivate the experimental design. The axes and annotations summarize scale, density, feedback semantics, and exposure evidence at a high level. The purpose is not to rank datasets by difficulty, but to show why each dataset supports a different interpretation scope: sparse interaction-only recommendation, rating-to-implicit conversion, sparse short-video watch-ratio recommendation, or randomized-exposure sensitivity analysis.

and the graded sign. This distinction matters for interpretation: observed negative-sign feedback is different from the sampled contrast items used by BPR, and neither should be described as a complete labelled negative class.

- *AmazonBook*: The interaction-only protocol reads the supplied train and test user–item lists and treats listed interactions as positive edges. Because no side features are part of the default contract, AmazonBook evaluates graph collaborative filtering and sampled training rather than feature-aware recommendation.
- *MovieLens-1M*: The explicit-rating protocol loads ratings, movie descriptors, user descriptors, and timestamps. The canonical label is obtained by thresholding the explicit rating according to the declared rating rule. The thesis reports this as converted implicit top- K ranking, not as prediction of the original star rating.
- *KuaiRec v2*: The default protocol uses the sparse watch-ratio view. The positive label is $\mathbb{I}(\text{watch_ratio} \geq 0.5)$ after invalid records have been removed or stabilized. Thresholds 0.75 and 1.0 are stricter sparse-protocol diagnostics. The small-matrix view is a separate dense-observation sensitivity protocol rather than the default ranking interpretation.
- *KuaiRand-1K*: The exposure-aware protocol keeps standard records and randomized-exposure records with exposure metadata. Canonical positives are derived from the configured short-video feedback rule over response fields such as click or long-view signals; non-positive exposures remain observed records but are not relevant held-out positives. The randomized-exposure-only view isolates randomized records, and the randomized-item-universe policy is a compact exposure-aware view rather than the standard-view result.

The evaluator uses positive relevance only: a record contributes to validation or test relevance when it is in the target split and its canonical label is positive. Non-positive observed records may provide exposure-related metadata, but they are not counted as relevant targets. This distinction is why the label and sign contract is reported separately: a positive label defines graph and ranking relevance, while the signed feedback descriptor remains a diagnostic field whose meaning depends on the dataset’s raw response signal.

3.2.3 Repeat interaction collapse and timestamp ordering

Raw logs may contain multiple records for the same user–item pair. The preprocessing protocol collapses repeats before final split construction so that one raw pair cannot contribute simultaneously to train, validation, and test labels. Let

$$\mathcal{D}_{ui} = \{d_n \in \mathcal{D} : u_n = u, i_n = i\} \quad (3.12)$$

Table 3.3: Canonical label and sign contract by dataset.

Dataset	Binary relevance label z_n	Graded sign γ_n	Interpretation rule
AmazonBook	Supplied graph interaction records are positive.	Always zero in the interaction-only protocol.	No observed negative or neutral feedback class is available.
MovieLens-1M	Rating at least 4 is positive.	$(\text{rating} - 3)/2$, so ratings below 3 are negative-sign, rating 3 is neutral, and ratings above 3 are positive-sign.	Converted implicit ranking from explicit ratings; non-positive ratings are not held-out relevant items.
KuaiRec v2	Watch ratio at least the declared threshold, 0.5 by default, is positive.	Watch ratio, clipped to $[0, 2]$, minus 1.	Sign records watch-ratio strength; label-positive records with watch ratio between 0.5 and 1.0 can still have negative sign.
KuaiRand-1K	The current KuaiRand-1K protocol treats click, like, or follow as positive.	Hate maps to -1.0 ; click plus long view to 0.3; follow to 0.7; like to 1.0; otherwise neutral.	Randomized and standard exposure records support feedback diagnostics, but relevance is still the binary label.

be the repeat group. The retained canonical record is selected by the split-retention priority declared for the dataset and then by timestamp tie-breaking; repeat summaries preserve count, mean response, maximum response, latest response, first and last timestamps, and available behaviour-count fields. If timestamps tie exactly, the protocol uses deterministic source order or stable record priority rather than random selection. This rule prevents a duplicate positive from leaking across splits while preserving enough repeat statistics for diagnostics.

Timestamp handling is similarly protocol-owned. Supplied timestamps are retained when available. If a dataset provides only a split file or no reliable event time, the protocol assigns deterministic pseudo-time sufficient for stable ordering and repeat collapse. Train-derived recency is always computed after the final training mask is known; validation and test timestamps are not allowed to alter training recency buffers.

3.2.4 Split construction, candidate filtering, and negative sampling

If a dataset supplies split masks, those masks are used. Otherwise, validation and test masks are derived according to the configured split rule after repeat collapse. Tiny diagnostic samples preserve split coverage, remap user and item IDs, slice aligned feature and propensity arrays, and recompute train-derived popularity after sampling.

At the data level, validation and test records are held-out observations. Validation records may be read by model-selection procedures, while test records are read only after the preprocessing protocol, item-universe policy, split rule, feature policy, and model configuration

are fixed. Neither validation nor test labels enter graph construction, train-derived context, feature encoding, negative-sampler positives, or training identity. The evaluation section later separates the same boundary by evidence role: validation can select, test can report.

Evaluation uses split-specific exclusion as defined in Equation (3.4). Full-catalog scoring over the declared item universe is the default interpretation unless a candidate-retrieval approximation is explicitly declared. Training uses sampled pairwise comparisons. For a positive training pair (u, i) , the negative sampler draws one or more contrast items j and filters them against the user’s known positive training items. EDGRec can use popularity-conditioned routing, motivated by DICE, for branch supervision; reference DICE keeps its own configured multi-negative and correction path. In all cases, a sampled j is an unobserved or non-positive contrast item, not a verified dislike.

3.2.5 Graph construction and train-derived context

The default graph policy is observed: graph edges, BPR positives, train-time popularity, and degree normalization use only positive training labels. The graph builder also stores the original observed split masks and label-aware positive masks for bookkeeping and seen-item exclusion. Edge signs are aligned only to the positive-label propagation edges that enter $\mathcal{G}_{\text{train}}$.

Thus records with $\text{sign} = 0$ or $\text{sign} < 0$ are not silently added as extra GCN propagation edges; they remain observed feedback evidence for interpretation unless they also satisfy the dataset’s positive-label rule. Figure 3.4 visualizes this contract on all four thesis datasets with a degree-aware positive-core sample and sign overlays. The visualization is a sampled topology and sign audit, not a complete graph drawing; it helps check whether the training graph is sparse, degree-skewed, and label/sign consistent before any model comparison is interpreted.

Let

$$p_i^{\text{train}} = \sum_{u \in \mathcal{U}} \mathbb{I}\{i \in \mathcal{J}_{u, \text{train}}^+\} \quad (3.13)$$

be the raw positive-train item count. The graph build stores p_i^{train} for AveragePopularity@ K , the mean training-popularity count of items appearing in top- K recommendation lists as defined in Section 3.6.3, a log-normalized copy for auxiliary popularity targets, and

$$p_{\max}^{\text{train}} = \max_{i \in \mathcal{I}} p_i^{\text{train}} \quad (3.14)$$

for CRRU’s recommendation-popularity denominator. Recent-history buffers and item recency are also computed from the final training split only. Validation and test interactions do not enter these tensors.

EDGRec GCN Training Topology

Each panel uses the same displayed train-row budget. Green marks label > 0 propagation rows; gray/red mark sign = 0 / sign < 0 feedback rows.

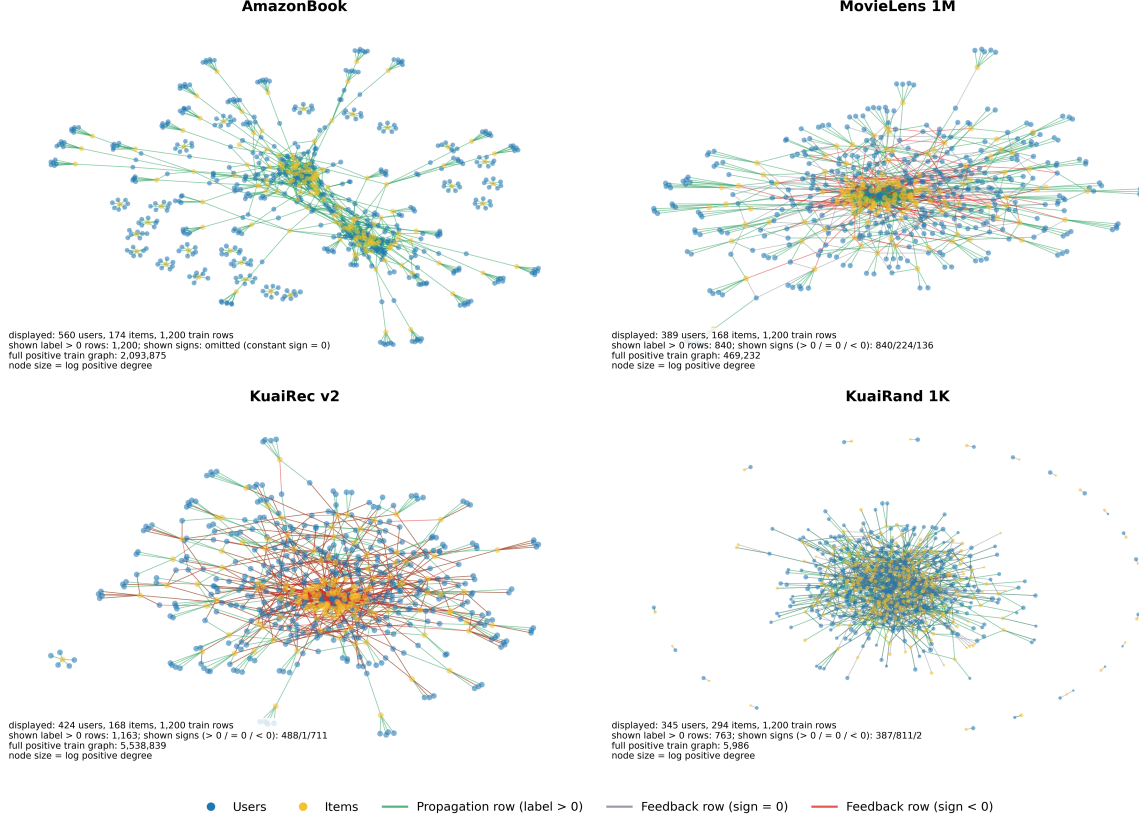


Figure 3.4: NetworkX spring-layout visualizations of degree-aware readable samples from the positive training core. Blue nodes are users and yellow nodes are items. Green edges are label-positive train interactions used for EDGRec and LightGCN propagation. The binary label is the graph-construction signal; the graded sign is a separate feedback diagnostic. Black and red overlays show sign = 0 and sign < 0 observed training records whose endpoints fall inside the same sampled core. Positive-sign records are counted in the figure metadata rather than overdrawn as a third overlay because they usually coincide with the green propagation graph. The overlays are diagnostic feedback records, not additional message-passing edges. A black or red overlay may coincide with a green propagation edge when the binary label is positive but the graded sign is non-positive. Node size is log-scaled positive train degree. Validation and test interactions are excluded from the graph and from the overlays. The full positive-train graph scale is reported in Figure 4.5.

3.2.6 Feature policy and leakage control

The primary thesis feature policy loads only structured descriptors treated as available before the prediction event. Post-treatment aggregates are excluded from ordinary model features because they can encode future engagement or exposure consequences rather than ex-ante item information. This policy is especially important for KuaiRec and KuaiRand, whose metadata files contain both stable video descriptors and engagement counters. The purpose of the policy is not to discard useful metadata; it is to ensure that any performance gain can be interpreted as a valid recommendation signal under the split protocol rather than as leakage from future outcomes. Table 3.4 records the feature classes, default use, and leakage-control rule applied in the experiments. Feature-subset evidence is defined as validation search evidence in Chapter 4 and reported as such in Chapter 5 rather than as final test evidence in this methods chapter.

Table 3.5 makes the column-level feature contract explicit. This is part of the methodology rather than supplementary bookkeeping: the same raw dataset can lead to different scientific claims depending on which columns are treated as pre-event descriptors, train-derived context, candidate propensity-calibration targets, or post-treatment leakage risks.

For retained structured features, numeric and temporal fields are min–max normalized, categorical fields are encoded deterministically with an explicit missing value, and list-valued category strings are multi-hot encoded. Repeated entity metadata is resolved deterministically before feature tensors are constructed. When feature fusion is enabled, item features are normalized, projected, and injected through learned gates initialized near zero. Side features are therefore a controlled ablation or dataset-specific configuration choice rather than an undeclared advantage. The context head may use train-derived popularity, train-derived recency, item age, safe item features, and optional propensity-calibration targets; it does not use validation or test outcomes or post-treatment engagement counters as default features.

3.3 Baseline Models

This section defines the reference methods used to interpret EDGRec. It first names the baseline taxonomy and then states the objective and propagation conventions that make comparisons meaningful.

3.3.1 Baseline and variant taxonomy

The baseline set is organized by evidence role rather than by a single aggregate ranking. Bayesian Personalized Ranking Matrix Factorization (BPR-MF) isolates the pairwise ranking objective without graph propagation when a matched complete evaluation is available. Reference LightGCN isolates full-graph normalized collaborative propagation. Reference

Table 3.4: Feature policy and leakage-control rules

Feature class	Examples	Default use	Leakage-control rule
Safe pre-treatment descriptors	Movie genre; KuaiRec and KuaiRand author and music IDs, video type and status, duration, resolution, upload time, category or tag IDs.	Optional model features under the declared dataset configuration; AmazonBook remains interaction-only.	Normalized and encoded before feature fusion; used only when available without validation or test outcomes.
Train-derived context	Positive-train popularity p_i^{train} , log-normalized popularity, train-only item recency, recent train history.	Context head, popularity auxiliary target, popularity-conditioned masks, Average-Popularity@K, CRRU denominator.	Recomputed from the final training split only; never recomputed from validation or test records.
Optional diagnostic features	Feature subsets, temporal-interest buffers, candidate propensity-calibration inputs, stricter KuaiRec thresholds, small-matrix or randomized-only views.	Explicit ablations or sensitivity analyses.	Require explicit protocol/configuration labels and comparison against the default contract.
Excluded post-treatment aggregates	KuaiRec and KuaiRand play counts, play duration aggregates, complete-play counts, likes, comments, follows, shares, downloads, reports, collect/cancel counters, statistic files as ordinary features.	Excluded from primary scoring features.	Not used as ordinary predictors because they can encode engagement after exposure.
Propensity-calibration-only fields	KuaiRand show-count field from statistic features.	Zero-filled in the default scorer; available only for explicit propensity-calibration or inverse-propensity-weighting (IPW) ablations.	May supply an experimental calibration target when the calibration loss and IPW path are explicitly enabled; treated neither as an observed true exposure propensity nor as input to item cardinality or default scores.

Table 3.5: Feature-lineage and encoding contract for the primary thesis datasets.

Dataset	Retained raw sources and columns	Encoded groups	Default status and exclusion rule
Amazon-Book	Supplied training and test user-item lists; no side-feature table is part of the primary thesis contract.	Graph-only interaction signal; train-positive popularity and recency are computed only after the final training split is known.	Graph-only collaborative-filtering benchmark; no side-feature effect is claimed.
MovieLens-1M	The rating table supplies ratings and timestamps for label and split construction; the movie table supplies genre descriptors; user demographics are available for audit but are not used by the current EDGRec scorer.	Movie genre as a multi-hot item group; user demographics are not part of the default scorer; zip code is treated as proxy-only.	Genre is opt-in and validation evidence currently favours no side features, so formal MovieLens claims are not side-feature claims.
KuaiRec v2	Sparse watch-ratio interactions supply clipped labels; safe item descriptors include author and music IDs, video type, duration, resolution, upload time, upload type, visibility, and category or tag sources.	Author and music, video metadata, resolution, upload-time, and category or tag feature groups.	Safe item descriptors are allowed only in named configurations; engagement counts, free text, and post-treatment aggregates are excluded from primary scoring features.
KuaiRand-1K	Standard and randomized exposure logs supply labels and exposure metadata; the basic video-descriptor table supplies author and music IDs, status, type, duration, resolution, upload fields, and tags.	Author and music, video metadata, resolution, upload-time, and category or tag feature groups.	Basic descriptors are named-configuration features; statistic and engagement aggregates are excluded as ordinary model inputs, with show-count information reserved only for explicit propensity-calibration ablations.

DICE and its Graph Convolutional Network instantiation denote the interest and conformity comparators derived from DICE, but each such comparison is interpreted according to its feasibility or completion status. Sampled LightGCN and popularity-conditioned EDGRec ablations test which components matter under the resource-aware training regime. Table 3.6 gives the taxonomy used in the thesis.

Table 3.6: Baseline and variant taxonomy

Family	What it keeps fixed	What it excludes or changes	Evidence role
BPR-MF	User and item embeddings, inner-product score, and BPR pairwise objective.	No graph propagation, no branch separation, no context head, no side-feature fusion.	Non-graph reference for sampled implicit-feedback ranking [3].
Reference LightGCN	Full-graph LightGCN propagation over the positive training graph, layer combination, and the simplified graph scoring specified by the source model.	No feature transformations, nonlinear activations, side features, EDGRec score-fusion module, or branch-specific losses.	Main graph collaborative-filtering baseline [6].
Reference DICE and its GCN instantiation	Separate interest and conformity branches, popularity-conditioned negatives, branch losses, and discrepancy term when the reference configuration and memory budget permit them.	Uses the reference DICE training contract instead of EDGRec’s sampled-training regime, context head, and learned score fusion.	Literature comparator for interest and conformity separation; final ranking baseline only when complete evaluations exist [5].
Sampled LightGCN ablations	EDGRec data protocol, sampled subgraphs, vectorized negatives, and interest-only scoring.	Conformity branch and context head disabled or fixed out of the score.	Tests whether EDGRec gains come from sampled graph training rather than disentanglement.
Popularity-conditioned EDGRec ablations	EDGRec branch infrastructure with interest and conformity losses or fixed branch fusion.	Uses EDGRec sampling, optimization, and scoring conventions rather than the reference DICE contract or its GCN instantiation.	Tests branch supervision under the efficient EDGRec training regime.
EDGRec variants	Sampled graph propagation, interest and conformity branches, context head, score fusion, and optional feature or propensity ablations.	Restricts claims to observed ranking, list behaviour, resource use, and enabled component diagnostics under the declared protocol.	Proposed method family for ranking, popularity-aware behaviour, runtime, and memory trade-offs.

3.3.2 Reference objectives and propagation

The matrix-factorization baseline scores $s_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$ and optimizes the BPR objective introduced in Chapter 2. It provides a lower-complexity reference that separates the value of

graph propagation from the value of the pairwise implicit-feedback objective. The LightGCN baseline scores $s_{ui} = \mathbf{z}_u^\top \mathbf{z}_i$ after the train-graph propagation in Equation (3.10). It provides the main graph collaborative-filtering comparison because it keeps normalized user-item propagation while avoiding extra neural transformations [6].

The reference DICE comparison and its GCN instantiation are kept distinct from EDGRec. They use the DICE interest and conformity architecture, popularity-conditioned negative sampling, and branch/discrepancy losses in the reference specification. EDGRec adopts the interest and conformity motivation, but it is an efficient graph recommendation method with sampled subgraphs, bounded auxiliaries, context scoring, and a different score fusion contract. Therefore, any result labelled as reference DICE or as the GCN instantiation of DICE refers to that reference specification, while a popularity-conditioned EDGRec ablation refers only to a component test within EDGRec’s training contract.

Table 3.7 states the distinction explicitly because DICE is the closest conceptual comparator. The table identifies which part of the source framework is used as scientific motivation and which parts are changed by EDGRec to support the resource-aware graph recommendation protocol studied in this dissertation. Consequently, an EDGRec result is evidence for the EDGRec operationalization of the branch hypothesis; a direct DICE result requires a reference DICE configuration completed under the same evidence contract.

3.4 EDGRec Architecture

EDGRec is introduced after the data protocol and baseline taxonomy because its components are meaningful only relative to the constructed graph, split-safe context, and sampled contrast convention. Chapter 2 motivates the view that observed interactions may mix personal-interest, conformity, exposure, and popularity-related behaviour. EDGRec maps that motivation into an operational dual graph scorer: one branch emphasizes interest-oriented structure in the train-only bipartite graph, a second branch emphasizes conformity-oriented structure, and an item-only context head remains separate from both propagation branches.

This separation is deliberate. It keeps collaborative, popularity-conditioned, and item-context signals in distinguishable score channels, which makes later ablations and diagnostics interpretable without treating any branch as an observed causal variable. It also marks the methodological difference from DICE: EDGRec inherits the interest-conformity motivation, but it changes the computational setting to train-only graph propagation, sampled neighbourhood training, controlled score fusion, and resource-aware reporting. The branch labels denote modelling roles rather than directly observed causal variables; their usefulness is assessed through ranking, list behaviour, resource, and diagnostic evidence. The design therefore answers a practical question that the source DICE formulation does not settle for

Table 3.7: Methodological distinction between reference DICE and EDGRec.

Dimension	Reference DICE	EDGRec contribution
Scientific role	Provides the interest and conformity representation hypothesis, popularity-conditioned branch supervision, and discrepancy regularization for disentangled causal embeddings [5].	Uses that hypothesis as a modelling guide for an efficient graph recommender; empirical claims are ranking, list behaviour, resource, ablation, and diagnostic claims, not identified causal-effect claims.
Graph backbone	Can integrate multiple backbone recommendation models in the source framework, including matrix-factorization and graph variants.	Fixes a propagation contract derived from LightGCN for the thesis method, with separate interest and conformity propagation depths and train-only graph construction.
Training regime	Follows the reference DICE training contract, including cause-specific sampling logic, branch losses, discrepancy supervision, and full reference-baseline semantics.	Uses sampled subgraph training, vectorized popularity-conditioned routing, bounded auxiliary losses, automatic batch-size metadata, and validation-to-test evidence gates.
Scoring	Combines interest and conformity branch scores according to the source framework.	Separates raw branch scores for losses from normalized branch logits for final fusion, adds an item-only context path, and controls active score weights through fixed or learned fusion.
Evaluation contract	Serves as the literature comparator for interest and conformity separation when complete matched evaluations are available.	Reports component-visible top- K ranking, AveragePopularity, Personalization, runtime, peak VRAM, CRRU, branch diagnostics, and matched ablations under explicit dataset and hardware protocols.

this thesis: whether the branch hypothesis remains useful when embedded in a split-safe, sampled graph recommender with a measured workstation resource budget.

Operationally, EDGRec is a graph recommender that uses Light Graph Convolutional Network (LightGCN) propagation with separate interest and conformity channels, an optional item-only context head, and a fused score used for top- K ranking. Figure 3.5 gives the computational schematic: it shows how train-only graph propagation, branch scores, context scoring, fusion, and resource diagnostics connect. At a high level, EDGRec separates raw branch scores used by branch-shaping losses from normalized or bounded logits used by the final ranking fusion layer. This separation is necessary because branch losses should train the representations, whereas the final ranker should not favour a channel merely because its raw dot products have a larger scale. Equation (3.15) writes this score decomposition as

$$s_{ui}^I = h_I(u, i, \mathcal{G}_{\text{train}}), \quad (3.15a)$$

$$s_{ui}^C = h_C(u, i, \mathcal{G}_{\text{train}}), \quad (3.15b)$$

$$r_i^X = g_X(\mathbf{c}_i), \quad (3.15c)$$

$$\ell_{ui}^I = \text{norm}(s_{ui}^I), \quad \ell_{ui}^C = \text{norm}(s_{ui}^C), \quad \ell_i^X = \tanh(r_i^X), \quad (3.15d)$$

$$\hat{y}_{ui} = \eta_{ui}^I \ell_{ui}^I + \eta_{ui}^C \ell_{ui}^C + \eta_{ui}^X \ell_i^X, \quad \sum_{r \in \{I, C, X\}} \eta_{ui}^r = 1. \quad (3.15e)$$

Here I denotes the interest branch, C denotes the conformity branch, and X denotes the context and popularity head. The first two scores are user-item collaborative branch scores, the third is an item-only context score, and the final line states the ranking score after scale control and fusion. The operator $\text{norm}(\cdot)$ denotes the configured norm-invariant branch logit used for final fusion, while the raw branch scores remain available for branch losses and diagnostics. The learned or fixed weights η_{ui}^r implement the configured fusion rule; variants disable components by setting the corresponding path inactive or assigning fixed weights. In words, EDGRec first computes separate evidence channels, then places them on compatible bounded scales, and only then learns or fixes their contribution to the final recommendation score.

Chapter 2 introduced the structural-causal-model motif in Figure 2.10; in this chapter, Figure 3.5 is the retained method schematic. The popularity-conditioned branch supervision is described in prose and equations rather than as a separate diagram, because the important scientific point is the routing rule and its interpretation scope rather than the visual arrangement.

Figure 3.5 clarifies EDGRec’s practical adaptation of branch vocabulary motivated by DICE. The method retains interest and conformity roles while adding sampled subgraph training, bounded auxiliary losses, a context score, and explicit resource diagnostics. These design choices are part of the thesis contribution and must be named when comparing

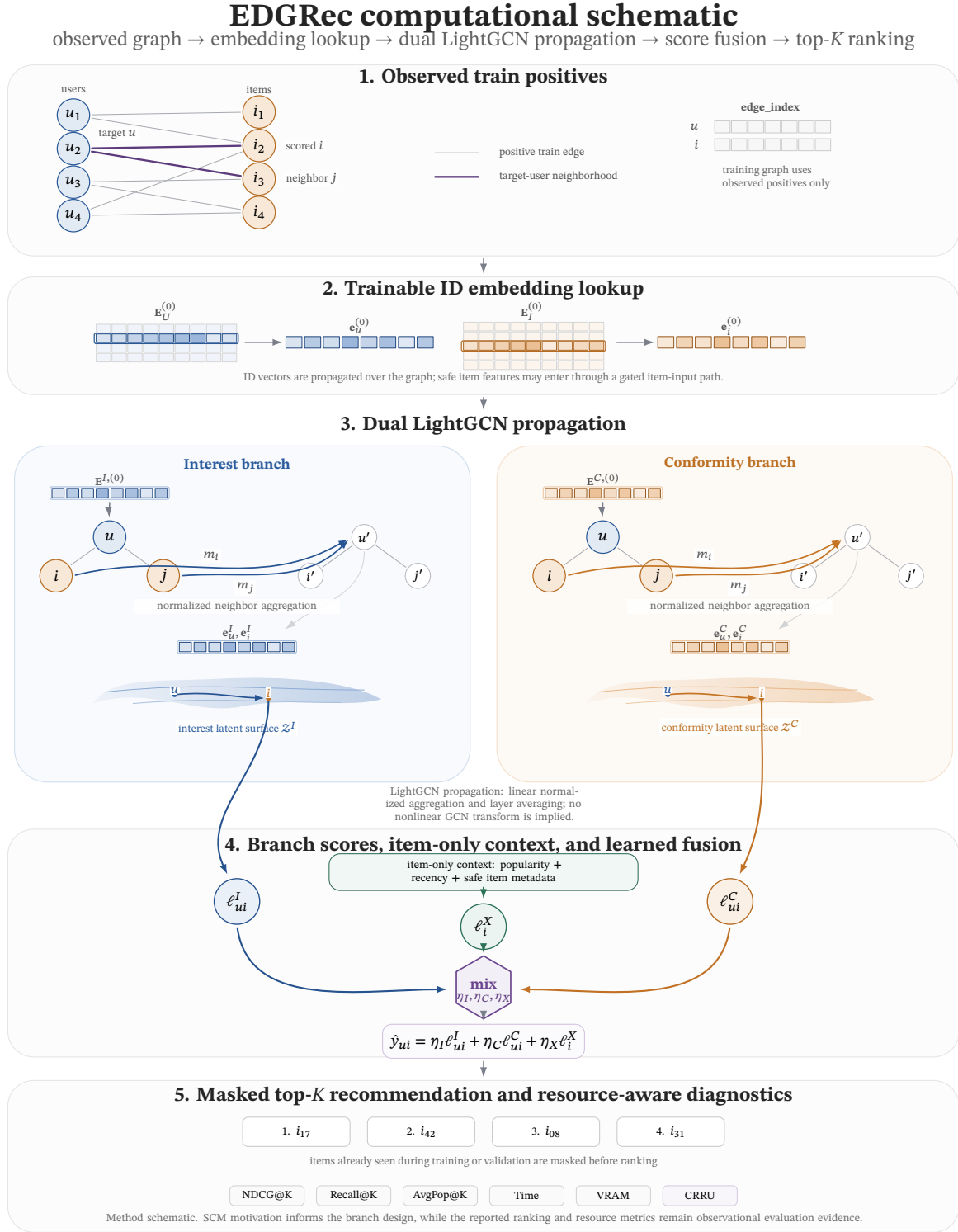


Figure 3.5: Computational schematic of the EDGRec method. Positive training interactions define the train-only graph; user and item identifier embeddings, with optional gated safe item features, feed two propagation branches that use LightGCN aggregation. The interest and conformity branch scores are fused with an item-only context score to produce split-masked top- K recommendations. Popularity-conditioned negative samples and auxiliary losses shape the branch roles, while the reported metrics and resource diagnostics summarize empirical behaviour. The figure is an operational schematic rather than an estimand diagram.

EDGRec with full-graph LightGCN or reference DICE baselines.

3.4.1 Embedding layer

EDGRec begins from trainable user and item identifier embeddings and then forms branch-specific propagation states. The embedding layer is kept simple for the same reason LightGCN removes unnecessary graph-convolutional transformations: the thesis wants graph propagation, branch supervision, and resource controls to be the interpretable sources of difference. In the default thesis configuration, the item identifier table can be shared before the two graph-propagation views are constructed; explicit branch-specific item tables are treated as a capacity ablation rather than as a requirement of the method. For notational clarity, let

$$\mathbf{e}_u^I, \mathbf{e}_i^I, \mathbf{e}_u^C, \mathbf{e}_i^C \in \mathbb{R}^d \quad (3.16)$$

denote the initial branch-state inputs for user u and item i . Optional item-feature projections may be added to item states under the feature gate described in Section 3.2.6. Train-derived metadata buffers store item popularity, recency, optional item age, optional safe feature encodings, and optional propensity-calibration targets. These buffers are modelling inputs and diagnostics rather than supervision labels for branch meanings or evidence that true exposure propensities are observed.

3.4.2 Interest branch

The interest branch propagates embeddings over the training graph to model collaborative preference structure. A LightGCN propagation layer can be written as

$$\mathbf{e}_u^{I,(\ell+1)} = \sum_{i \in \mathcal{N}_{\text{train}}(u)} \frac{\omega_{ui}^I}{\sqrt{d_u d_i}} \mathbf{e}_i^{I,(\ell)}, \quad \mathbf{e}_i^{I,(\ell+1)} = \sum_{u \in \mathcal{N}_{\text{train}}(i)} \frac{\omega_{ui}^I}{\sqrt{d_i d_u}} \mathbf{e}_u^{I,(\ell)}, \quad (3.17)$$

where d_u and d_i are training-graph degrees and ω_{ui}^I is an edge weight that reduces to 1 in the basic positive-edge setting. The final interest embedding is a layer combination

$$\mathbf{z}_u^I = \sum_{\ell=0}^{L_I} \alpha_\ell^I \mathbf{e}_u^{I,(\ell)}, \quad \mathbf{z}_i^I = \sum_{\ell=0}^{L_I} \alpha_\ell^I \mathbf{e}_i^{I,(\ell)}. \quad (3.18)$$

The raw interest score used in branch losses and diagnostics is

$$s_{ui}^I = (\mathbf{z}_u^I)^\top \mathbf{z}_i^I. \quad (3.19)$$

For final ranking fusion, EDGRec uses the corresponding norm-invariant interest logit

$$\ell_{ui}^I = \frac{(\mathbf{z}_u^I)^\top \mathbf{z}_i^I}{\|\mathbf{z}_u^I\|_2 \|\mathbf{z}_i^I\|_2 + \epsilon}, \quad (3.20)$$

where $\epsilon > 0$ is a numerical stabilizer. This distinction is methodological rather than cosmetic: raw dot products remain available to branch-shaping losses, while normalized logits keep the final fusion layer from favouring a branch merely because its embedding norms are larger.

3.4.3 Conformity branch

The conformity branch uses a parallel embedding channel intended to capture popularity- or conformity-aligned collaborative structure. Its propagation has the same LightGCN propagation form with branch-specific parameters and optional edge weights:

$$\mathbf{z}_u^C = \sum_{\ell=0}^{L_C} \alpha_\ell^C \mathbf{e}_u^{C,(\ell)}, \quad \mathbf{z}_i^C = \sum_{\ell=0}^{L_C} \alpha_\ell^C \mathbf{e}_i^{C,(\ell)}, \quad (3.21)$$

with raw branch score

$$s_{ui}^C = (\mathbf{z}_u^C)^\top \mathbf{z}_i^C. \quad (3.22)$$

The final scorer uses the norm-invariant conformity logit

$$\ell_{ui}^C = \frac{(\mathbf{z}_u^C)^\top \mathbf{z}_i^C}{\|\mathbf{z}_u^C\|_2 \|\mathbf{z}_i^C\|_2 + \epsilon}. \quad (3.23)$$

Its empirical meaning is assessed through ranking, popularity-aware list behaviour, personalization, and branch diagnostics. The branch is a learned component encouraged by popularity-conditioned losses and independence regularization rather than a directly observed conformity variable.

3.4.4 Context and popularity head

The context head is item-only. Let $\mathbf{c}_i$ collect train-derived context features such as log-normalized training popularity, recency, item age, and optional safe item features. Calibrated propensity targets are kept behind a separate opt-in propensity/IPW gate and enter the default context path only as zero-filled inactive inputs. The context head is included to expose item-level popularity and catalogue context inside the scoring model instead of forcing the propagation branches to absorb those signals implicitly. This design choice matters because item popularity can improve short-run ranking while also increasing list concentration; separating the context path makes that trade-off visible in ablations and diagnostics. Because the head has no user-specific input, its contribution is interpreted as item-context evidence within the fused score rather than as a standalone personalization or debiasing mechanism.

The context score is

$$r_i^X = g_\phi(\mathbf{c}_i), \quad (3.24)$$

where g_ϕ is a small trainable head. The fused scorer uses the bounded context logit $\ell_i^X = \tanh(r_i^X)$ so that the item-only path contributes on a controlled scale. Its role is to supply weak popularity or context evidence to the learned score fusion while the collaborative branches remain the user-specific scoring channels.

3.4.5 Score fusion

EDGRec combines normalized branch logits and the bounded context logit through a learned fusion module. Fusion is necessary because the branches and context head have different roles and score scales: the interest branch carries the main collaborative relevance signal, the conformity branch captures popularity-conditioned branch evidence, and the context head supplies item-level train-derived context. Normalizing and bounding these logits makes the fused score auditable as a controlled score combination rather than as an accidental consequence of raw vector norms. Let

$$\eta_{ui} = (\eta_{ui}^I, \eta_{ui}^C, \eta_{ui}^X), \quad \eta_{ui}^I, \eta_{ui}^C, \eta_{ui}^X \geq 0, \quad \sum_{r \in \{I, C, X\}} \eta_{ui}^r = 1, \quad (3.25)$$

be the score fusion weights after any configured floor or normalization. The fused recommendation score is

$$\hat{y}_{ui} = \eta_{ui}^I \ell_{ui}^I + \eta_{ui}^C \ell_{ui}^C + \eta_{ui}^X \ell_i^X. \quad (3.26)$$

Fixed-weight variants are obtained by setting one or more η terms to constants. For example, a single-branch variant using LightGCN propagation sets the interest component to one and disables conformity and context. These variants are useful because they make the comparison question explicit: a component is credited only when the matched evaluation improves visible ranking, list behaviour, or resource metrics relative to a configuration in which that component is absent or fixed.

3.4.6 Computational design choices

EDGRec is designed to support sampled subgraph training, bounded fan-out, vectorized negative sampling, cached train-derived metadata, and chunked evaluation. These choices are methodological rather than merely implementation-level details: they determine which part of the graph is active in each optimizer step, which train-only quantities can enter the loss, and which resource costs are comparable with full-graph baselines. Reference LightGCN and reference DICE use full-graph training where required by their baseline

contracts.

EDGRec’s sampled mode extracts local k -hop subgraphs for sampled triples. This reduces the active training working set and enables larger feasible batches, but it also makes batch size, fan-out, and negative-sampling policy part of the experimental configuration. Consequently, EDGRec can be faster per epoch despite having more score components: the dominant cost is often the amount of graph propagation, negative sampling, auxiliary-loss work, and candidate scoring executed per optimizer step, not the number of conceptual branches in the model diagram. The expected benefit is therefore systems-level efficiency under a declared training regime, not a claim that dual-branch scoring is intrinsically cheaper than a single LightGCN score.

Table 3.8 records the current implementation boundary used by the experiments. The table is intentionally method-level rather than a low-level implementation inventory: it states which model component owns each scientific role and which interpretation is not supported by that component alone. This guards against treating implementation affordances, such as feature gates or propensity heads, as active evidence when the corresponding configuration has not enabled them.

Algorithm 1 states the same methodological order after the objective is defined: construct split-safe labels and graph edges, sample popularity-conditioned triples where configured, optimize the weighted loss, retain the best validation state, and report final test metrics with the correct seen-item mask.

3.5 Training Objective

This section explains how EDGRec is optimized once the architecture is fixed. It introduces the main Bayesian Personalized Ranking (BPR) loss, branch-specific shaping terms, independence regularization, optional ablations, and validation-guided stopping.

3.5.1 Main BPR loss

For each sampled triple (u, i, j) with $i \in \mathcal{J}_{u,\text{train}}^+$ and sampled contrast item j , the fused-score BPR margin is

$$\Delta_{uij} = \hat{y}_{ui} - \hat{y}_{uj}. \quad (3.27)$$

The main recommendation loss is

$$\mathcal{L}_{\text{rec}} = -\frac{1}{|\mathcal{B}|} \sum_{(u,i,j) \in \mathcal{B}} \log \sigma(\Delta_{uij}), \quad (3.28)$$

where $\mathcal{B}$ is the current mini-batch of triples. In words, the loss rewards the model when

Table 3.8: Current EDGRec implementation contract used by the thesis experiments.

Implementation surface	Current role in EDGRec	Evidence boundary
Embedding and metadata layer	Builds user and item identifier embeddings, optional side-feature projections, train-only popularity and recency buffers, recent-train user histories, and optional propensity-calibration targets. Default EDGRec shares the item table before branch propagation; branch-specific item tables are a capacity variant.	Metadata buffers are split-safe model inputs or diagnostics; causal or propensity interpretations require explicit calibrated evidence.
Dual propagation derived from LightGCN	Runs interest and conformity propagation with explicit branch depths and precomputed train-graph normalization. EDGRec uses chunked edge-list aggregation by default, while reference baselines keep their full-graph contracts.	Branch labels describe modelling roles and are evaluated through ranking, list behaviour, resource, and diagnostic evidence.
Scoring and context layer	Produces raw branch scores for branch losses, norm-invariant branch logits, bounded item-only context logits, learned score fusion weights, and final fused scores.	The final ranking score is scale-controlled across active channels; raw branch scores remain diagnostic and loss-facing.
Optional propensity path	Uses an item-side propensity estimator only when calibration targets and positive calibration weight are available. The default scorer zero-fills this path.	Inverse propensity weighting (IPW) supports exposure-correction interpretation only for explicitly calibrated runs with matching targets.
Training and evaluation runtime	Uses one mini-batch trainer, sampled subgraph batches for EDGRec, full-graph training for reference LightGCN and DICE contracts, validation-best checkpointing, and full-graph propagated states for evaluation.	Runtime and peak VRAM are properties of the declared training regime, fan-out, batch policy, and hardware, not architecture-intrinsic constants.

the fused score of the observed positive item exceeds the fused score of the sampled contrast item. It is therefore the main relevance-shaping term for EDGRec: it aligns training with top- K ordering while preserving the interpretation that sampled contrasts are unobserved items, not verified negative labels.

3.5.2 Branch-specific losses

The branch-specific losses use the same sampled triples but condition the supervision on relative train popularity. Their purpose is to give the two branches different training pressure: the interest branch is encouraged to keep the observed positive above a more popular contrast, while the conformity branch is allowed to represent the popularity-aligned alternative. Let $\delta_{\text{pop}} \geq 0$ denote the configured popularity margin used by the sampler or loss fallback, and define

$$q_{ij} = \mathbb{I}(p_j^{\text{train}} - p_i^{\text{train}} > \delta_{\text{pop}}) \quad (3.29)$$

to indicate that the sampled contrast item is sufficiently more popular than the observed positive item. Setting $\delta_{\text{pop}} = 0$ recovers the schematic DICE relative-popularity rule, while the implemented sampler can use a stricter margin and pass the resulting mask directly to the loss layer. Let Z_I denote the configured reduction denominator for the interest branch: $Z_I = |\mathcal{B}|$ under batch-mean reduction, and $Z_I = \max(1, \sum_{(u,i,j) \in \mathcal{B}} q_{ij})$ under active-mask reduction. The corresponding interest loss is then active on these popularity-dominated negatives:

$$\mathcal{L}_I = -\frac{1}{Z_I} \sum_{(u,i,j) \in \mathcal{B}} q_{ij} \log \sigma(s_{ui}^I - s_{uj}^I). \quad (3.30)$$

This term expresses the rule that, when an unobserved contrast item is more popular than the observed positive, the interest branch should still rank the observed positive higher. It is intended to preserve interest-oriented evidence in cases where popularity alone would favour the contrast item.

The conformity branch reverses the ordering for popularity-dominated negatives and otherwise follows the positive-over-negative direction. Its schematic batch-mean form is

$$\mathcal{L}_C = -\frac{1}{|\mathcal{B}|} \sum_{(u,i,j) \in \mathcal{B}} [q_{ij} \log \sigma(s_{uj}^C - s_{ui}^C) + (1 - q_{ij}) \log \sigma(s_{ui}^C - s_{uj}^C)]. \quad (3.31)$$

In words, this term lets the conformity branch prefer the more popular contrast when the popularity gap is large, but otherwise keeps the ordinary positive-over-contrast direction. These losses are branch-shaping terms: they impose a popularity-conditioned training signal on the two score channels, while the empirical interpretation remains tied to ranking, list behaviour, resource, and diagnostic evidence. The mask is therefore a design rule derived

from train-only popularity, not an observation of conformity or exposure propensity.

3.5.3 Disentanglement and independence regularization

EDGRec includes an independence term to discourage trivial branch collapse. Without such a term, two branches can learn nearly identical embeddings, making branch-specific supervision difficult to interpret. A cosine-squared version over active batch users and items is

$$\mathcal{L}_{\text{ind}} = \frac{1}{|\mathcal{V}_{\mathcal{B}}|} \sum_{v \in \mathcal{V}_{\mathcal{B}}} \left(\frac{(\mathbf{z}_v^I)^\top \mathbf{z}_v^C}{\|\mathbf{z}_v^I\|_2 \|\mathbf{z}_v^C\|_2} \right)^2, \quad (3.32)$$

where $\mathcal{V}_{\mathcal{B}}$ is the set of active users and items in the batch. In words, the term penalizes high cosine similarity between the two branch representations for the same active entity, so the branches are discouraged from collapsing into the same embedding space. Reference DICE may instead use its configured distance-correlation discrepancy. In both cases, the term is representation regularization rather than causal validation.

3.5.4 Optional geometry, propensity, or feature ablations

Optional losses are treated as declared ablations rather than as default evidence. A context popularity loss can supervise the item-only context head against a train-derived popularity target:

$$\mathcal{L}_{\text{pop}} = \frac{1}{|\mathcal{B}_I|} \sum_{i \in \mathcal{B}_I} (r_i^X - \tilde{p}_i^{\text{train}})^2, \quad (3.33)$$

where $\tilde{p}_i^{\text{train}}$ is log-normalized training popularity and $\mathcal{B}_I$ is the batch item set. In words, the head is trained to reproduce a known train-only item-context signal, so popularity information is represented in a named path rather than being forced into the branch embeddings. This loss makes the context head learn a declared train-derived signal rather than an implicit mixture of branch information. It is included because item popularity is empirically important in recommendation logs, but it is kept separate from the final relevance claim: a lower or higher popularity score is a list behaviour diagnostic unless it also improves ranking under the declared protocol.

Contrastive terms, DirectAU-motivated alignment/uniformity, explicit propensity-calibration or inverse propensity weighting (IPW) components, temporal-history scoring, and side-feature fusion are used only when their weights or configurations are explicitly enabled, so their evidence role is ablation-specific. In particular, propensity calibration requires a positive calibration weight, model propensity scores, and batch propensity targets; otherwise the recommendation loss is not reweighted by uncalibrated inverse propensities.

3.5.5 Optimization and early stopping

The total loss for an EDGRec run is

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_I(t)\mathcal{L}_I + \lambda_C(t)\mathcal{L}_C + \lambda_{\text{ind}}(t)\mathcal{L}_{\text{ind}} + \lambda_{\text{pop}}(t)\mathcal{L}_{\text{pop}} + \sum_r \lambda_r(t)\mathcal{L}_r + \lambda_2\|\Theta\|_2^2, \quad (3.34)$$

where t is the epoch and $\mathcal{L}_r$ denotes optional ablation losses. In words, the total objective is a weighted sum of the main ranking loss, branch-shaping terms, branch-separation regularization, context supervision, optional ablation losses, and weight decay. The epoch-dependent weights make the objective a training contract: the main BPR term remains the primary ranking signal, while branch, independence, context, and optional terms can be introduced with declared weights and schedules.

The default EDGRec configuration uses the fused-score BPR loss, interest and conformity branch BPR terms motivated by DICE, independence regularization, and context popularity supervision with a linear-ramp schedule. Its branch masks, also motivated by DICE, use active-mask reduction by default so sparse popularity-dominated masks do not erase branch supervision. Contrastive, DirectAU, IPW, propensity-calibration, temporal-interest, and feature-fusion contributions are implemented but inactive unless a named configuration enables them.

EDGRec uses AdamW where applicable, while reference baselines use their configured optimizers. Validation metrics are computed at configured intervals; the best validation state is retained for final evaluation. Hyperparameter search and early stopping are validation-only. Algorithm 1 summarizes the split-safe training and evaluation order, and Figure 3.6 summarizes how the score components and auxiliary losses enter the weighted objective. The figure should be read as a bookkeeping aid for the objective: the fused score supplies the main BPR ranking loss, auxiliary terms shape branches or context when enabled, and ablation losses enter only through explicitly named configurations.

3.6 Evaluation Protocol

This section defines how runs are measured and compared. It covers the validation–test boundary, ranking metrics, list behaviour diagnostics, resource reporting, CRRU, and the limits of aggregate utility.

3.6.1 Validation-test boundary and evidence roles

Validation and test splits have different scientific roles. Validation metrics may drive hyperparameter search, early stopping, feature subset screening, and configuration selection for test evaluation; they are therefore model-selection evidence. Test metrics are computed

Algorithm 1 EDGRec split-safe training and evaluation protocol. Each step states the mathematical operation and its methodological role, so the algorithm can be read as both an implementation order and an evidence-control checklist.

Require: Dataset configuration D ; model configuration ρ ; seed s ; split masks $M_{\text{train}}, M_{\text{val}}, M_{\text{test}}$; fan-outs F ; epoch budget E .

Ensure: Validation-selected state θ^* ; test metrics; resource and diagnostic fields.

```

1: procedure TRAINANDEVALUATEEDGREC( $D, \rho, s$ )
2:   Fix  $D, \rho, s$ , and random streams; load canonical records  $(u_n, i_n, z_n, t_n, \gamma_n)$ .
3:   Collapse repeated  $(u, i)$  records; assign one split mask and positive label  $z_n$  per pair.
4:    $\mathcal{G}_{\text{train}} \leftarrow \{(u_n, i_n) : M_{\text{train}}(n) \wedge z_n > 0\}$ ; use only positive training labels for message passing.
5:   Cache train-only  $p_i^{\text{train}}$ , recency, histories, safe features, candidate masks, and optional propensity targets.
6:   Initialize identifier embeddings, optional branch item tables,  $g_\phi$ , the score-fusion module, optimizer, and  $J^* \leftarrow -\infty$ ; start with no validation-selected state.
7:   for epochs  $e = 1, \dots, E$ , one complete pass over sampled positive-edge batches do
8:     for all mini-batches  $\mathcal{B}^+ \subset \mathcal{G}_{\text{train}}$  of positive training edges do
9:       Sample  $\mathcal{B} = \{(u, i, j) : (u, i) \in \mathcal{B}^+, j \notin \mathcal{I}_{u, \text{train}}^+\}$ ; each  $j$  is an unobserved training contrast.
10:       $q_{ij} \leftarrow \mathbb{I}(p_j^{\text{train}} - p_i^{\text{train}} > \delta_{\text{pop}})$ , or an aligned sampler mask; record whether the contrast item is sufficiently more train-popular than the positive item for branch routing.
11:       $\mathcal{S}_B \leftarrow \{u, i, j : (u, i, j) \in \mathcal{B}\}$ ; sample  $F$ -bounded train neighbours and induce  $(\mathcal{G}_B, \mathcal{V}_B)$  so each update propagates only through a controlled local training subgraph.
12:      for branches  $r \in \{I, C\}$ , propagating interest and conformity states separately do
13:        Set  $\mathbf{e}_v^{r, (0)}$  from identifier embeddings and enabled safe item-feature projections.
14:        for propagation depths  $\ell = 0, \dots, L_r - 1$  within branch  $r$  do
15:           $\mathbf{e}_v^{r, (\ell+1)} \leftarrow \mathbf{A}_B^r \mathbf{e}_v^{r, (\ell)}$ ; aggregate normalized branch- $r$  neighbour messages without adding a nonlinear graph-convolution transform.
16:        end for
17:         $\mathbf{z}_v^r \leftarrow \sum_{\ell=0}^{L_r} \alpha_\ell^r \mathbf{e}_v^{r, (\ell)}$ ; combine the initial embedding and propagated states into the final branch representation.
18:      end for
19:      Compute branch and context logits  $\ell^I, \ell^C, \ell^X$ ; score each positive and contrast item through the active channels before fusion places them on a shared ranking scale.
20:      Compute simplex weights  $\eta$ ; fuse logits into  $\hat{y}_{ui}, \hat{y}_{uj}$  and margin  $\Delta_{uij}$ , the positive-minus-contrast score used by BPR to reward higher positive scores.
21:      Compute  $\mathcal{L}_{\text{rec}}, \mathcal{L}_I, \mathcal{L}_C, \mathcal{L}_{\text{ind}}$ , and enabled  $\mathcal{L}_{\text{pop}}, \mathcal{L}_r$ ; keep disabled components out of the loss so inactive ablations do not affect gradients.
22:       $\mathcal{L} \leftarrow$  Equation (3.34); combine recommendation, branch, context, and enabled auxiliary terms into the declared objective, then update  $\theta$  with AdamW.
23:    end for
24:    if  $e$  is a validation epoch then
25:      Rank validation candidates after train-item masking; compute metrics and  $J_{\text{val}}(\theta)$  using validation data only for model selection.
26:      if  $J_{\text{val}}(\theta) > J^*$  then
27:         $\theta^* \leftarrow \theta$  and  $J^* \leftarrow J_{\text{val}}(\theta)$ ; retain the current validation-best state.
28:      end if
29:    end if
30:  end for
31:  Rank test candidates using  $\theta^*$  after masking train- and validation-observed items.
32:  Compute and log test ranking, list behaviour, runtime, VRAM, and branch/context diagnostics; use these fields for final reporting, not for model selection.
33: end procedure

```

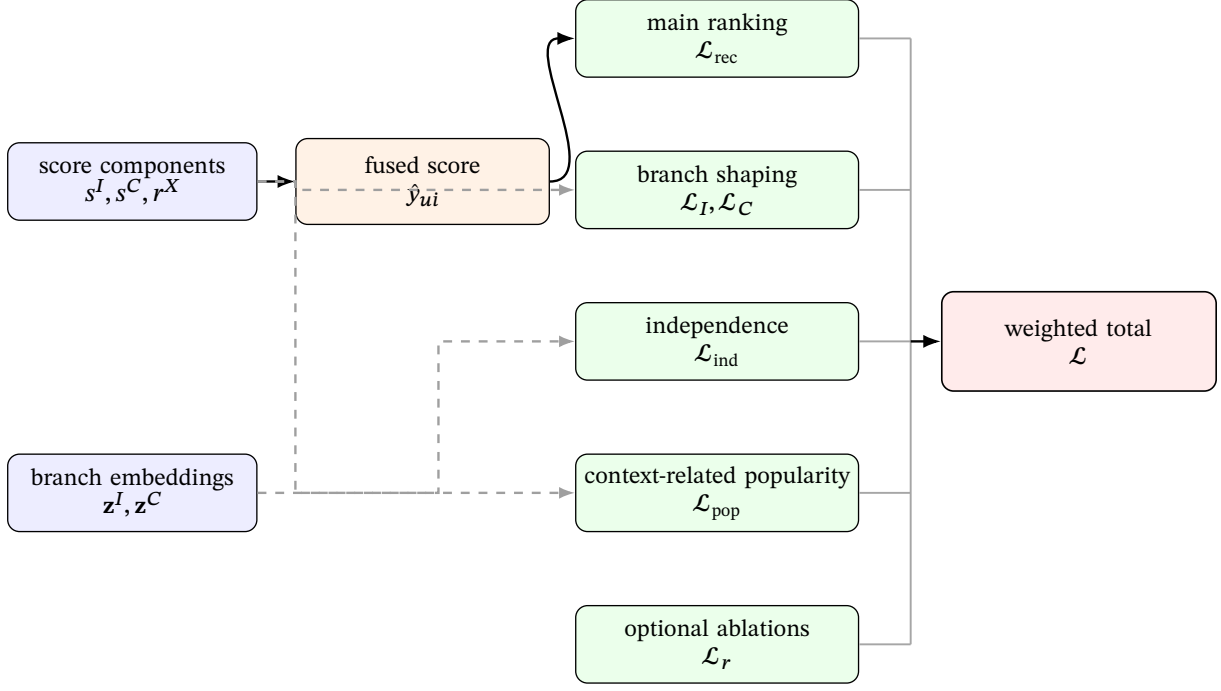


Figure 3.6: EDGRec score and loss composition. Branch and context scores feed the fused recommendation score for the main BPR loss, while branch-specific, independence, and context-related popularity losses enter the weighted total objective as representation-shaping terms rather than causal effect evidence.

only after the data protocol, feature policy, model configuration, and selection criterion are fixed; they are final reporting evidence. Runtime feasibility evaluations and diagnostic evaluations may explain feasibility or component behaviour, but they do not become final ranking evidence unless rerun as complete test evaluations under the declared protocol. This evidence firewall is the evaluation counterpart of the leakage policy in Section 3.2.6: validation and test labels are excluded from train-side quantities, and test outcomes are excluded from selection decisions.

The evaluator applies split-specific seen-item masking. Validation ranks candidate items after excluding the user’s training-observed items, whereas test excludes both training-observed and validation-observed items. Relevance is always the conjunction of the target split and a positive canonical label. Full-graph propagation is performed once per evaluation call and then reused for batched user scoring; diagnostic score-component exports are reserved for final post-training evaluation rather than for every validation pass. This keeps validation selection aligned with the primary metric bundle while avoiding diagnostic metrics becoming hidden selection criteria.

3.6.2 Ranking metrics

The primary ranking metrics are Normalized Discounted Cumulative Gain (NDCG), Recall, and HitRatio at cutoffs $K = 20$ and $K = 40$. These are computed on validation or test target

positives after the corresponding seen-item exclusion. Chapter 2 gives the per-user metric definitions; Chapter 3 fixes their use in the experimental protocol.

3.6.3 Popularity and personalization metrics

AveragePopularity@ K is computed from raw positive-train item counts assigned to the recommended items. It is a train-count concentration diagnostic, not an exposure-propensity estimate and not a normalized popularity statistic across different item universes. For run m , top- K recommendation list $\pi_u^K(m)$, and evaluation-user set $\mathcal{U}_{\text{eval}}(m)$,

$$\text{AveragePopularity}_K(m) = \frac{1}{|\mathcal{U}_{\text{eval}}(m)|K} \sum_{u \in \mathcal{U}_{\text{eval}}(m)} \sum_{i \in \pi_u^K(m)} p_i^{\text{train}}. \quad (3.35)$$

Personalization@ K is measured as one minus average inter-user list overlap:

$$\text{Personalization}_K(m) = 1 - \frac{2}{|\mathcal{U}_{\text{eval}}(m)|(|\mathcal{U}_{\text{eval}}(m)| - 1)} \sum_{\substack{\{u,v\} \subseteq \mathcal{U}_{\text{eval}}(m) \\ |\{u,v\}|=2}} \frac{|\pi_u^K(m) \cap \pi_v^K(m)|}{K}. \quad (3.36)$$

Lower AveragePopularity@ K indicates lower concentration around training-popular items under this metric and matched protocol. Fairness, exposure-bias, and causal-debiasing analyses are separate evaluation targets with their own data and metric requirements.

3.6.4 Runtime and VRAM measurement

Resource reporting uses average epoch duration and peak training VRAM. Let

$$M, T : \mathcal{M} \rightarrow \mathbb{R}_{\geq 0} \quad (3.37)$$

denote peak training VRAM in MB and average epoch duration in seconds for run set $\mathcal{M}$. Validation and test scoring can also be a bottleneck, so evaluation chunking and candidate policy are reported separately when relevant. The CRRU definition below uses M and T as training resource costs unless a run explicitly declares a different resource convention.

3.6.5 CRRU definition

CRRU, the Composite Resource-aware Recommendation Utility, is a configurable post hoc utility for completed evaluations. Fix a dataset D , split and candidate protocol s , hardware-reporting convention h , completed evaluation m , and weight vector θ ; CRRU is then an absolute per-evaluation value under those fixed conditions, not a score normalized over whichever comparisons appear in a report. Let $\mathcal{K}$ be the evaluated cutoff set, here $\{20, 40\}$. CRRU is used for validation selection and post hoc comparison only where explicitly declared.

It remains outside the neural training loss in Equation (3.34) and is interpreted after its ranking, list behaviour, runtime, and memory inputs are visible. The framework is reusable because another study can choose different indicators or weights; the weight vector below is the dissertation-specific expression of this experiment’s resource-aware preference.

For a fixed K , define the ranking-accuracy component

$$\begin{aligned} A_K(m; \mathbf{a}) &= \text{NDCG}_K(m)^{a_N} \text{Recall}_K(m)^{a_R} \text{HitRatio}_K(m)^{a_H}, \\ \mathbf{a} &= (a_N, a_R, a_H) \in \Delta^2. \end{aligned} \quad (3.38)$$

Here $\Delta^{d-1} = \{\mathbf{x} \in \mathbb{R}_{\geq 0}^d : \sum_{r=1}^d x_r = 1\}$. The exponents in $\mathbf{a}$ state the relative importance of rank-sensitive quality, coverage of held-out positives, and hit occurrence within the ranking component.

The average recommendation popularity normalized for CRRU is

$$\bar{P}_K(m) = \frac{\log(1 + \text{AveragePopularity}_K(m))}{\log(1 + p_{\max}^{\text{train}})}, \quad (3.39)$$

and the inverse recommendation-popularity score is

$$I_K(m) = 1 - \bar{P}_K(m). \quad (3.40)$$

Thus $\bar{P}_K$ converts raw train-count recommendation popularity to the unit interval used by CRRU, and I_K rewards lower concentration around train-popular items under the declared item universe.

The popularity-aware personalization component is

$$\begin{aligned} B_K(m; \mathbf{b}) &= \text{Personalization}_K(m)^{b_{\text{pers}}} I_K(m)^{b_{\text{pop}}}, \\ \mathbf{b} &= (b_{\text{pers}}, b_{\text{pop}}) \in \Delta^1. \end{aligned} \quad (3.41)$$

This component gives higher values to recommendation lists that are both more personalized across users and less concentrated on train-popular items.

The resource component converts lower-is-better costs into higher-is-better scores:

$$\begin{aligned} S_M(m) &= \frac{1}{1 + \log(1 + M(m)/M_0)}, \\ S_T(m) &= \frac{1}{1 + \log(1 + T(m)/T_0)}. \end{aligned} \quad (3.42)$$

with $M_0 = 1$ MB and $T_0 = 1$ second. Then

$$\begin{aligned} C_{\text{res}}(m; \mathbf{e}) &= S_M(m)^{e_M} S_T(m)^{e_T}, \\ \mathbf{e} &= (e_M, e_T) \in \Delta^1. \end{aligned} \quad (3.43)$$

The resource component is therefore higher for runs that use less peak training VRAM and less epoch time under the same hardware-reporting convention.

The final utility, reported later as CRRU@ K , is

$$\text{CRRU}_K(m; \theta) = A_K(m; \mathbf{a})^{w_A} B_K(m; \mathbf{b})^{w_B} C_{\text{res}}(m; \mathbf{e})^{w_C}, \quad (3.44)$$

where

$$\begin{aligned} \theta &= (\mathbf{a}, \mathbf{b}, \mathbf{e}, \mathbf{w}), \\ \mathbf{w} &= (w_A, w_B, w_C) \in \Delta^2. \end{aligned} \quad (3.45)$$

Zero-weighted factors are omitted from the corresponding product; if numerical evaluation uses an epsilon floor to avoid exact-zero collapse under fractional powers, the floor must be reported.

The weight vector used in the experiments is

$$\begin{aligned} \mathbf{a} &= (0.50, 0.35, 0.15), \\ \mathbf{b} &= (0.40, 0.60), \\ \mathbf{e} &= (0.50, 0.50), \\ \mathbf{w} &= (0.55, 0.30, 0.15). \end{aligned} \quad (3.46)$$

These values are the experimental preference vector used in this thesis; they are not part of the CRRU framework definition. Another recommendation setting may choose a different θ if the indicators, transformations, weights, and interpretation context are declared before comparison. This weighting is accuracy-dominant but not accuracy-only: ranking metrics carry the largest outer weight, while popularity-aware list behaviour and resource cost are still allowed to change the ordering when ranking evidence is close or when computational differences are large.

When the same formula is computed on validation-split metrics, the result is denoted ValidationCRRU. The ValidationCRRU@20–@40 mean is the arithmetic mean of ValidationCRRU@20 and ValidationCRRU@40, and it may be used as a single model-selection objective. This quantity is not a separate metric family; it is the validation-split version of CRRU used before final test evaluation.

3.6.6 CRRU limitations and sensitivity checks

CRRU is a parameterized comparison utility for a fixed dataset, split, candidate protocol, hardware-reporting convention, and weight vector. It summarizes ranking accuracy, popularity-aware personalization, and training resource cost under declared weights, without min–max normalization over the comparisons currently displayed in a report. Its interpretation is therefore component-first: the scalar expresses one declared resource-aware preference over visible measurements, while counterfactual outcomes, treatment effects, propensities, policy value, protected-group parity, and distributional harms remain separate

evaluation targets with their own data requirements.

Every CRRU table therefore reports the component metrics and the chosen weight vector. Sensitivity checks should vary the weights or compare component-level rankings when a conclusion depends on the scalar utility. The thesis interpretation gives priority to the visible components: ranking quality, AveragePopularity@K, Personalization@K, epoch duration, and peak VRAM. ValidationCRRU is selection evidence, whereas final-test CRRU is evaluation evidence computed after the selected configuration has been rerun under the declared protocol.

3.7 Reproducibility

This section records method-level conditions needed to interpret and rerun experiments. The hardware and software environment is treated as an experimental condition in Chapter 4, where runtime, peak VRAM, feasible batch size, sampled fan-out, and full-catalog evaluation chunks are defined and reported with Table 4.5. The present section therefore focuses on random seeds and model-state recording conventions.

3.7.1 Random seeds

Random seeds control data sampling, initialization, negative sampling, and hyperparameter search where applicable. The thesis reports seed usage for formal comparisons and distinguishes repeated complete evaluations from validation trials, runtime feasibility evaluations, and diagnostic experiments.

3.7.2 Experiment records and model states

Experiment recording is structured around a persistent scientific record. A structured result store is treated as the authoritative record for completed evaluations, while any auxiliary experiment-tracking interface is used only as an audit mirror when enabled [30]. The accompanying implementation and experiment-generation materials are available online¹, providing implementation provenance for the methodological contracts described in this chapter. Stored model states are keyed by the scientific training identity: dataset, preprocessing protocol, split, item-universe policy, model configuration, and training-defining hyperparameters. Evaluation identities record comparability conditions such as candidate policy and metric cutoffs, while checkpoint filenames are derived from the training identity so that recovery does not silently mix incompatible configurations.

Completed evaluations use the best validation-selected state for test evaluation when available. When automatic batch-size resolution is active, the resolved batch-size context is

¹github.com/LazarosPan/Efficient-Disentangled-Graph-Recommender.

preserved with compatible model-state records so that recovery does not silently change the training identity. Runtime feasibility evaluations are reported as feasibility or approximation evidence and are excluded from ranked formal result tables unless explicitly rerun as complete test evaluations.

Chapter 4

Experimental Setup

The experimental contract specifies how the Efficient Disentangled Graph Recommender (EDGRec) is evaluated. Chapter 3 defined the observed top- K task, train-only graph construction, model family, losses, ranking and popularity diagnostics, and Composite Resource-aware Recommendation Utility (CRRU). The present chapter fixes how those definitions are instantiated in experiments: which questions are asked, which dataset views are used, which baselines and EDGRec configurations are eligible for formal comparison, how validation evidence is separated from final test evidence, and how runtime and memory are measured. Final outcomes are reserved for Chapter 5, where component metrics are reported before CRRU is interpreted.

The setup is motivated by three constraints. First, implicit-feedback recommendation uses observed positives and unobserved pairs rather than fully labelled preferences, so missing entries and sampled negatives are treated as unobserved contrasts rather than verified dislikes [3]. Second, the graph collaborative-filtering backbone must remain comparable with the Light Graph Convolutional Network (LightGCN) simplification, in which normalized neighbourhood aggregation is retained while feature transformations and nonlinear activations are removed [6]. Third, the interest and conformity split follows Disentangling User Interest and Conformity for Recommendation with Causal Embedding (DICE) as a modelling idea inspired by causal recommendation, while EDGRec changes the training and scoring contract through sampled graph propagation, bounded auxiliary objectives, item-context scoring, and resource-aware reporting.

The experiment therefore reports the observable quantities available under the logged protocol: ranking, list behaviour, branch diagnostics, runtime, and peak training VRAM [4, 5]. This design makes efficiency a formal comparison dimension rather than an implementation afterthought: an EDGRec configuration can support the thesis when it improves the feasible ranking–resource frontier under a declared protocol, even if it is not uniformly superior on every accuracy, popularity, personalization, or memory component.

4.1 Experimental Questions

The empirical study is organized around six research questions, abbreviated as RQ1–RQ6. The numbering is reused when summarizing findings in Chapter 5, where each answer is tied to the evidence category and dataset protocol that produced it. The questions are phrased as comparative trade-off questions because the thesis evaluates multiple datasets, item-universe policies, and training regimes whose evidence must be interpreted within their declared protocols. The phrasing is intentional: an efficiency-oriented result can answer the thesis only when the corresponding relevance, list behaviour, memory, and evidence-role qualifications remain visible.

RQ1: Does EDGRec improve or preserve top- K ranking quality relative to reference LightGCN and available sampled diagnostic comparators under the same preprocessing, split, and candidate protocol?

RQ2: What popularity-concentration and personalization behaviour is observed for EDGRec’s interest and conformity design relative to the displayed comparators?

RQ3: What epoch-time and peak VRAM trade-offs does EDGRec show relative to completed LightGCN baselines, and what resource-feasibility evidence do single-epoch reference DICE evaluations provide when complete reference DICE baselines are unavailable?

RQ4: Which EDGRec components are supported by matched ablation evidence?

RQ5: How dataset-dependent are the results across AmazonBook, MovieLens-1M, KuaiRec v2, and KuaiRand-1K?

RQ6: Does CRRU select different configurations from raw NDCG or Recall, and which visible component metrics explain that difference?

Table 4.1 fixes the role of each metric family. Ranking metrics are the primary evidence for relevance. Popularity, personalization, runtime, and memory are component metrics because they explain trade-offs that raw ranking quality can hide. CRRU is comparative: it summarizes a declared resource-aware preference vector after the components are visible. This ordering reflects the multi-objective nature of recommender evaluation and the methodological cautions associated with weighted composite indices [2, 7]. It also anticipates the main empirical pattern: an efficient graph recommender may be scientifically useful because it reaches a favourable Pareto region of ranking quality and resource use, even when it is not uniformly best on every accuracy or list behaviour metric. Efficiency is therefore evaluated as part of the scientific objective rather than as an informal implementation benefit.

Table 4.1: Metric roles used throughout the experimental setup.

Role	Metrics	Used to answer	Interpretation rule
Primary ranking evidence	NDCG@ K , Recall@ K , HitRatio@ K at the declared cutoffs	RQ1 and the ranking component of RQ5	A ranking-quality claim must be supported by these metrics, not by CRRU alone.
Popularity and personalization diagnostics	AveragePopularity@ K , Personalization@ K	RQ2, RQ5, and component interpretation for RQ6	Lower AveragePopularity means lower concentration around training-popular items under the logged train-count metric and matched item universe; exposure propensity, fairness, and causal-debiasing questions require separate metrics.
Resource diagnostics	Average epoch duration, peak training VRAM, evaluation chunking status	RQ3 and resource parts of RQ6	Runtime and VRAM are interpreted with the training regime, batch-size policy, fan-out, and scoring convention.
Ablation diagnostics	Component metrics above, plus branch and score fusion diagnostics when available	RQ4	Ablations are interpreted component-by-component before any scalar utility is used.
Composite utility	CRRU@20, CRRU@40, and ValidationCRRU@20–@40 mean when used for validation selection	RQ6	ValidationCRRU is selection evidence; final-test CRRU is evaluation evidence. CRRU tables must show the weight vector and component metrics, and sensitivity is required when a conclusion depends on the scalar utility.

4.2 Dataset-Specific Configurations

The thesis uses four dataset families because they represent different observation regimes. MovieLens-1M is an explicit rating dataset that is converted into an implicit top- K task [12]. KuaiRec provides both a larger sparse view and a small-matrix dense-observation view, but the default thesis setting is the sparse watch-ratio protocol [14]. KuaiRand-1K provides standard and randomized video-exposure logs, so its ranking evaluations follow the same metric contract but require conservative exposure-aware interpretation when item-universe policies differ [15]. AmazonBook is used as a sparse interaction-only collaborative-filtering benchmark in the LightGCN benchmark tradition [6].

MovieLens-20M and Taobao were screened and schema-checked as large-catalog candidates, but they are excluded from the completed evaluation set because the available thesis hardware budget did not support full evaluation under the same reporting contract. Figure 3.3 in Chapter 3 gives the compact visual map of the evaluated dataset regimes, so the present chapter states the operational contract without duplicating the setup figure.

The dataset-specific contract is stated below as protocol text, while Chapter 3 and Appendix A.1 record the source-column lineage. For every final result entry, the result table must still report or make reconstructible the number of users, items, positive training edges, validation positives, test positives, item-universe policy, positive-label rule, randomized-exposure status when applicable, and feature policy. These quantities determine graph size, embedding-table size, candidate denominators, sampling difficulty, evaluation cost, resource interpretation, and the admissibility of any side-feature claim.

- *AmazonBook*: The interaction-only protocol treats provided graph interactions as positive events and uses positive training edges only. It supports ranking, speed, and raw train-popularity concentration trade-off claims for sparse collaborative filtering.
- *MovieLens-1M*: The rating-to-implicit protocol converts explicit ratings into positives under the configured threshold and split policy. Genre features are opt-in and labelled when used. Conclusions apply to converted top- K ranking rather than rating prediction.
- *KuaiRec v2*: The default sparse protocol uses watch-ratio feedback, with positives defined by $\text{watch_ratio} \geq 0.5$. Safe structured item descriptors may be enabled only in explicit feature configurations. Small-matrix and stricter-threshold evaluations are sensitivity views unless reported separately.
- *KuaiRec v2 small matrix*: The dense-view protocol follows the small-matrix labels while still constructing a train-only graph. It supports sensitivity analysis, and it remains separate from the sparse watch-ratio interpretation.

- *KuaiRand-1K*: The exposure-aware protocol uses configured video-feedback labels over standard and randomized exposure records. Statistic and engagement aggregates are excluded from standard model features; compact evaluations may restrict the item universe to randomized-exposure items. Ranking metrics, list behaviour metrics, runtime, and VRAM are still computed under the same reporting contract; conclusions are limited when compact and non-compact protocols are compared.
- *KuaiRand-1K random-only*: The random-only protocol uses randomized-exposure records and the randomized item universe. It supports exposure-sensitivity interpretation rather than broad platform-level deployment claims.

All dataset views use split-aware repeat handling: one raw user-item pair is kept within a single split after repeat collapse. Popularity, recency, recent-history buffers, masks motivated by DICE, AveragePopularity@ K denominators, and CRRU popularity denominators are derived from the final training split only. Feature construction follows the leakage policy from Chapter 3: post-treatment engagement aggregates are excluded from primary scoring features, and any configuration with side features is labelled separately.

Figures 4.1, 4.2, 4.3, and 4.4 are per-dataset setup diagnostics rather than model-performance evidence. The profiles follow the same reading order: scale and density, user and item long tails, binary label versus graded sign, split or temporal structure, and dataset-specific context such as feature availability or exposure policy. They therefore explain why the same EDGRec method has different interpretation scope on each dataset.

KuaiRand-1K is described as exposure-aware because its randomized-exposure records and compact item-universe variants make exposure sensitivity visible; the same ranking and resource metrics are reported, but cross-protocol conclusions are restricted when the item universe changes. The graph-scale setup check in Figure 4.5 is retained because it connects the dataset contract to the positive training graph used by propagation.

Unlike the sampled topology visualization in Figure 3.4, these histograms use every positive training edge and deliberately avoid drawing non-positive feedback records as if they were graph edges. The aligned label and sign counts are reported in the supplementary dataset summaries. The figure is therefore setup evidence for graph sparsity, item-popularity skew, train-only popularity controls, and popularity-conditioned negative sampling rather than ranking-performance evidence.

4.3 Hyperparameter Search and Candidate Selection

Model selection is validation-only. Test metrics are not used to choose hyperparameters, select configurations from validation search for final test evaluation, tune CRRU weights, choose feature groups, or decide whether an evaluation is eligible for formal comparison. This rule is necessary because the same evidence archive contains validation searches,

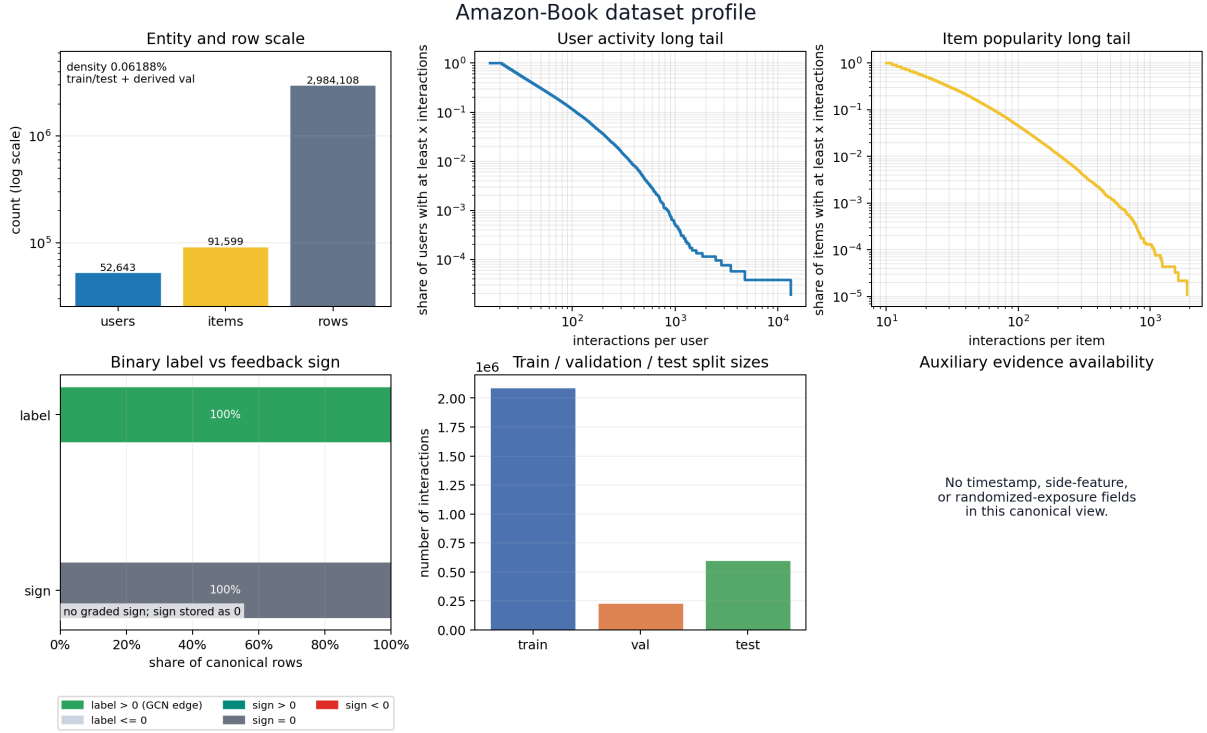


Figure 4.1: AmazonBook dataset profile for the sparse interaction-only protocol. The figure summarizes entity and record scale, user and item long tails, the constant positive-label graph signal, split structure, and the absence of timestamp, feature, and randomized-exposure auxiliary fields. It supports the protocol definition for AmazonBook but does not add a model-performance claim.

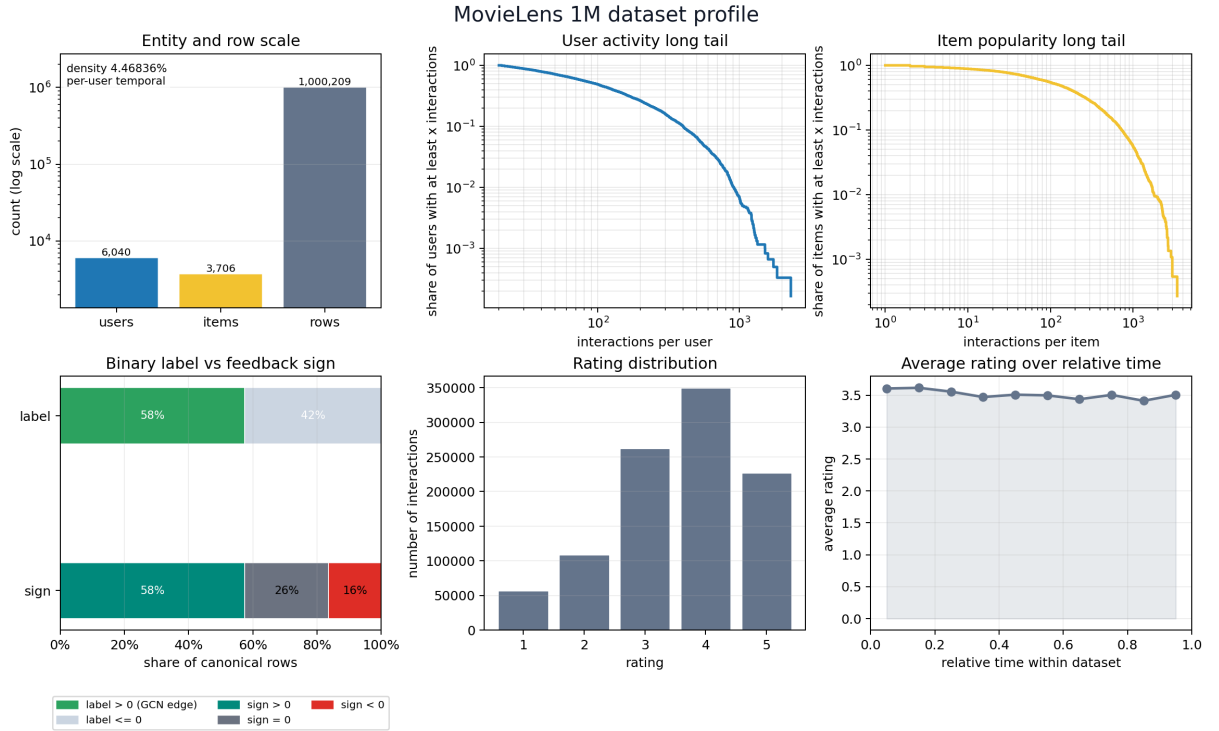


Figure 4.2: MovieLens-1M dataset profile for the rating-to-implicit protocol. The figure records scale, long-tail structure, the separate binary label and graded sign distributions, rating distribution, and rating movement over relative time. It frames the experiment as an implicit top- K task rather than rating prediction or causal exposure correction.

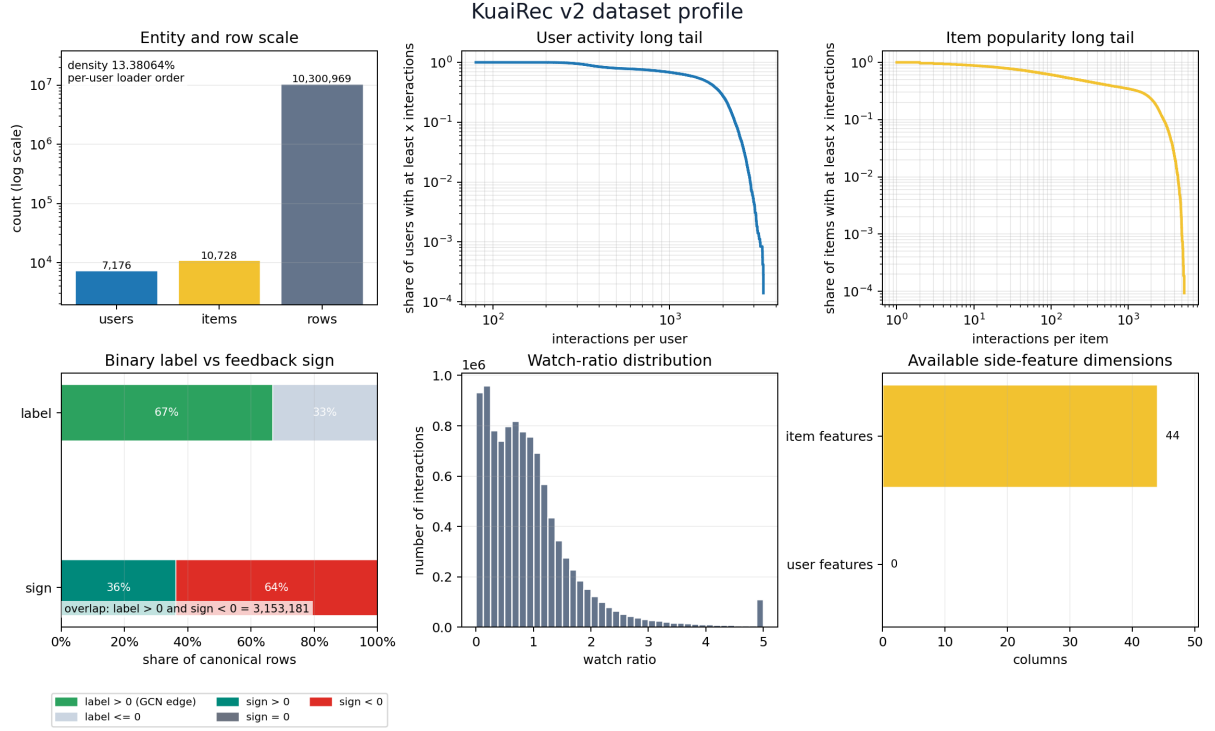


Figure 4.3: KuaiRec v2 dataset profile for the sparse watch-ratio protocol. The figure summarizes scale, long-tail structure, binary label versus graded sign semantics, watch-ratio distribution, and available item-side metadata so that later KuaiRec results are read under the intended sparse setting.

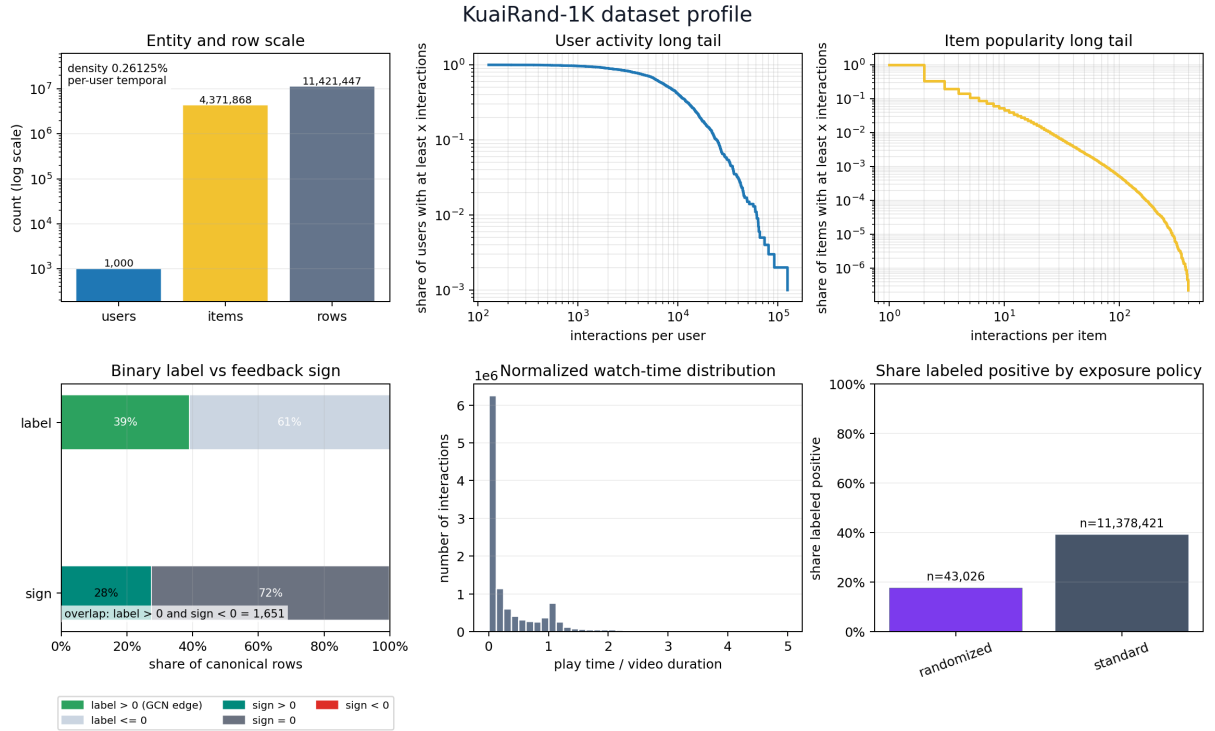


Figure 4.4: KuaiRand-1K dataset profile for the exposure-aware protocol. The figure summarizes scale, long-tail structure, binary label versus graded sign semantics, normalized watch-time distribution, and randomized-versus-standard exposure context. It is included to make compact randomized-exposure item-universe choices visible when interpreting ranking and resource results.

Full Positive-Train Degree Distributions

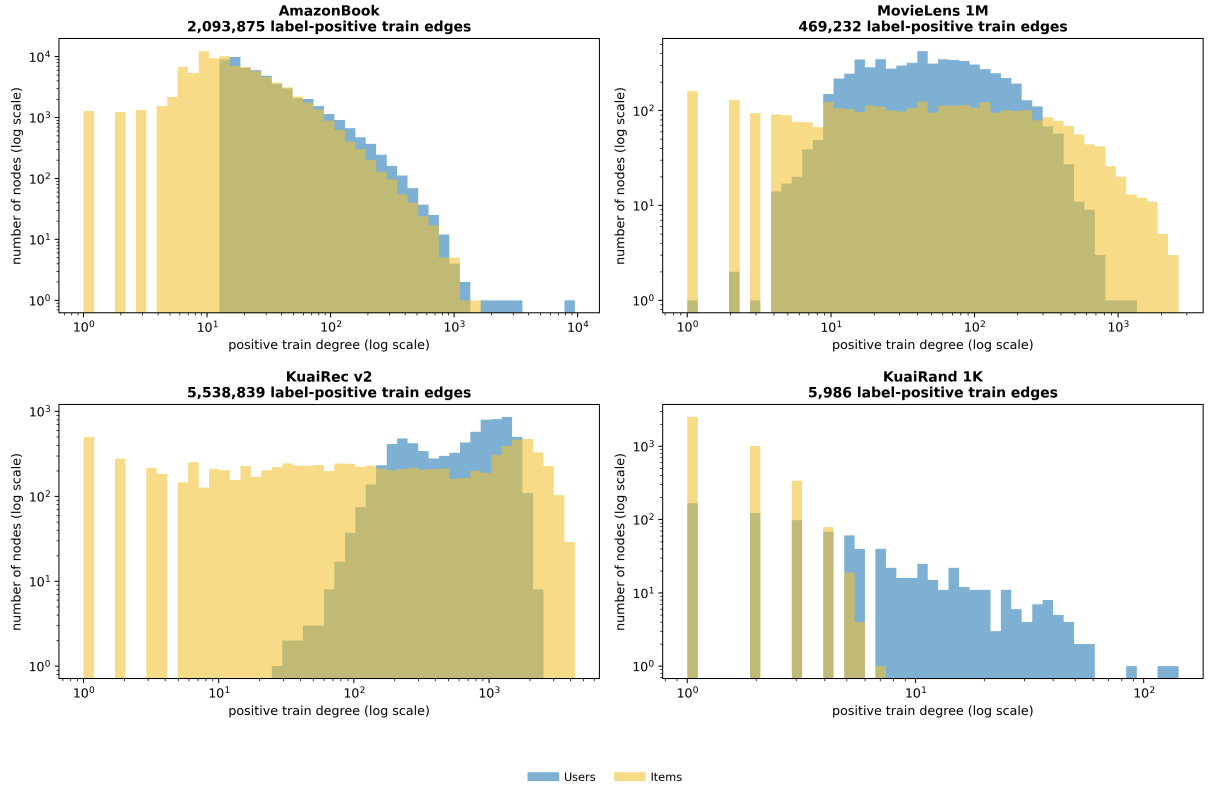


Figure 4.5: Positive-training-degree distributions for the four thesis datasets. The histograms use the complete label-positive training graph: AmazonBook has 2,093,875 positive train edges, MovieLens-1M has 469,232, KuaiRec v2 has 5,538,839, and KuaiRand-1K has 5,986 under the compact randomized-exposure item universe used for the displayed reference evaluation. Non-positive-label and non-positive-sign records are not added as extra propagation edges unless they are also label-positive. The aligned label/sign summaries are reported separately in the supplementary dataset summaries. The distributions motivate train-only popularity reporting and popularity-conditioned sampling; they are setup evidence rather than ranking-performance evidence.

runtime feasibility evaluations, diagnostics, and completed test evaluations; without an explicit selection boundary, post hoc comparison could silently become test-set tuning.

Optuna, the hyperparameter-optimization framework used for validation searches, may be used with a fixed search space, a declared objective, a sampler, a pruner, an epoch cap, and a search space revision, meaning the versioned combination of search space rules and configuration defaults [31]. If a search space changes, trials from the earlier revision do not count toward the fresh-trial budget of the revised experiment.

Figure 4.6 shows the original Optuna system-design diagram at the point where the thesis introduces validation search. In the Optuna paper, each worker executes an objective-function instance for a study, the objective uses Optuna APIs to create or update trial state, and shared storage records the study history so independent workers can coordinate through the storage backend [31]. The thesis uses this figure to clarify the methodological boundary of Optuna search: the unit being optimized is an objective-evaluating trial, and the stored trial history supports candidate selection, but final test evidence still requires the validation-to-test gate defined below.

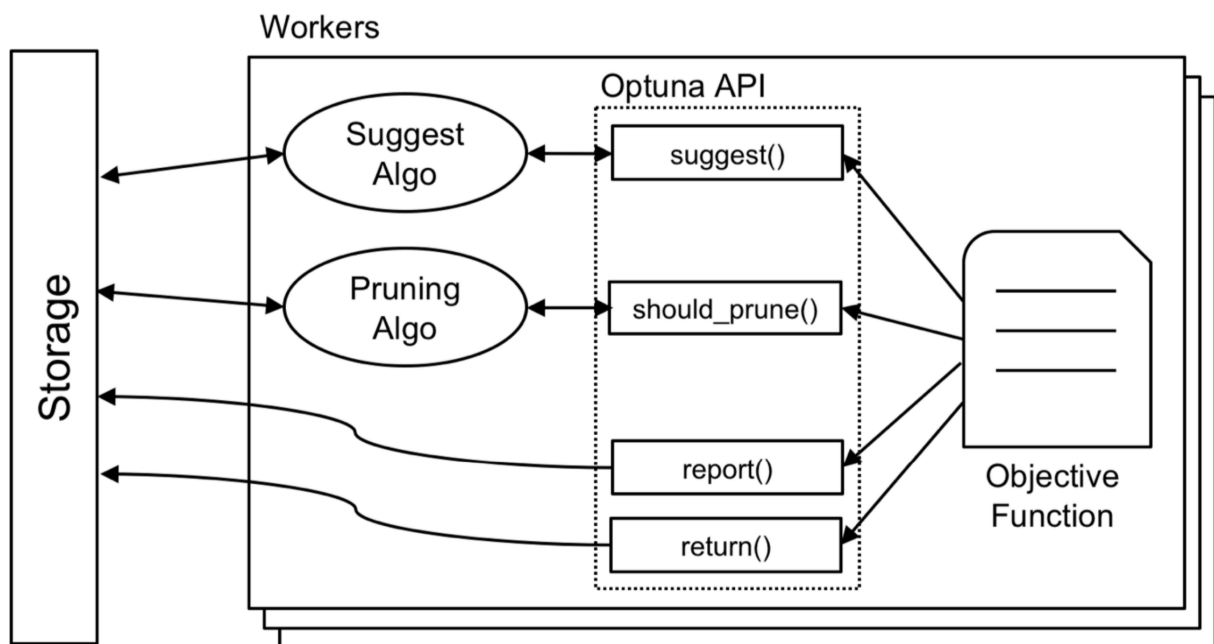


Figure 4.6: Optuna validation search system design. The source figure depicts workers running objective functions inside a study and sharing progress through storage; in this thesis, the diagram motivates the validation search trial log and candidate-selection workflow, not a claim that validation trials are final test evidence. Source: Akiba et al. [31].

Table 4.2 defines the evidence categories. This table is strict because the evidence corpus contains formal evaluations, validation searches, runtime feasibility evaluations, archival records, and diagnostics. Mixing these categories would confound selection evidence, evaluation evidence, feasibility evidence, and component evidence. The baseline evidence-label contract in Section 4.4 complements the table by giving the corresponding model family crosswalk. The content is kept as text rather than as a text-heavy raster figure, because the

distinction is part of the formal evidence contract.

Table 4.2: Evidence categories for validation search and final reporting.

Category	Definition	Permitted use	Exclusion rule
Optuna validation search candidate	A completed validation trial under a named search space, search space revision, objective, sampler, pruner, and epoch cap.	Select candidate hyperparameters; inspect validation ranking, popularity, resource, and CRRU components.	Not a test result and not a final ranking claim.
Final test configuration	A named configuration selected from validation evidence and rerun under the declared complete test protocol.	Eligible for final ranking, popularity, runtime, VRAM, and CRRU comparison.	Must retain its validation provenance and configuration description.
Runtime feasibility evaluation	A single-epoch, reduced, automatic batch-size feasibility, CPU fallback, or profiling evaluation.	Feasibility and systems-cost evidence, especially for reference DICE.	Not final accuracy evidence unless rerun as a final test configuration.
Archival record	A record produced under an earlier experimental pipeline, search space revision, metric convention, or reporting convention.	Background context only when explicitly labelled.	Does not satisfy the declared protocol unless rerun or reconciled.
Diagnostic-only or protocol-qualified evidence	A diagnostic check, sensitivity view, feature subset validation result, branch diagnostic, or compact exposure-aware comparison that changes the item-universe or training regime contract.	Component interpretation, future selection, or explicitly labelled supporting evidence when a complete test evaluation exists.	Not a primary result without selection for final testing, complete evaluation, and a comparable item-universe policy.

The current evidence ledger is also audited before it is used for Chapter 5. In the generated result report, CRRU is computed for 288 rows, no rows are skipped, and the audit checks that AveragePopularity@20 and AveragePopularity@40 do not exceed the corresponding largest train item count and that CRRU@20 and CRRU@40 remain in $[0, 1]$. The Optuna validation archive contains 19 loaded studies and 801 dataset-scoped completed candidates. These counts are not empirical conclusions; they document the scope of the fixed evidence snapshot from which validation figures, feature-subset summaries, and final test comparisons are drawn. If a later rerun changes the ledger, the candidate-selection tables and Chapter 5 result tables must be regenerated together.

Figure 4.7 and Algorithm 2 state the same evidence firewall procedurally. They are included because the thesis contains validation searches, runtime feasibility evaluations, feature subset diagnostics, and final test evaluations in one evidence corpus; the reader needs to see where an evaluation becomes eligible for a Chapter 5 claim and where it remains diagnostic. A validation candidate becomes final evidence only after selection as an exact

configuration and completion of the test evaluation. Evaluations that fail this gate can still explain feasibility, search behaviour, or future-work choices, but they cannot be reinterpreted later as final test evidence.

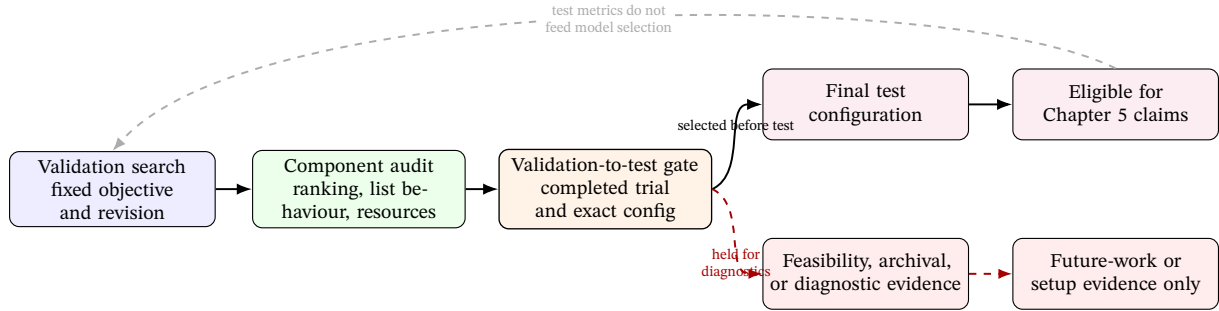


Figure 4.7: Validation-to-test evidence gate. Validation trials may select or motivate a configuration, but they become final test evidence only after a named configuration is rerun under the complete test protocol. Runtime feasibility evaluations, archival records, feature subset diagnostics, and other non-final evidence remain diagnostic unless explicitly rerun.

Table 4.3 gives the compact search space contract used for model selection. The exact numeric ranges are versioned by the declared search space revision and stored with each completed trial; empirical result tables report the effective configuration for final test evaluations rather than treating a model family label as sufficient evidence. In the table, the GCN instantiation of DICE denotes the Graph Convolutional Network variant of the DICE framework.

Validation objectives may include ranking-oriented objectives. They may also use the ValidationCRRU@20–@40 mean, defined as the arithmetic mean of CRRU@20 and CRRU@40 computed on the validation split only. This quantity differs from final-test CRRU in evidence role: ValidationCRRU may select configurations, whereas test CRRU is reported only after complete test evaluation and does not influence model selection. A configuration selected by ValidationCRRU is labelled as such because a scalar utility can prefer a model for resource or popularity reasons even when raw NDCG or Recall is not maximal. The final interpretation therefore checks whether that validation-selected preference survives as visible component evidence on the test split.

Figures 4.8, 4.9, and 4.10 are therefore supplementary validation search diagnostics: they audit candidate selection. The frontier plot shows the explored validation trade-off surface; the component plot shows which normalized terms drive the validation utility; and the importance plot is an association summary from the search process rather than evidence that a hyperparameter causally controls the objective.

The active search protocol is coarse-to-focused. Compact EDGRec searches and dataset-local candidate searches use fixed search space revisions, Tree-structured Parzen Estimator sampling, Hyperband pruning, validation-only objectives, and epoch caps rather than exhaustive grids. Some spaces optimize ValidationCRRU@20–@40 directly, while coarse

Algorithm 2 Validation search to final-test protocol. The procedure separates validation selection, provenance audit, and complete test evaluation before a configuration becomes final evidence.

Require: Dataset protocol D ; revision r ; validation objective J_{val} ; trial logs $\mathcal{T}$.

Ensure: Final test configuration $\pi^\star$ or a recorded diagnostic status.

```

1: procedure SELECTVALIDATIONCANDIDATEFORTEST( $D, r, J_{\text{val}}, \mathcal{T}$ )
2:   Freeze  $D$ , search space  $r$ , sampler, pruner, feature policy, candidate masks, epoch budget, and
   objective formula so selection cannot change after validation evidence is inspected.
3:    $\mathcal{E} \leftarrow \emptyset$ ; start with no eligible validation candidates.
4:   for all trials  $\tau \in \mathcal{T}$  recorded in the validation search log do
5:     if  $\tau$  is completed, current-revision, same-protocol, non-duplicate, not a feasibility evalua-
     tion, and has required validation metrics then
6:       Recompute  $J_{\text{val}}(\tau)$  from validation metrics only; add  $(\tau, J_{\text{val}}(\tau))$  to  $\mathcal{E}$  as selectable evi-
       dence with no test-set information.
7:     end if
8:   end for
9:   if  $\mathcal{E} = \emptyset$ , meaning no trial satisfies the selection contract then
10:    return a recorded diagnostic status because no eligible validation candidate exists.
11:  end if
12:   $\tau^\star \leftarrow \arg \max_{(\tau, J) \in \mathcal{E}} J$ ; select the candidate with the largest validation objective and break
   exact ties by predeclared deterministic ordering.
13:  Audit  $\tau^\star$ : NDCG, Recall, HitRatio, AveragePopularity, Personalization, epoch time, VRAM,
   and recorded configuration identity.
14:  if audit fails, provenance is missing, or the selected candidate uses test metrics then
15:    return a recorded diagnostic status because selection evidence is incomplete or contami-
    nated by test information.
16:  end if
17:  Materialize  $\pi^\star$  by copying  $\tau^\star$  hyperparameters, preprocessing protocol, feature gates, seed
   policy, and masks; this fixes the configuration that may be tested.
18:  Rerun  $\pi^\star$  from training data under the complete protocol  $D$ ; save selected model state, metrics,
   resources, and protocol record for final evidence.
19:  if the final test evaluation is complete and the recorded protocol identity matches  $D$  then
20:    Attach validation provenance to  $\pi^\star$  and return final test evidence.
21:  else
22:    return a recorded diagnostic status because the final-test rerun did not complete under
    the declared protocol.
23:  end if
24: end procedure

```

Table 4.3: Compact hyperparameter and search space contract.

Family	Key hyperparameters exposed	Fixed or default controls	Selection criterion
BPR-MF reference	Embedding dimension, learning rate, L2 regularization or weight decay, batch size, negative-sampling count and policy, epoch cap, and early-stopping patience.	No graph propagation, no branch structure, no context head, same training, validation, and test masks and candidate filtering as graph models.	Excluded from the current reported result set; supports non-graph comparisons only after a complete test evaluation under the declared protocol.
LightGCN	Embedding dimension, propagation depth, learning rate, regularization, batch size, optimizer convention, and full-graph versus sampled-training label.	Reference LightGCN keeps the full-graph convention; sampled LightGCN is a separate ablation under EDGRec’s sampled training regime.	Reference complete evaluation when feasible; otherwise evidence type states validation, feasibility, or diagnostic status.
DICE and its GCN instantiation	Branch dimension, learning rate, batch size, number of negatives, popularity-conditioned sampler settings, branch-loss weights, discrepancy weight, optimizer convention, and epoch cap.	Reference DICE keeps the source branch and sampler convention where hardware permits; feasibility changes are reported as resource-adjusted variants or feasibility evaluations.	Final comparison only when rerun under the formal protocol; runtime feasibility evaluations support feasibility claims only.
EDGRec	Branch dimension, interest and conformity depths, fan-out list, batch size or automatic batch-size policy, learning rate, weight decay, score fusion floor, context-head status, auxiliary-loss weights and schedules, negative-sampling strategy, feature gates, and temporal-history switches.	Train-only graph, seen-item filtering, popularity-conditioned negatives by default, bounded sampled subgraphs, declared configuration family, and logged effective configuration.	Dataset-local validation objective over accuracy or CRRU at $K \in \{20, 40\}$, followed by final test evaluation.

mechanism searches may optimize validation accuracy and then reconstruct Validation-CRRU for cross-study audit.

Automatic batch-size selection is treated as runtime metadata when enabled: a larger feasible batch is not interpreted as a causal model component, and final result tables must report the resolved effective batch size. Archival graph-augmentation or approximate-nearest-neighbour rows are provenance only. CUDA-accelerated graph construction and approximate nearest-neighbour search (CAGRA) is relevant to GPU approximate-nearest-neighbour graph construction [32], but the active EDGRec training protocol does not use CAGRA graph augmentation as a speed mechanism; the graph policy remains the observed positive training graph.

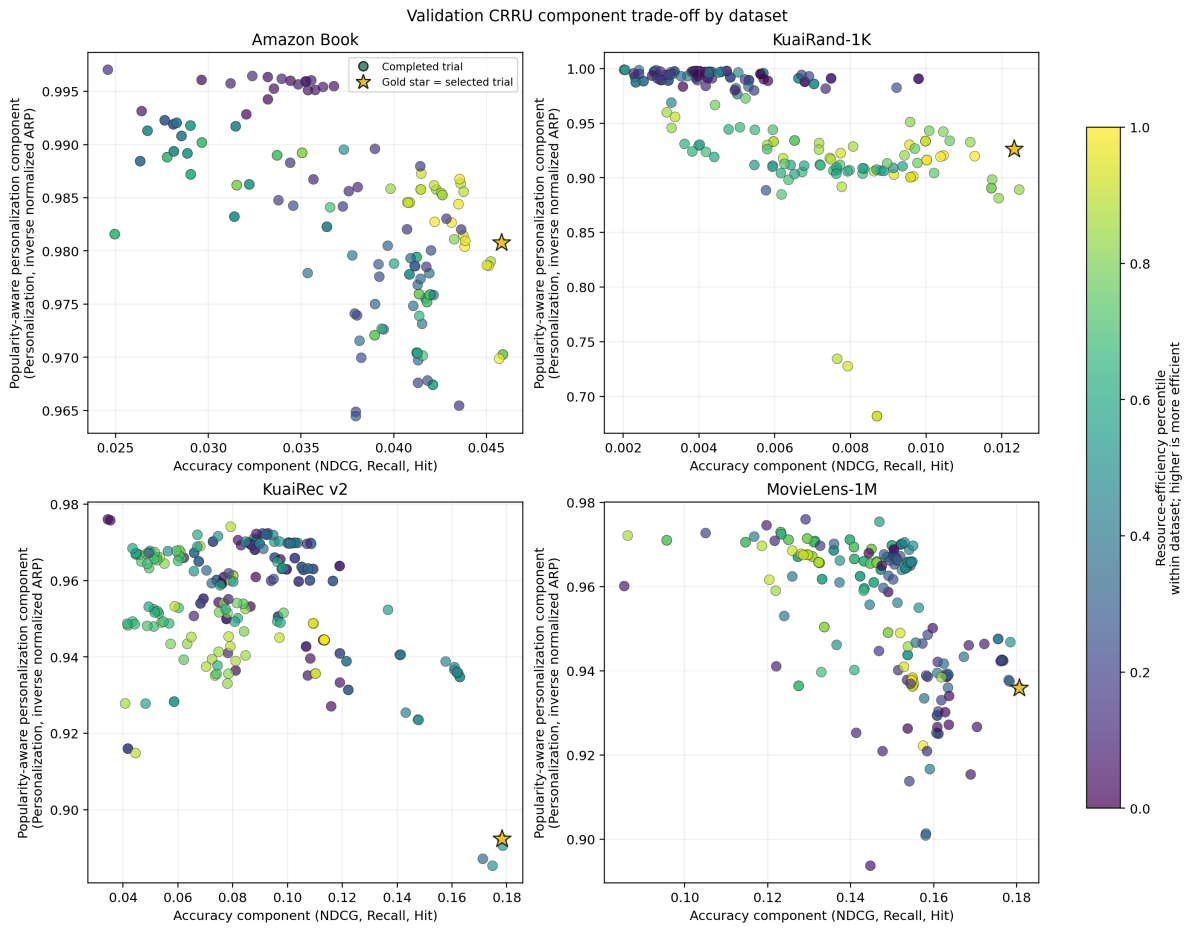


Figure 4.8: Validation search selection frontier by dataset. The figure plots ranking accuracy against popularity-aware personalization, with resource-efficiency percentile shown within each dataset. Colors are therefore dataset-local diagnostics rather than absolute efficiency values comparable across datasets. Its role is methodological: it documents the selection procedure before test evaluation; final test evidence is reported only after complete test evaluation.

Feature-subset search is a candidate-selection procedure rather than final testing. Its purpose is to identify eligible, leakage-controlled feature groups that deserve complete reruns before any final side-feature claim is made. Table 4.4 records the search design, and Figure 4.11 visualizes the validation-only movements. The dot plot compares ValidationCRRU

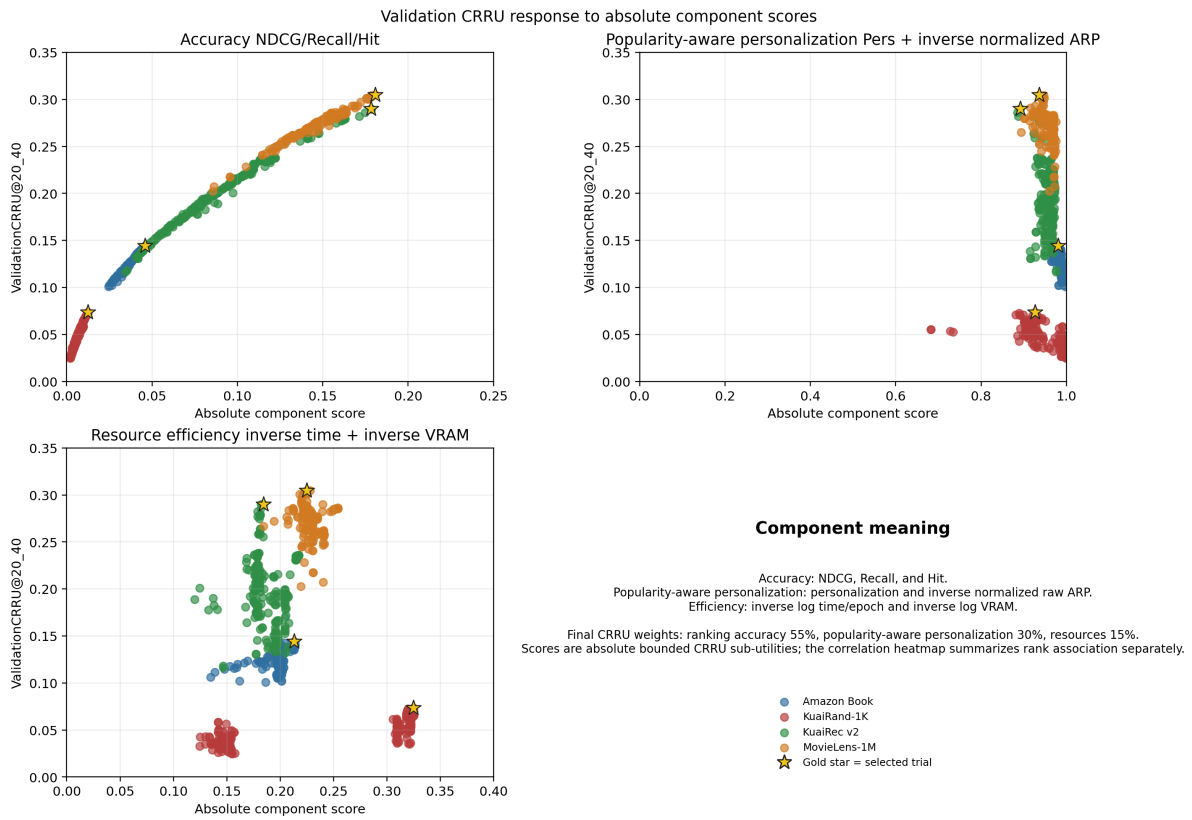


Figure 4.9: ValidationCRRU components by dataset. The figure separates the ranking, list behaviour, and resource terms that enter the validation utility under the declared weighting scheme. It is included to prevent the scalar CRRU value from obscuring which experimental component drives candidate selection.

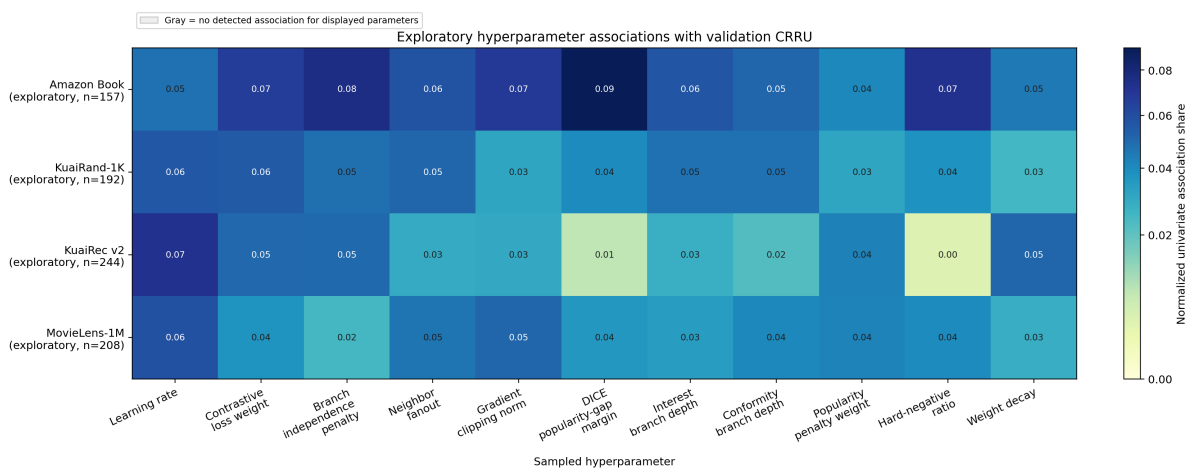


Figure 4.10: Validation search hyperparameter importance by dataset. The figure summarizes which sampled configuration dimensions were most associated with the validation objective within each displayed completed-trial set. The interpretation is diagnostic and conditional on the search design, sampled ranges, and validation objective.

changes against a dataset-local reference, so positive movement means a feature group is worth considering for final-test selection rather than that the corresponding model with side features has already won on the test split. The observed feature movements are interpreted again with the empirical evidence in Chapter 5, where they remain explicitly separated from final test claims.

Table 4.4: Feature-subset search design.

Configuration family	Question answered	Interpretation rule
Interaction-only reference	What is the reference configuration when no side information is used?	Baseline for judging whether features should be selected for complete testing; absence of side features is a declared protocol state.
All eligible groups with closed or open gates	Do safe descriptors help when exposed together, and does the learned gate suppress them?	Validation diagnostic for configuration construction; not a final side-feature claim.
Single feature group	Which raw feature family has independent validation signal?	Identifies candidate groups such as KuaiRec resolution or KuaiRand category/tags.
Drop-one configuration	Which feature group is useful when removed from the full eligible set?	Checks whether a candidate group remains useful in context.
Pair and selected triple configurations	Do groups complement each other under the validation objective?	Elevates only named combinations to candidate status; does not imply that all feature combinations were exhaustively solved.

The minimum reporting schema is a checklist rather than a full run inventory. A result without an evidence type is not formal evidence. Each empirical table must make the following fields visible or reconstructable:

- *Empirical setting*: dataset view, model family, protocol description, seed policy, and evidence type.
- *Data protocol*: positive-label rule, item-universe policy, feature policy, candidate filtering, and training regime.
- *Ranking evidence*: NDCG, Recall, and HitRatio at the reported cutoffs.
- *List behaviour*: AveragePopularity and Personalization at the reported cutoffs.
- *Resource evidence*: average epoch seconds, peak training VRAM, evaluation chunk status, and feasibility or automatic batch-size status when applicable.
- *Composite utility*: report final-test CRRU and ValidationCRRU together with the declared weights, split role, and visible component metrics.

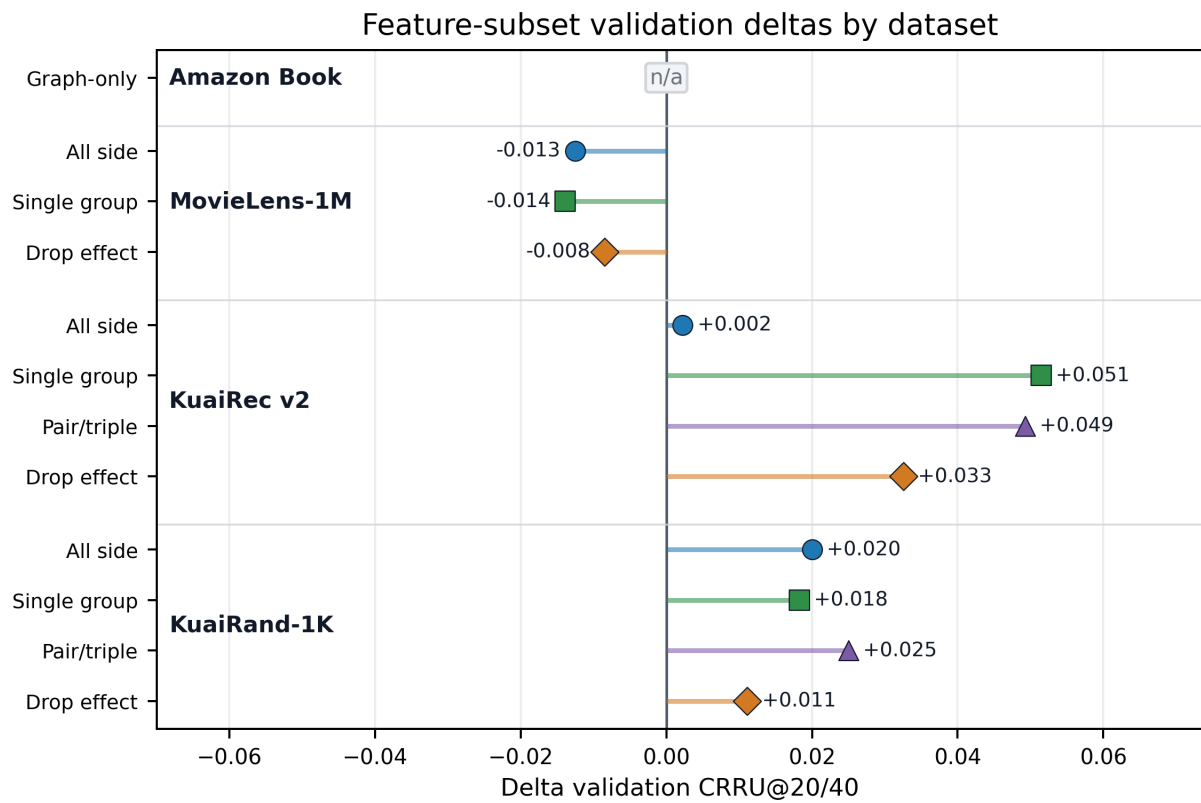


Figure 4.11: Validation-only feature subset movement from the validation feature subset search. The figure summarizes feature-group deltas used for candidate selection and auditability. It must be read as validation/search evidence only: a feature configuration can support a final thesis claim only after a matching complete test evaluation is reported.

The main EDGRec selection path is dataset-local selection from completed validation evidence, followed by formal evaluation under the declared final test protocol. Feature-subset searches, compact validation priors, small-batch sweeps, temporal configurations, and deeper fan-out comparisons are supporting evidence unless they are explicitly rerun as complete test evaluations. The same distinction applies to baselines: a reference final test LightGCN evaluation is not interchangeable with a sampled LightGCN ablation, and a reference DICE runtime feasibility evaluation is not interchangeable with a complete DICE ranking evaluation.

4.4 Baseline Configuration

The baseline set is separated by model family and evidence role. Bayesian Personalized Ranking (BPR) supplies the pairwise implicit-feedback training principle [3]; a standalone Bayesian Personalized Ranking Matrix Factorization (BPR-MF) result would therefore be the natural non-graph reference, but the reported result set does not contain a matched BPR-MF final test evaluation. Reference LightGCN supplies the simplified full-graph collaborative-filtering reference [6]. Reference DICE and its GCN instantiation supply the closest literature comparators for interest and conformity disentanglement, subject to the feasibility constraints recorded below [5].

The evidence-label contract is as follows.

- *Reference LightGCN*: The main graph baseline uses full-graph propagation, the observed train graph only, simplified scoring, and source-specification optimization conventions; hardware-adjusted variants are labelled separately.
- *Reference DICE and its GCN instantiation*: The DICE comparator uses dual-branch training with popularity-conditioned negatives and discrepancy regularization where hardware permits. It is a final ranking baseline only when complete test evaluations exist; single-epoch evaluations are runtime feasibility evaluations.
- *Sampled LightGCN*: This ablation uses EDGRec’s sampled training with one interest branch. It is not a reference LightGCN result.
- *Popularity-conditioned EDGRec ablation*: This component ablation uses EDGRec’s sampled training regime with dual branches and fixed interest and conformity mixing. It is not a reference DICE result or a GCN instantiation of DICE.
- *Resource-adjusted variants*: These evaluations preserve the source architecture but change batch feasibility or execution controls for the local workstation. They are resource evidence and remain separate from reference baselines.

Every empirical baseline table must identify whether an entry is a final test evaluation, validation search candidate, runtime feasibility evaluation, archival record, or component diagnostic. The adjective reference describes the baseline specification, not the evidence quality: a reference configuration becomes a final comparison only when its evidence type is a final test configuration. This requirement is especially important for DICE: when only runtime feasibility evaluations exist for a dataset, the result is a feasibility comparison rather than a final DICE ranking comparison.

4.5 EDGRec Configuration Families

The main EDGRec configuration uses the full EDGRec method from Chapter 3. It operationalizes the interest and conformity motivation from DICE in a practical graph recommender that combines LightGCN propagation, sampled training, and bounded resource cost. The configuration is defined by components rather than by a single model label because branch depth, score fusion, context use, sampling, and resource controls all change both empirical meaning and computational cost.

- *Branches and fusion:* EDGRec uses dual interest and conformity branches with LightGCN propagation and learned bounded fusion over active score components. A small mix floor prevents active components from being silently assigned zero weight and makes score fusion diagnostics meaningful.
- *Context head:* The context head is item-only and uses train-derived popularity, recency, or safe item context. User features do not enter context scoring, which separates item-context effects from user preference embeddings.
- *Sampling and losses:* Popularity-conditioned negatives, following the DICE motivation, are the default contrast policy; standard negatives appear only in explicit ablations. Branch-specific losses, independence regularization, and context popularity supervision make component costs and benefits measurable rather than leaving popularity-related behaviour entirely hidden inside one score.
- *Disabled components:* Contrastive terms, DirectAU-motivated geometry, calibrated IPW, temporal-history interest, and feature fusion are off unless a named configuration enables them. Optional components are therefore not attributed to default EDGRec.
- *Resource controls:* Sampled subgraph training, declared fan-out, batch-size policy, automatic mixed precision (AMP) use, and capped quadratic auxiliaries are reported because resource comparisons depend on the training regime, not only on the scoring formula.

Dataset-specific configurations may change fan-out, branch depth, embedding dimension, batch size, item-universe policy, feature groups, or auxiliary-loss weights. Such changes must be listed with the corresponding comparison because they alter both accuracy and resource consumption. A completed test evaluation is eligible for empirical tables only when its evidence role is explicit. Validation-selected candidates, feature subset searches, temporal configurations, small-batch sweeps, and deeper fan-out comparisons remain supporting evidence unless they are rerun under the declared final test protocol.

The branch-depth, learning-rate, score fusion, fan-out, and runtime diagnostics generated from Optuna are supporting audit material. The body uses their scientific implication rather than treating the dense plots as final results: these quantities must be reported as experimental conditions because they change both accuracy and resource use, while validation search associations remain selection evidence.

Figures 4.12, 4.13, and 4.14 document these validation search associations for auditability. The heatmaps make branch depth visible as a capacity and cost choice, the learning-rate/branch-fusion landscape checks whether optimization and score fusion collapse into narrow regions, and the fan-out/runtime panels show that neighbourhood sampling controls both validation utility and epoch cost. These diagnostics justify reporting the selected configuration.

4.6 Ablation Design

The ablation design isolates EDGRec components by comparing each ablation with a matched base configuration on the same dataset, split, item-universe policy, candidate protocol, hardware convention, and evaluation cutoff. An ablation is not interpreted by CRRU alone. It is first interpreted through ranking metrics, AveragePopularity@ K , Personalization@ K , average epoch duration, peak VRAM, and available branch or score fusion diagnostics. This component-first rule matters because an ablation can improve one objective, such as personalization, while degrading ranking or resource utility.

The available ablation surface is deliberately small: the default EDGRec configuration, EDGRec with side features, EDGRec with contrastive regularization, EDGRec with inverse propensity weighting, EDGRec without the popularity head, and EDGRec without independence regularization. These variants support component-level design interpretation only when matched evaluations actually exist. Dense branch, score fusion, and representation diagnostics remain supporting material; the body interprets ablations through visible metric deltas rather than through hidden model internals or post hoc labels.

- *Mainline EDGRec*: The full default configuration establishes the matched comparison point. Dataset view, split, train graph, candidate filtering, and hardware convention stay fixed.

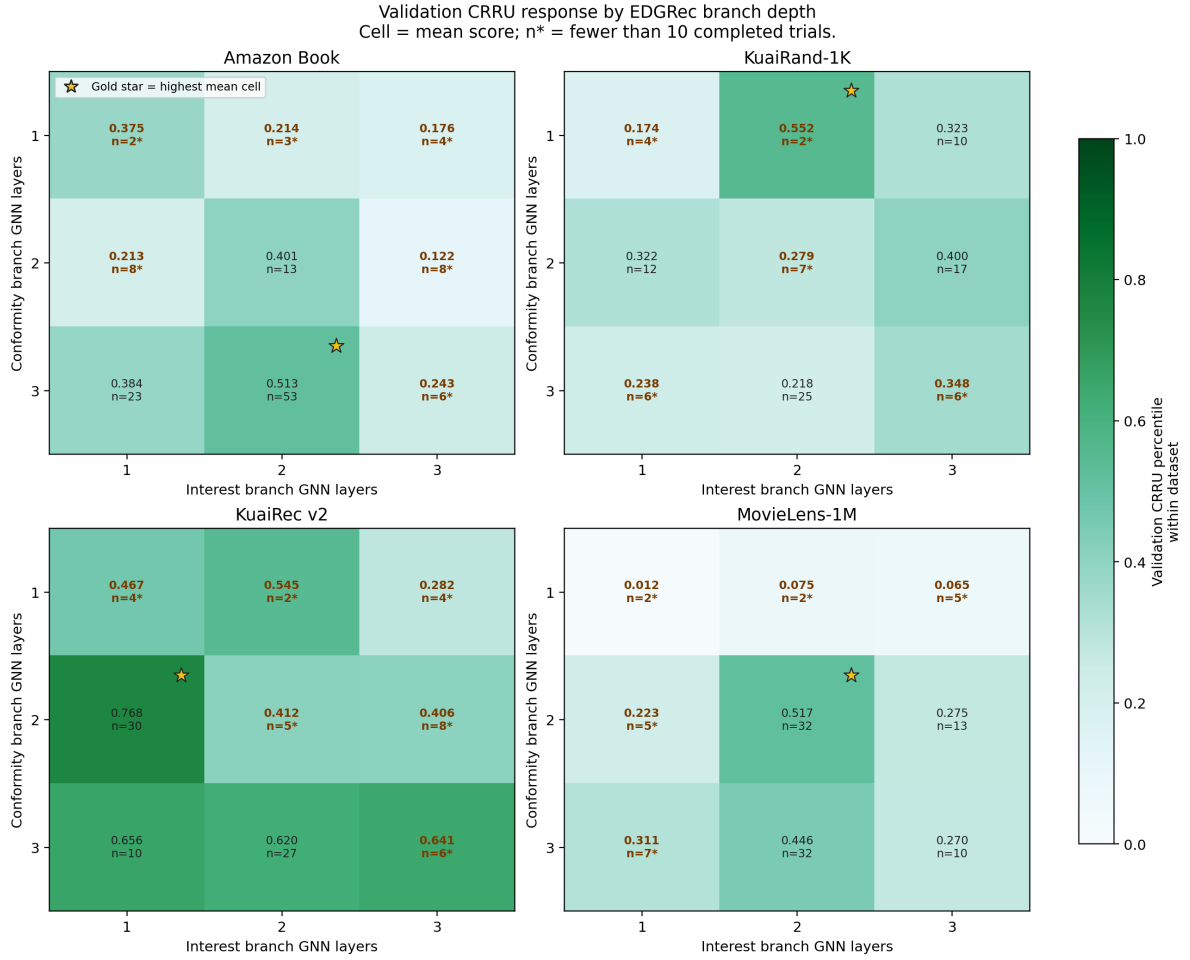


Figure 4.12: Validation search branch-depth diagnostics. The heatmaps summarize how interest and conformity propagation depth interacts with the validation objective in the explored EDGRec configurations. The figure supports configuration reporting and ablation design, with interpretation restricted to the displayed dataset and search-space revision.

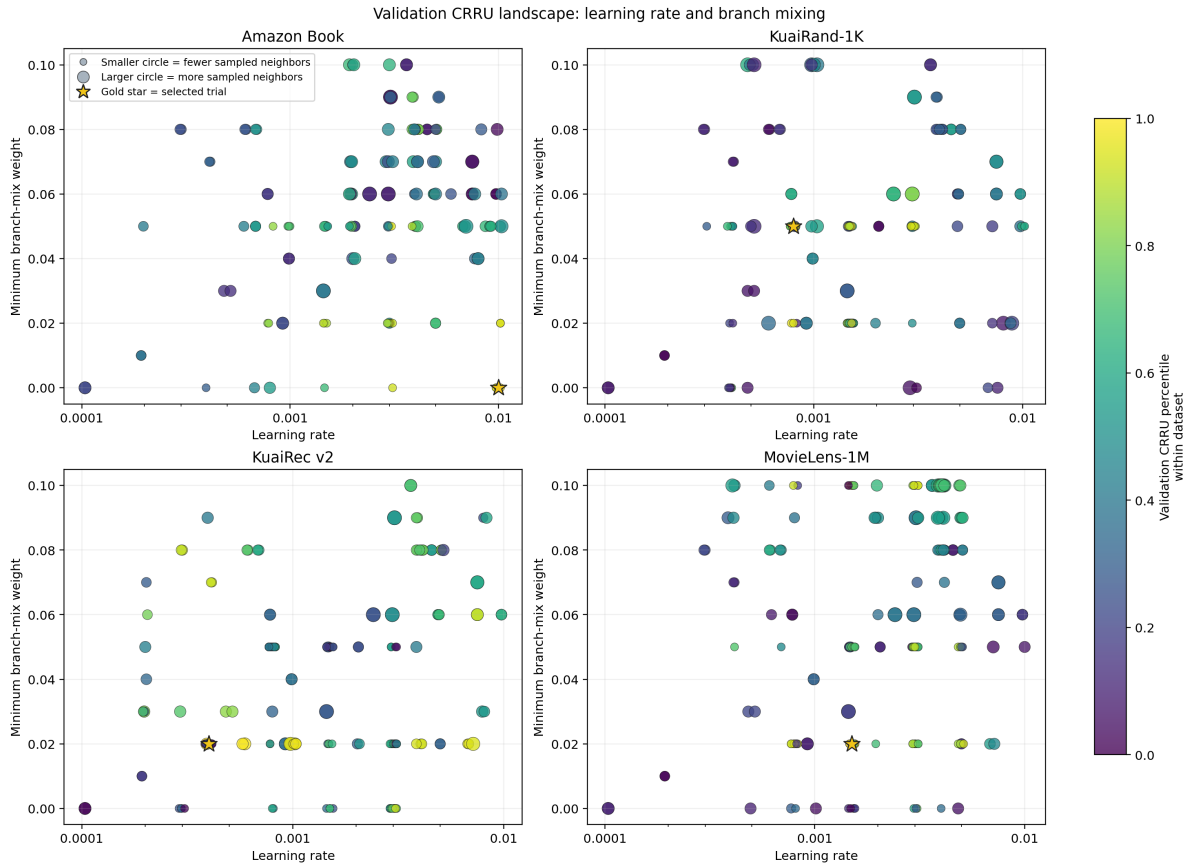


Figure 4.13: Validation search learning-rate and branch-fusion landscape. The visualization records how optimization scale and score fusion controls behave within the validation search. It is included to make candidate selection auditable, especially when a selected configuration is chosen for a trade-off rather than for the single highest ranking metric.

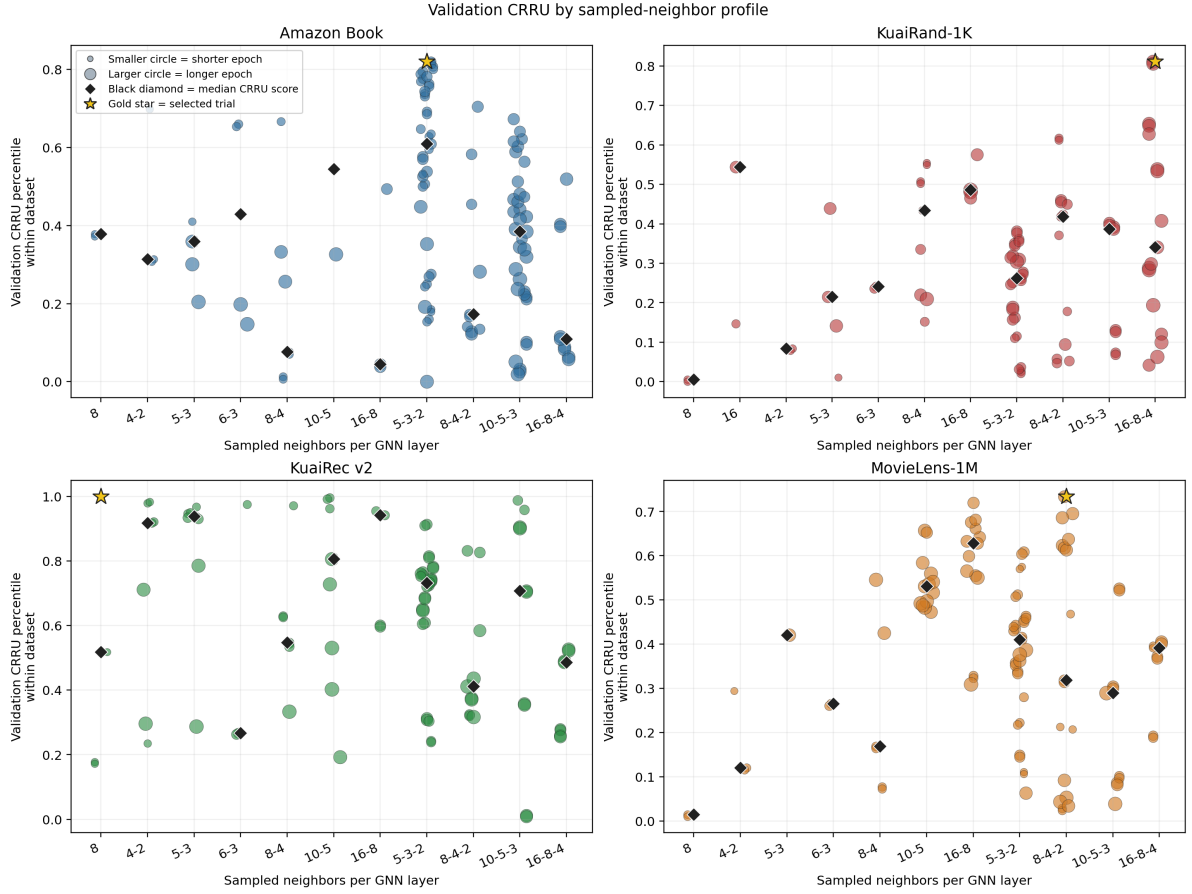


Figure 4.14: Validation search fan-out and runtime trade-offs. The panels relate sampled-neighbourhood choices to validation utility and epoch duration during search. The figure documents resource-sensitive candidate selection and reporting policy; final model comparisons remain grounded in complete test evaluations.

- *Component removals*: The no-popularity-head and no-independence variants remove one modelling component at a time. They test whether the item-only context path and branch-separation regularizer change relevance metrics, cost, or raw train-popularity concentration.
- *Additive components*: Variants with side features, contrastive regularization, and IPW turn on components that are disabled in default EDGRec. They support feature, representation-geometry, or explicit propensity-calibration/IPW diagnostic interpretation only when a matched completed evaluation exists for the same protocol.
- *Remaining design tests*: Complete cross-dataset ablations are still needed for removal of the conformity branch, fixed versus learned fusion, standard versus popularity-conditioned negative sampling, and safe features versus interaction-only inputs. Validation-only versions of these checks can motivate later final-test selection, but final interpretation requires matched complete reruns.
- *Temporal and feature subset evidence*: Temporal-history configurations and feature subset searches are treated as named configurations or validation search diagnostics rather than interchangeable ablation evaluations. Validation-only evidence can motivate reruns, but it remains separate from final test evidence.

Optional ablations are reported only when actual evaluations exist under the current configuration revision. If an optional component has validation-only evidence, the empirical discussion may use it to motivate future complete evaluations while keeping it outside final test interpretation.

4.7 Runtime and Memory Measurement Protocol

The default hardware convention is the local workstation reported in Table 4.5: an NVIDIA RTX 5080 GPU, an AMD Ryzen 9 9900X CPU, and 32 GiB of system RAM. Runtime is reported as average epoch duration unless a table explicitly states another timing statistic. VRAM is reported as peak training VRAM, with the exact logged field stated when multiple profiler fields are available.

These measurements are part of the comparison because full-graph propagation, sampled fan-out, batch size, negative sampling, and evaluation chunking can change the cost profile of graph recommender training [17]. The values therefore characterize the declared workstation protocol, not hardware-independent model constants. Table 4.6 states the reporting fields required for resource comparisons.

The CPU is reported as a 12-core / 24-thread processor rather than as a 24-core processor. Thread count affects data loading and preprocessing capacity, but it is not the number of physical cores. The SMBIOS “Current Speed: 4400 MHz” field is treated as the nominal

Table 4.5: Primary experimental hardware and software environment.

Component	Configuration
CPU	AMD Ryzen 9 9900X, AM5, 12 physical cores / 24 threads. The nominal/base frequency is 4.40 GHz, the reported maximum boost range is approximately 5.65–5.66 GHz, and boost is active under <code>amd-pstate-epp</code> .
GPU	NVIDIA GeForce RTX 5080, 16 GB GDDR7 VRAM, 10,752 CUDA cores.
System memory	32 GiB DDR5 system memory, installed as 2×16 GiB Kingston KF560C30-16 dual in-line memory modules (DIMMs), non-error-correcting-code (non-ECC). System Management BIOS (SMBIOS) reports 4800 mega-transfers per second (MT/s) configured speed; the DDR5-6000 latency-class-30 Extended Profiles for Overclocking (EXPO) profile is a separate memory setting and must be explicitly enabled before claiming 6000 MT/s.
Storage	Kingston NV3 NVMe SSD.
Motherboard	Gigabyte B850 AORUS Elite WIFI7 ICE.
Operating system	CachyOS x86_64. The Linux kernel version is recorded per run or environment export; the local environment export records 7.1.2-3-cachyos.
Software stack	Python, PyTorch [33], PyTorch Geometric, Optuna, CUDA, and related accelerator-library versions are logged per experiment rather than assumed from the thesis text alone.
Execution mode	Single-machine local experiments on the workstation GPU.

configured memory speed, not as a live per-core CPU clock. CPU boost behaviour and memory-profile configuration are therefore recorded separately: processor boost is active under the reported Linux performance policy, whereas DDR5-6000 EXPO must be enabled explicitly before memory-speed claims use the DIMM profile rather than the SMBIOS 4800 MT/s setting.

All reported runtime and VRAM measurements are hardware-dependent. Epoch duration, maximum feasible batch size, fan-out limits, and peak VRAM are interpreted as measurements under the local hardware/software environment rather than as architecture-intrinsic constants. This is particularly important for EDGRec because its sampled subgraph path, auxiliary losses, embedding tables, and full-catalog evaluation can shift the runtime and memory bottleneck across training and evaluation stages.

Peak VRAM is reported from the experiment logger using PyTorch CUDA memory counters where available. When PyTorch peak allocated memory is used, it corresponds to the maximum GPU memory occupied by tensors since the last reset of the CUDA memory statistics. When reserved memory or external profiler values are reported instead, the table explicitly names the logged field.

Runtime feasibility evaluations are retained because reference full-graph baselines may be expensive or infeasible under a fixed hardware budget. A feasibility evaluation enters the resource-feasibility ledger and becomes eligible for final ranked result tables only after rerun under the formal protocol. In particular, reference DICE runtime feasibility evaluations

Table 4.6: Runtime and memory reporting contract.

Field	Required reporting convention	Reason
Hardware	GPU, CPU, RAM, CUDA/AMP convention when available.	Resource conclusions are hardware-dependent.
Average epoch duration	Prefer logged average epoch time; otherwise state the fallback, such as total training time divided by completed epochs or feasibility-evaluation seconds per epoch.	Prevents evaluation-only or model-state loading time from being confused with training throughput.
Peak training VRAM	Report peak training VRAM in MB and name the profiler field when ambiguity exists.	EDGRec can be faster while using more VRAM; both dimensions are needed.
Training regime	Full-graph propagation or sampled-subgraph training, fan-out list, batch-size policy, automatic batch-size status, and negative-sampling strategy.	Speed differences can be caused by the training regime rather than model simplicity.
Evaluation policy	Candidate filtering, full-catalog scoring status, evaluation chunk size, and diagnostic scorer status.	Evaluation can create a different memory bottleneck from training.
Runtime feasibility status	Label single-epoch, reduced-data, CPU fallback, automatic batch-size feasibility, or approximation evaluations.	Feasibility evaluations support feasibility analysis but not final accuracy claims.

support systems feasibility statements, while completed matched reruns would be required for final DICE ranking quality. The corresponding feasibility figure appears in Chapter 5 beside the DICE feasibility-evaluation table, where the visual scale difference is interpreted as resource evidence.

4.8 Threats to Experimental Validity

The protocol is designed to make the main validity threats explicit at reporting time. The checklist below states how each threat constrains empirical interpretation.

- *Observational feedback:* Logged interactions reflect exposure, ranking policy, popularity, and user response; missing entries are unobserved outcomes rather than verified dislikes. Top- K metrics are interpreted as offline relevance under the logged protocol [4, 10].
- *Negative sampling:* Sampled negatives may contain relevant but unexposed items. They are treated as contrast items for pairwise optimization, consistent with the positive-only BPR setting [3].
- *Graph and feature leakage:* Validation and test interactions are excluded from message passing, popularity, recency, recent histories, popularity-conditioned masks, and

CRRU popularity denominators. Post-treatment engagement aggregates are excluded from primary scoring features, and every configuration with side features is labelled.

- *Validation overfitting and selection effects:* Search-space revisions require fresh-trial budgets, and test metrics are reserved for configurations selected before final testing. Validation objectives, including ValidationCRRU, are interpreted as selection evidence rather than as final generalization evidence.
- *Limited robustness:* Single-seed or limited-seed evidence identifies empirical regimes but does not quantify variance across initialization, sampling, search, or split construction.
- *Dataset heterogeneity:* AmazonBook, MovieLens-1M, KuaiRec, and KuaiRand differ in feedback semantics, density, exposure evidence, and side information. Conclusions are stated per dataset before synthesis.
- *Resource variability:* Runtime and VRAM vary with hardware, CUDA allocation, batch-size fallback, evaluation chunks, fan-out, and diagnostics. Resource claims report hardware, training regime, batch policy, evaluation policy, and feasibility status.
- *Composite utility dependence:* CRRU can change model ordering when weights or transformations change. Claims driven by CRRU require visible components and sensitivity checks.
- *Comparator coverage:* Reference DICE is available as runtime feasibility evidence in the present result set. Those evaluations support systems discussion; final DICE ranking comparisons require completed matched test evaluations.

These validity conditions also determine the language used in the empirical results chapter. EDGRec may be described as informed by causal recommendation, motivated by disentanglement, bias-aware, or resource-aware when the evidence supports those terms. Claims about causal effects, recovered latent factors, protected-group outcomes, or deployment-wide system value require dedicated evaluation designs beyond the present protocol.

4.9 Experimental Protocol Summary

This chapter fixes the experimental contract for the remainder of the thesis. The default comparison uses train-only graph construction, explicit dataset protocols, validation-only candidate selection, labelled evidence categories, separation between reference baselines and diagnostic baselines, matched EDGRec ablations, and hardware-aware runtime reporting. The empirical reporting contract requires NDCG, Recall, HitRatio, AveragePopularity,

Personalization, time per epoch, and peak VRAM to remain visible before CRRU is used as a secondary summary of the declared trade-off.

Reproducibility information is distributed deliberately: Chapter 3 defines train-only construction, model, metric, seed, and logging conventions; Chapter 4 defines the experimental contract, evidence categories, hardware environment, and runtime reporting; Appendix [A.2](#) records the environment checklist; and the supplementary experimental records store the effective configurations, experiment logs, and final-result provenance.

Chapter 5

Results and Discussion

The empirical evidence for the Efficient Disentangled Graph Recommender (EDGRec) is reported under the experimental contract from Chapter 4. The numerical values are backed by archived experiment records, while the main tables foreground the scientific comparison factors: dataset view, model family, training regime, evaluation cutoff, item-universe policy where relevant, and evidence status. EDGRec is interpreted through offline ranking, list behaviour, resource, and diagnostic evidence; dataset-specific protocol differences are part of the scientific interpretation rather than background bookkeeping.

5.1 Result Reporting Protocol

The evidence labels from Chapter 4 govern interpretation throughout the results. Completed final test evaluations support ranking, popularity-aware list behaviour, and resource claims under their declared dataset protocol. Supporting evaluations and diagnostic analyses are used for the comparison they instantiate, such as matched engineering comparison, feasibility analysis, or component diagnostics; primary dataset-level claims are reserved for complete test evaluations. ValidationCRRU and feature subset movements provide model-selection evidence; final-test CRRU is interpreted after a selected configuration has been evaluated on the test split. Component-wise interpretation therefore precedes each dataset synthesis.

Component metrics are reported before any scalar utility. A ranking claim must be supported by Normalized Discounted Cumulative Gain (NDCG)@ K , Recall@ K , and HitRatio@ K under a declared dataset and evaluation protocol. A popularity or personalization claim must show AveragePopularity@ K and Personalization@ K directly. AveragePopularity@ K is the raw train-popularity count of recommended items; lower values mean lower concentration on train-popular items under the logged protocol, while exposure propensity and fairness require separate metrics. Raw AveragePopularity is most interpretable within a matched training graph and item universe. CRRU is then used as a declared post hoc utility over visible components. Unless a table states otherwise, all epoch-time and peak VRAM values

in this chapter use the local workstation and measurement conventions reported in Table 4.5 and Table 4.6.

5.2 Comparison Structure

Table 5.1 summarizes the empirical comparisons used in the results. Internal identifiers are omitted because the scientific comparison factors are the dataset view, model family, training regime, item universe, and evidence status. Additional EDGRec configurations are treated as selection or sensitivity evidence unless explicitly included as completed test evaluations. The reporting order follows the experimental contract: data and preprocessing context, validation search and feature-subset evidence, completed test evaluations, CRRU decomposition, feasibility evaluations, matched ablations, dataset-level discussion, research-question answers, and limitations.

Table 5.1: Reader-facing structure of the empirical comparisons.

Dataset	Evaluation setting	Models compared	Interpretation boundary
AmazonBook	Sparse interaction-only collaborative filtering.	LightGCN full graph; EDGRec sampled dual branch.	Metrics favour LightGCN for ranking and list behaviour; EDGRec contributes a speed trade-off.
MovieLens-1M	Explicit ratings converted to implicit top- K recommendation.	LightGCN full graph; EDGRec sampled dual branch.	Close ranking with an EDGRec speed benefit under the converted-rating protocol.
KuaiRec v2	Sparse watch-ratio short-video recommendation with pronounced exposure and popularity structure.	LightGCN full graph; selected EDGRec sparse watch-ratio evaluation.	Main positive EDGRec ranking-and-runtime result; dense small-matrix evaluations remain separate sensitivity evidence.
KuaiRand-1K	Randomized-exposure short-video setting with compact item-universe variants.	Supporting LightGCN; compact EDGRec.	Diagnostic stress-test evidence: compact EDGRec improves runtime, peak VRAM, and Recall, while stronger LightGCN evidence remains preferable on NDCG, HitRatio, CRRU, and raw popularity concentration.

5.3 Data and Preprocessing Context

The final comparisons inherit the dataset contracts, positive-label rules, split masks, train-only graph construction, feature policy, and item-universe definitions from Chapters 3 and 4. This context is stated before the search and result tables because a metric value is comparable only within the same dataset view, split, preprocessing rule, and candidate

policy. Chapter 4 provides the dataset profile figures and train-degree distributions; the present chapter therefore does not repeat those setup diagnostics, but it keeps their interpretation scope active when reading the results. In particular, AmazonBook is interaction-only, MovieLens-1M is a rating-to-implicit task, KuaiRec v2 separates sparse watch-ratio evidence from dense small-matrix sensitivity evidence, and KuaiRand-1K requires care when compact randomized-item-universe evaluations are compared with non-compact evaluations.

5.4 Validation Search and Feature Evidence

Validation search establishes model-selection evidence. The Optuna studies and feature subset searches are therefore read before the completed test tables so that candidate selection is separated from final empirical comparison. The search evidence supports the following bounded interpretation: KuaiRec v2 and MovieLens-1M contain EDGRec candidate configurations for final test evaluation; AmazonBook remains interaction-only in the reported feature policy; and KuaiRand-1K contains compact randomized-exposure trade-off candidates whose broad ranking interpretation still requires comparable final test evidence under matched item-universe policies.

Optuna study groupings are treated as provenance labels, not as public ablation names. Chapter 4 defines the validation-to-test gate and the ValidationCRRU objective; the present section records which feature groups were eligible for selection and which ones remain candidate configurations rather than final test claims. The validation numbers therefore identify where a configuration deserves a final test rerun, not where side features have already improved the final test result.

The feature-engineering contribution is an auditable chain from raw feature eligibility to leakage exclusion, encoding, validation selection, and final-test status. Chapter 3 defines which columns are eligible; Chapter 4 defines how eligible feature groups are searched; Table 5.2 states what the reported evidence supports. This chain is especially important because safe feature eligibility, validation movement, and final test improvement are separate claims.

This evidence explains why the thesis uses only selected feature groups in primary claims. A column must first be eligible under the leakage policy, then belong to an encoded feature group, then show enough validation evidence to motivate a named configuration, and finally receive final test support before it can appear in a primary result. The supported interpretation is dataset-specific: AmazonBook is interaction-only; MovieLens genres are documented but not supported; KuaiRec safe descriptors are candidate features; and KuaiRand feature movements remain validation-only evidence. Compact or protocol-qualified evaluations therefore cannot be used retroactively to attribute an improvement to the validation-selected feature groups. Under this rule, excluded engagement aggregates are not failed features;

Table 5.2: Feature-subset evidence and final test status.

Dataset	Best validation configuration	Validation movement	Final test status	Thesis use
Amazon-Book	Interaction-only; ValidationCRRU 0.144116.	No eligible side-information group.	Side-information evaluation is inapplicable.	Interaction-only evidence.
MovieLens-1M	Interaction-only; ValidationCRRU 0.262050.	All eligible features -0.013 ; genre -0.014 .	No complete feature-enabled test evaluation.	No feature claim.
KuaiRec v2	Resolution-only feature group; ValidationCRRU 0.215856.	Resolution $+0.051$; video metadata plus resolution $+0.049$; all groups $+0.002$.	No complete sparse feature-enabled test evaluation.	Validation-only candidate.
KuaiRand-1K	Author and music, upload-time, and category or tag groups; ValidationCRRU 0.062106.	All groups $+0.020$; best triple $+0.025$.	No matched complete feature-enabled test evaluation.	Validation-only candidate.

they are inadmissible ordinary predictors for the declared task because they can encode post-exposure outcomes.

The Optuna global-best rows are handled with the same boundary. The validation archive selects dataset-local candidates by reconstructed ValidationCRRU@20–40 when all required ranking, list-behaviour, time, VRAM, and train-popularity denominator inputs are available. Some global validation candidates use very large automatic batches, high peak VRAM, feature gates, reduced training-edge policies, or accuracy-optimized studies whose CRRU is reconstructed post hoc. Those rows are useful for final-test selection decisions and for the search diagnostics in Figures 4.8–4.14, but they are not substituted for the completed test comparisons in Tables 5.4–5.7. This distinction is why the chapter can report, for example, validation evidence for KuaiRec resolution features and KuaiRand feature combinations while still treating the displayed final results as interaction-only or protocol-qualified unless a matched feature-enabled test rerun exists.

Table 5.3 records the resulting bridge from validation evidence to final-test interpretation. The table is intentionally conservative: it states what validation evidence motivated and whether that evidence has been converted into a primary Chapter 5 claim.

Figure 5.1 is retained as a supplementary qualitative feature-space diagnostic. It is useful for inspecting whether encoded descriptors co-locate with train popularity or signed-feedback patterns, but it does not change the validation-only status of the feature evidence in Table 5.2. Its Uniform Manifold Approximation and Projection (UMAP) neighborhoods show how encoded safe item descriptors, train-degree popularity, and non-positive-sign overlays co-locate visually; they do not estimate feature importance or establish that feature configurations improve final ranking [34].

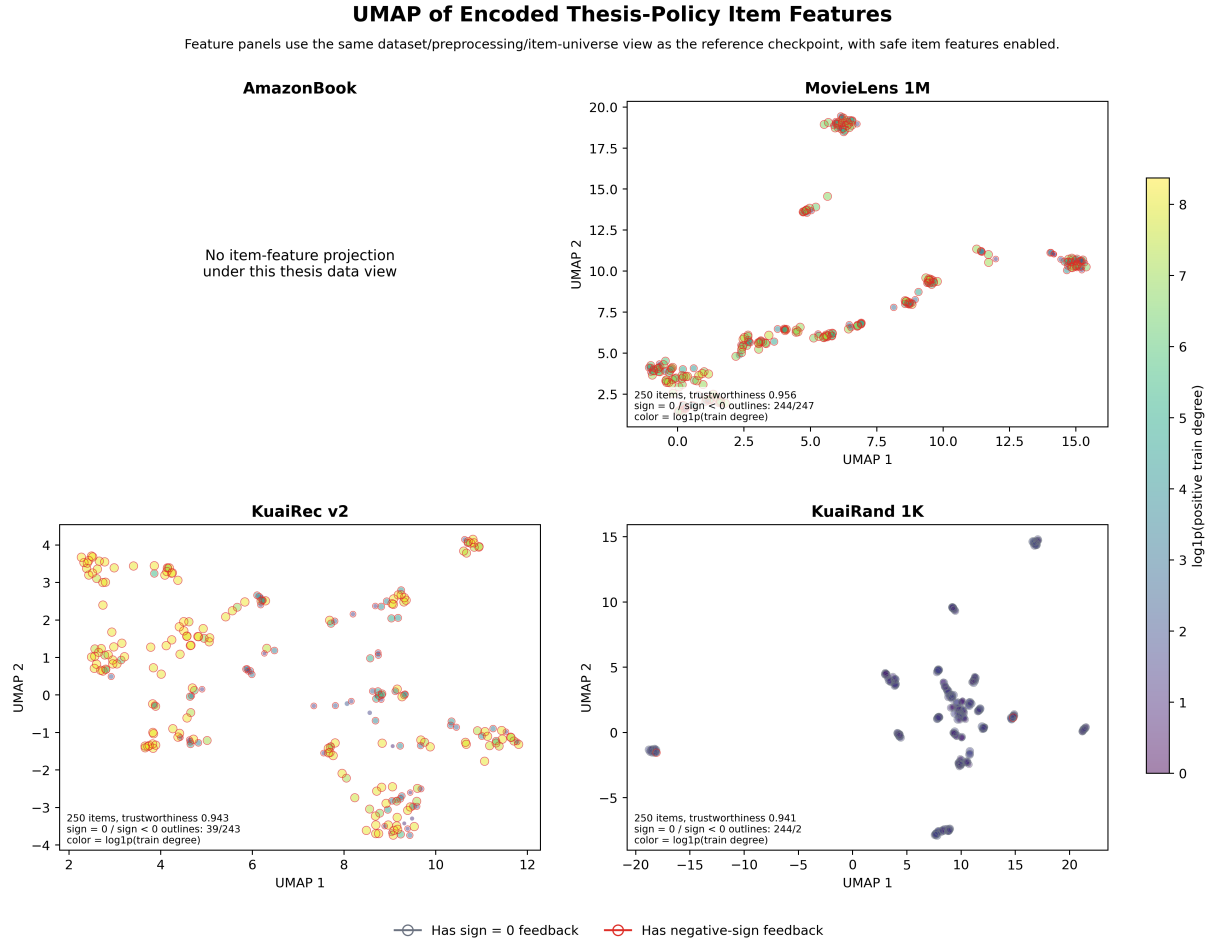


Figure 5.1: UMAP projections of encoded thesis-policy item features [34]. Points are items and color denotes $\log(1 + \text{positive train degree})$, so darker high-degree regions indicate where feature-space neighbourhoods coincide with train-popular items. Black and red outlines mark sampled items with $\text{sign} = 0$ or $\text{sign} < 0$ training feedback where available. AmazonBook has no panel because it is interaction-only under the thesis feature policy. The figure supports feature-engineering interpretation and column-selection discussion; feature usefulness still depends on the validation and final test evidence in Table 5.2.

Table 5.3: Validation-to-test status for the main EDGRec evidence.

Dataset	Validation evidence used	Displayed final-test EDGRec evidence	Interpretation boundary
AmazonBook	Interaction-only global and feature-subset candidates; no eligible side-information group.	EDGRec sampled dual-branch test rows compared with reference LightGCN.	Ranking remains less favourable than LightGCN; speed is the supported EDGRec benefit.
MovieLens-1M	Near-parity EDGRec candidates; genre features did not improve ValidationCRRU relative to interaction-only.	Core EDGRec test profile compared with reference LightGCN.	Close ranking and faster epochs, but no final feature or popularity-concentration claim.
KuaiRec v2	ValidationCRRU search and feature-subset evidence, including resolution-only candidates.	Sparse watch-ratio EDGRec test profile compared with reference LightGCN.	Primary favourable EDGRec result; feature-specific improvement remains validation-only.
KuaiRand-1K	Compact randomized-exposure and feature-combination candidates.	Compact and supporting EDGRec evaluations compared with stronger completed LightGCN evidence.	Resource and Recall stress-test evidence, not an unqualified accuracy win.

5.5 Dataset-Wise Metric Tables

Tables 5.4–5.7 report completed test evaluations one dataset at a time. This layout keeps the metrics vertical and places the compared models in the header, so ranking metrics, list-behaviour diagnostics, resource measurements, and CRRU utility values can be read together rather than across separate tables. Higher is better for metrics marked $\uparrow$; lower is better for metrics marked $\downarrow$.

The final three columns report the absolute delta from the displayed LightGCN comparator to EDGRec, the multiplicative ratio, and the relative percentage delta. The absolute delta is $\text{EDGRec} - \text{LightGCN}$, the ratio is $\text{EDGRec}/\text{LightGCN}$, and the percentage delta is $(\text{EDGRec} - \text{LightGCN})/\text{LightGCN} \times 100\%$, always using the displayed LightGCN comparator as the denominator. AmazonBook, MovieLens-1M, and KuaiRec v2 use the reference full-graph LightGCN comparator; KuaiRand-1K uses the completed LightGCN comparison with stronger top-position ranking evidence because the reference full-graph LightGCN row is not the most informative completed evidence for that stress-test regime.

Positive absolute and percentage deltas, or ratios above $\times 1$, mean EDGRec has a larger numeric value; ratios between $\times 0$ and $\times 1$ mean EDGRec has a lower numeric value. The metric arrow determines whether that direction is preferable. Within each dataset table, bold values mark the preferred value among the displayed evaluations for that metric; bold values are table-local and do not override the evidence-role boundaries from Chapter 4.

Some evaluations differ in training regime or item-universe policy, so table-local bolding is interpreted through the prose qualifications rather than as a universal model ranking. The

table headers use compact model labels to reserve space for the three delta views; captions and surrounding prose state the full training regime boundary for each displayed evaluation.

Figure 5.2 gives a compact overview of the same LightGCN comparisons before the detailed tables. It is included after the validation search evidence because it summarizes final test comparisons rather than model-selection evidence. The colours encode signed $\log_2$ fold-change so that gains and losses are visible in one panel, while the printed cell values keep the underlying metric scale auditable. The figure maps eligible comparisons but does not replace the tables: evidence-role labels, missing-baseline status, and dataset-specific protocol caveats still control the interpretation. The KuaiRec small-matrix dense-observation sensitivity evaluations are cross-referenced later in Figure 5.10 rather than merged into the main sparse-regime claim.

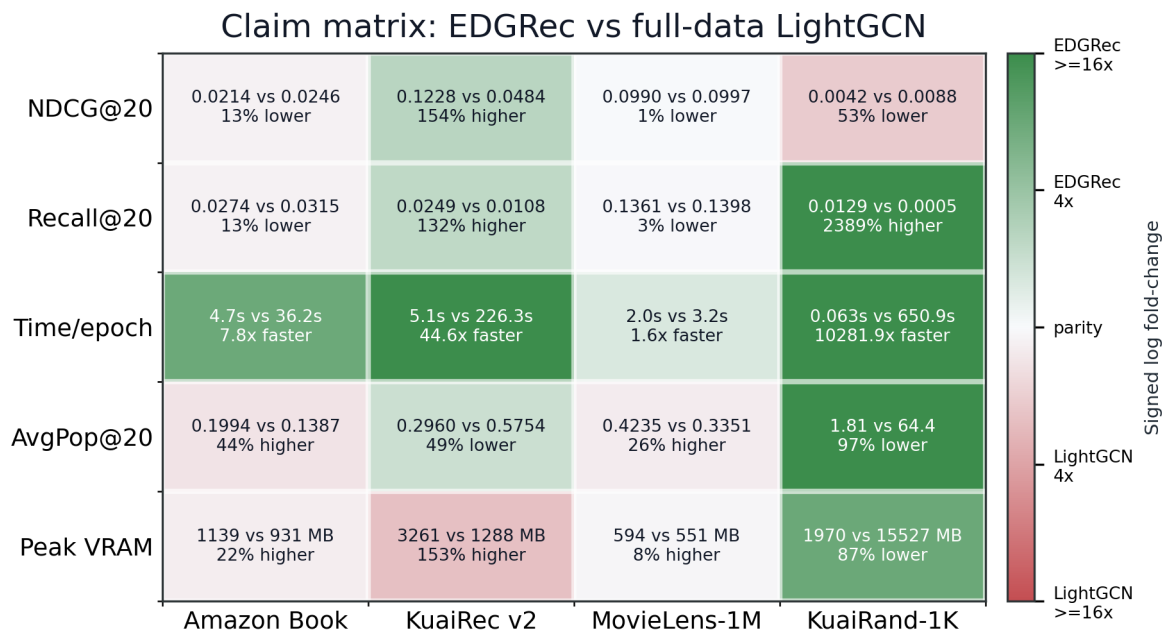


Figure 5.2: Claim matrix for the main final test comparison against reference LightGCN. Green cells indicate an EDGRec advantage and red cells indicate a LightGCN advantage under the displayed metric. The color scale is a signed $\log_2$ fold-change relative to LightGCN, clipped at the plotted maximum for readability; the cell text reports raw values. The sparse KuaiRec watch-ratio evaluation is used as the primary KuaiRec comparison, while dense small-matrix sensitivity evaluations are excluded from this main comparison figure.

The dataset-wise results are heterogeneous. On AmazonBook, LightGCN has preferable values on every reported ranking metric, the list-behaviour diagnostics, the CRRU utility value, and peak VRAM, while EDGRec supports an epoch-time trade-off. On MovieLens-1M,

Table 5.4: AmazonBook metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.

Metric	Light-GCN	EDGRec	Δ abs.	$\times$ ratio	Δ (%)
NDCG@20 ($\uparrow$)	0.0246	0.0214	−0.0032	$\times 0.87$	−13.01%
Recall@20 ($\uparrow$)	0.0315	0.0274	−0.0041	$\times 0.87$	−13.02%
HitRatio@20 ($\uparrow$)	0.1958	0.1801	−0.0157	$\times 0.92$	−8.02%
NDCG@40 ($\uparrow$)	0.0326	0.0284	−0.0042	$\times 0.87$	−12.88%
Recall@40 ($\uparrow$)	0.0528	0.0463	−0.0065	$\times 0.88$	−12.31%
HitRatio@40 ($\uparrow$)	0.2931	0.2741	−0.0190	$\times 0.94$	−6.48%
CRRU@20 ($\uparrow$)	0.1232	0.1177	−0.0055	$\times 0.96$	−4.46%
Average Popularity@20 ($\downarrow$)	0.1387	0.1994	+0.0607	$\times 1.44$	+43.76%
Personalization@20 ($\uparrow$)	0.9781	0.9069	−0.0712	$\times 0.93$	−7.28%
Epoch (s) ($\downarrow$)	36.2	4.7	−31.5	$\times 0.13$	−87.02%
VRAM (MB) ($\downarrow$)	931	1139	+208	$\times 1.22$	+22.34%

Table 5.5: MovieLens-1M metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.

Metric	Light-GCN	EDGRec	Δ abs.	$\times$ ratio	Δ (%)
NDCG@20 ($\uparrow$)	0.0997	0.0990	−0.0007	$\times 0.993$	−0.70%
Recall@20 ($\uparrow$)	0.1398	0.1361	−0.0037	$\times 0.974$	−2.65%
HitRatio@20 ($\uparrow$)	0.4977	0.4979	+0.0002	$\times 1.0004$	+0.04%
NDCG@40 ($\uparrow$)	0.1274	0.1258	−0.0016	$\times 0.987$	−1.26%
Recall@40 ($\uparrow$)	0.2257	0.2177	−0.0080	$\times 0.965$	−3.54%
HitRatio@40 ($\uparrow$)	0.6490	0.6375	−0.0115	$\times 0.982$	−1.77%
CRRU@20 ($\uparrow$)	0.2722	0.2700	−0.0022	$\times 0.992$	−0.81%
Average Popularity@20 ($\downarrow$)	0.3351	0.4235	+0.0884	$\times 1.26$	+26.38%
Personalization@20 ($\uparrow$)	0.9329	0.8634	−0.0695	$\times 0.93$	−7.45%
Epoch (s) ($\downarrow$)	3.2	2.0	−1.2	$\times 0.62$	−37.50%
VRAM (MB) ($\downarrow$)	551	594	+43	$\times 1.08$	+7.80%

Table 5.6: KuaiRec v2 sparse watch-ratio metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.

Metric	Light-GCN	EDGRec	Δ abs.	$\times$ ratio	Δ (%)
NDCG@20 ($\uparrow$)	0.0484	0.1228	+0.0744	$\times 2.54$	+153.72%
Recall@20 ($\uparrow$)	0.0108	0.0249	+0.0141	$\times 2.31$	+130.56%
HitRatio@20 ($\uparrow$)	0.4164	0.7133	+0.2969	$\times 1.71$	+71.30%
NDCG@40 ($\uparrow$)	0.0462	0.1214	+0.0752	$\times 2.63$	+162.77%
Recall@40 ($\uparrow$)	0.0193	0.0457	+0.0264	$\times 2.37$	+136.79%
HitRatio@40 ($\uparrow$)	0.5717	0.8300	+0.2583	$\times 1.45$	+45.18%
CRRU@20 ($\uparrow$)	0.1239	0.2056	+0.0817	$\times 1.66$	+65.94%
Average Popularity@20 ($\downarrow$)	0.5754	0.2960	-0.2794	$\times 0.51$	-48.56%
Personalization@20 ($\uparrow$)	0.9695	0.8591	-0.1104	$\times 0.89$	-11.39%
Epoch (s) ($\downarrow$)	226.3	5.1	-221.2	$\times 0.023$	-97.75%
VRAM (MB) ($\downarrow$)	1288	3261	+1,973	$\times 2.53$	+153.18%

Table 5.7: KuaiRand-1K diagnostic metric profile across ranking, list behaviour, and resource evidence under the local workstation protocol.

Metric	Light-GCN	EDGRec	Δ abs.	$\times$ ratio	Δ (%)
NDCG@20 ($\uparrow$)	0.0202	0.0042	-0.0160	$\times 0.21$	-79.21%
Recall@20 ($\uparrow$)	0.0007	0.0129	+0.0122	$\times 18.43$	+1,742.86%
HitRatio@20 ($\uparrow$)	0.3000	0.0177	-0.2823	$\times 0.06$	-94.10%
NDCG@40 ($\uparrow$)	0.0186	0.0047	-0.0139	$\times 0.25$	-74.73%
Recall@40 ($\uparrow$)	0.0012	0.0148	+0.0136	$\times 12.33$	+1,133.33%
HitRatio@40 ($\uparrow$)	0.4560	0.0243	-0.4317	$\times 0.05$	-94.67%
CRRU@20 ($\uparrow$)	0.0564	0.0513	-0.0051	$\times 0.91$	-9.04%
Average Popularity@20 ($\downarrow$)	0.3642	1.8131	+1.4489	$\times 4.98$	+397.83%
Personalization@20 ($\uparrow$)	0.9656	0.9933	+0.0277	$\times 1.03$	+2.87%
Epoch (s) ($\downarrow$)	25.3	0.063	-25.237	$\times 0.0025$	-99.75%
VRAM (MB) ($\downarrow$)	12008	1970	-10,038	$\times 0.16$	-83.59%

EDGRec is close at $K = 20$, including a marginally higher HitRatio@20, but remains slightly lower in NDCG and Recall at both cutoffs and less favourable on the list behaviour metrics. These two datasets show why speed is not interpreted as a standalone win: efficiency matters only as part of the reported Pareto trade-off.

On KuaiRec v2, the sparse EDGRec watch-ratio evaluation is favourable relative to the reference LightGCN baseline on NDCG, Recall, HitRatio, CRRU, lower raw train-count AveragePopularity, and epoch time. This is the most favourable EDGRec result, with Personalization and peak VRAM still favouring LightGCN.

On KuaiRand-1K, compact EDGRec is a protocol-qualified stress test: Recall, Personalization, epoch time, and peak VRAM are favourable, but NDCG, HitRatio, CRRU, and raw train-count AveragePopularity are less favourable than the LightGCN comparison that provides the stronger top-position ranking evidence. The large Recall ratios arise from small reference denominators and should be read with the absolute Recall values. Candidate and item-universe differences make KuaiRand evidence a compact item-universe feasibility and sensitivity result rather than an unqualified model-superiority claim.

5.6 Resource-Aware Utility and Diagnostic Plots

CRRU@20 is reported in each dataset-wise table, but the scalar is only interpretable through its components. It uses the declared weighted geometric utility from Equation (3.44):

$$\text{CRRU}_K = \text{RankingAccuracy}_K^{0.55} \cdot \text{PopularityAwarePersonalization}_K^{0.30} \cdot \text{TrainingResourceUtility}^{0.15}.$$

RankingAccuracy uses $\text{NDCG}^{0.50}$, $\text{Recall}^{0.35}$, and $\text{HitRatio}^{0.15}$. PopularityAwarePersonalization uses $\text{Personalization}^{0.40}$ and inverse log-normalized AveragePopularity^{0.60}. TrainingResourceUtility uses inverse-log transformations of peak VRAM and epoch time with equal inner weights. CRRU therefore summarizes the declared resource-aware trade-off for this study, while conclusions remain grounded in the visible component metrics in the dataset-wise tables.

Figure 5.3 decomposes CRRU after its visible inputs have been reported. It is placed here so the scalar utility is interpreted beside the ranking, list behaviour, runtime, and peak VRAM components that define it. The grouped bars should be read component-wise: a high CRRU can come from different mixtures of accuracy, popularity-aware personalization, and resource utility, so the figure explains how the declared utility is formed.

Figure 5.4 and Figure 5.5 provide companion diagnostics for inspecting additional completed evaluations, while Tables 5.4–5.7 and Figure 5.3 remain the primary trade-off evidence. This keeps the main claim tied to visible numeric components rather than to dense diagnostic scatter plots. The accuracy–efficiency frontier shows whether completed evaluations occupy useful NDCG/runtime regions, including outlier context, while the accuracy–popularity plot

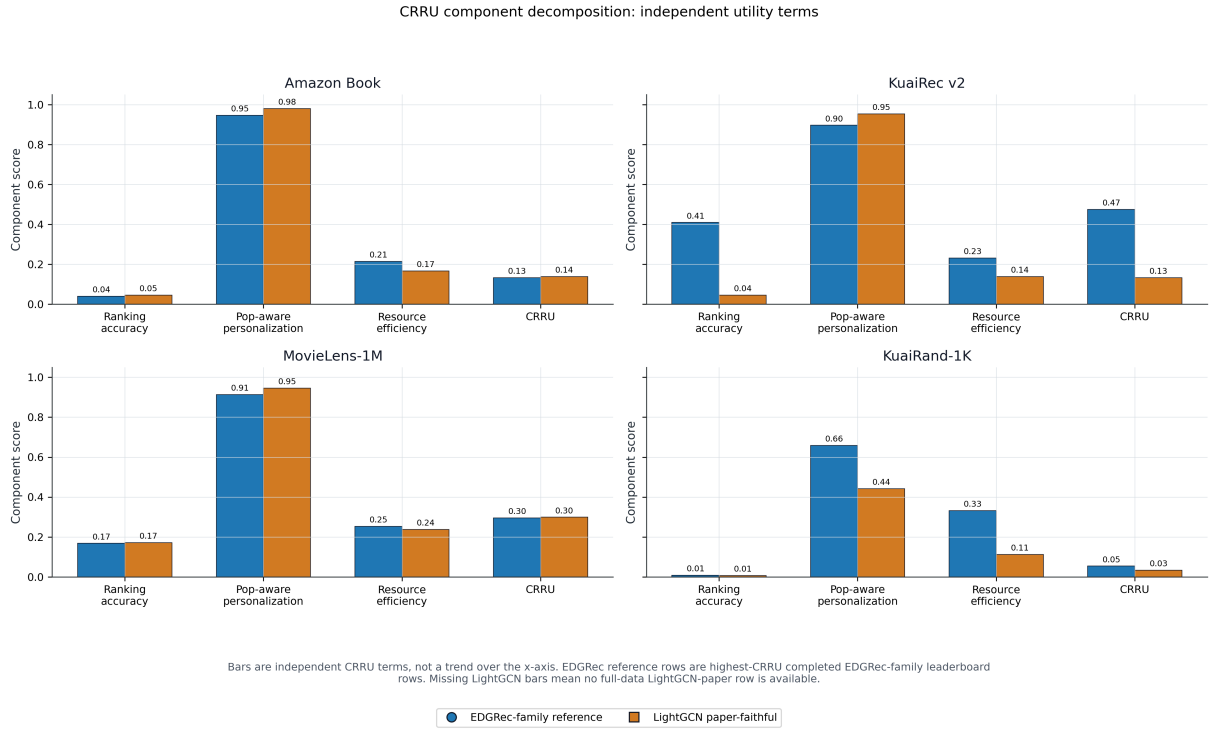


Figure 5.3: CRRU component decomposition. The grouped bars show which visible components drive the scalar utility under the declared weights. The figure explains the utility; it does not claim that the chosen weights are uniquely correct, and conclusions that depend on CRRU should still be checked against the component metrics.

checks whether lower popularity concentration coincides with accuracy loss or gain. Both plots are descriptive over completed evaluations and preserve the evidence roles, protocol boundaries, and dataset-specific comparison rules established in Chapter 4.

The resource results support configuration-specific efficiency claims with mixed VRAM evidence. They should be read as training regime measurements on the declared workstation, because full-graph propagation, sampled fan-out, batch-size policy, and item-universe choices directly change the active computational graph.

- *AmazonBook*: EDGRec is approximately $7.7\times$ faster per epoch than LightGCN, but it has lower ranking quality, higher raw AveragePopularity, lower Personalization, and higher peak VRAM.
- *MovieLens-1M*: EDGRec is approximately $1.6\times$ faster per epoch and remains close in ranking quality, but LightGCN keeps slightly higher CRRU and Personalization, as well as lower raw AveragePopularity and lower peak VRAM.
- *KuaiRec v2*: the selected sparse EDGRec evaluation is approximately $44.4\times$ faster per epoch than reference LightGCN and has higher ranking metrics plus lower raw AveragePopularity; it also uses more peak VRAM and has lower Personalization, so the favourable result is ranking-and-runtime evidence with mixed list behaviour and VRAM evidence.

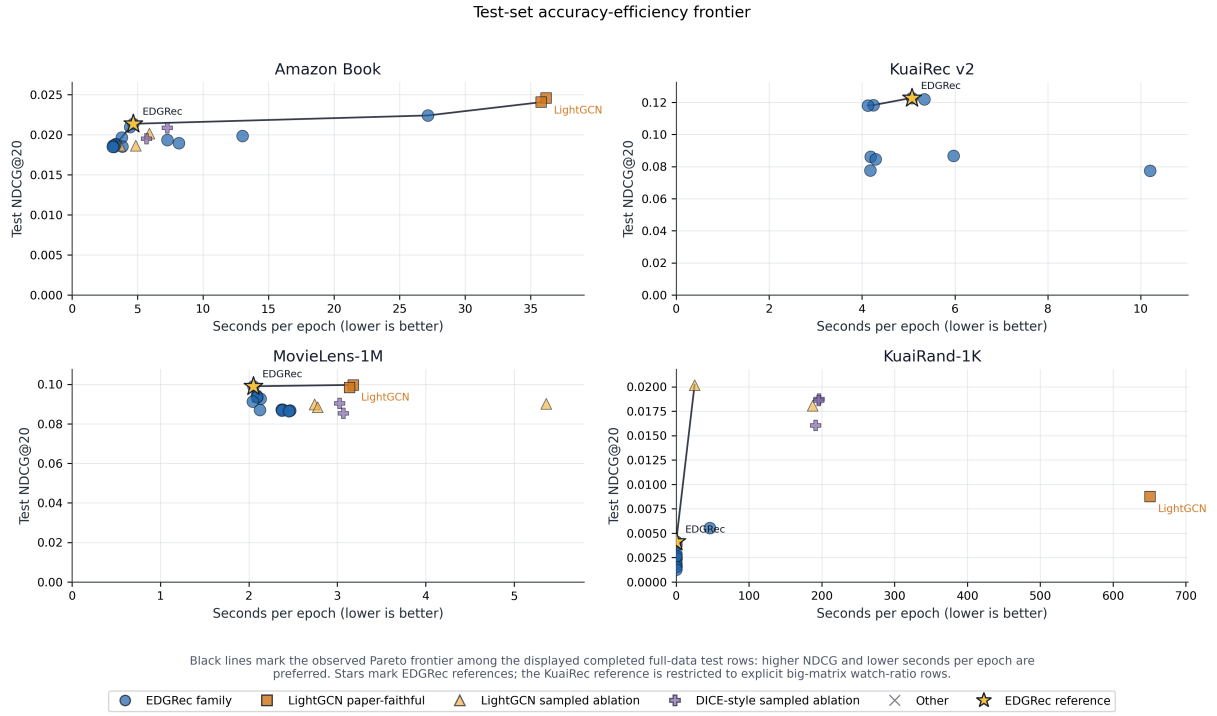


Figure 5.4: Full-range accuracy–efficiency frontier over completed test evaluations. The figure relates ranking quality to seconds per epoch so that the resource-aware claim can be read as a Pareto trade-off rather than inferred from a wide table. It is a supplementary diagnostic; the metric tables remain the primary evidence.

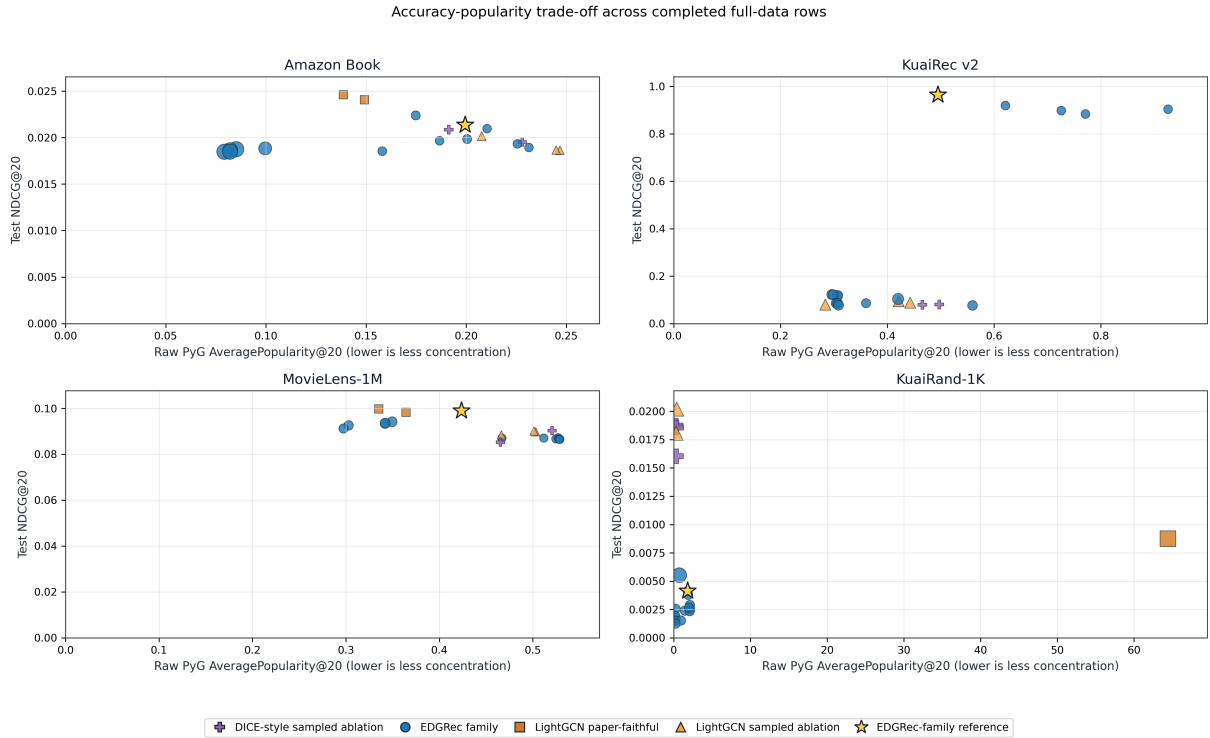


Figure 5.5: Full-range accuracy–popularity–concentration trade-off over completed test evaluations. The figure relates ranking quality to raw AveragePopularity, making the RQ2 list behaviour component visible beside the ranking evidence. Lower AveragePopularity indicates lower concentration on train-popular items within the logged protocol. It is a supplementary diagnostic rather than a replacement for the dataset-wise metric tables.

- *KuaiRand-1K*: compact EDGRec is substantially faster and uses less peak VRAM than the supporting LightGCN comparison and improves Recall and Personalization, but it is lower on NDCG, HitRatio, CRRU, and raw AveragePopularity. The result is a protocol-qualified resource and Recall trade-off rather than a positive top-position quality or scalar-utility claim.

The speedups in Tables 5.4–5.7 are training-regime evidence rather than evidence that the EDGRec scoring function is intrinsically simpler. The reference LightGCN evaluations preserve full-graph propagation, so each epoch is governed by operations on the complete positive training graph under the baseline’s fixed training convention.

EDGRec instead trains on bounded sampled neighbourhoods around mini-batch triples, uses larger feasible batches where the hardware allows them, applies vectorized popularity-conditioned sampling while excluding known positives, and caps expensive auxiliary estimators. The speed differences are therefore consistent with the implemented training paths even though EDGRec has more score components: per-epoch cost is governed by sampled subgraphs, fan-out, negative-sampling count, batch-size resolution, and auxiliary caps rather than by an exact full-graph propagation pass at every optimizer step.

This mechanism also explains why several favourable outcomes are driven primarily by efficiency rather than by accuracy. EDGRec changes the training path before it changes the evaluation target: it reduces the active graph per update, amortizes optimizer overhead through larger feasible batches, and bounds auxiliary computations. The resulting speedups are methodologically substantive because they enlarge the feasible experimental frontier under a fixed workstation budget: more validation candidates, ablations, diagnostics, and eventually repeated seeds can be executed for the same resource envelope. The contribution is therefore practical as well as modelling-oriented: EDGRec turns a causal-disentanglement motivation that can be expensive to train into a protocol that can be evaluated, tuned, and audited on the available hardware. Relevance evidence remains primary, so an efficient configuration is scientifically useful when its ranking, list behaviour, and memory trade-offs remain visible; a speedup accompanied by a large accuracy loss is interpreted as a resource trade-off rather than as general model superiority.

The benefit of interest and conformity separation is dataset- and protocol-dependent. It can improve ranking when the dataset protocol supplies useful popularity, exposure, or item-context structure for the extra branches and context path; on sparse interaction-only or converted-rating regimes, the same structure may add capacity and sampling differences without enough additional relevance signal. EDGRec’s supported contribution is therefore a practical training and evaluation trade-off: the branch hypothesis motivated by DICE is embedded in an efficient graph-recommendation pipeline, and its value is judged by dataset-specific relevance, list behaviour, resource, and ablation evidence. This explains why several positive outcomes are primarily efficiency outcomes: the method changes the feasible optimization and validation budget before it guarantees a uniform accuracy gain. Accuracy

comparisons against reference baselines remain valid only under the declared evidence role, and speed comparisons must name the training regime, hardware, and item-universe policy.

5.7 Reference DICE Feasibility

The available DICE rows are single-epoch feasibility evaluations, and Figure 5.6 with Table 5.8 report them as resource evidence. This boundary is scientifically important because DICE is the closest literature comparator for interest and conformity disentanglement, while EDGRec is a methodological adaptation that preserves the interest and conformity motivation under a different sampled-training and scoring contract. The figure compares single-epoch feasibility cost with completed EDGRec resource evaluations to document the resource gap that motivates the efficient training contract; ranking comparisons against DICE and its GCN instantiation require complete final test evaluations under the same reporting contract.

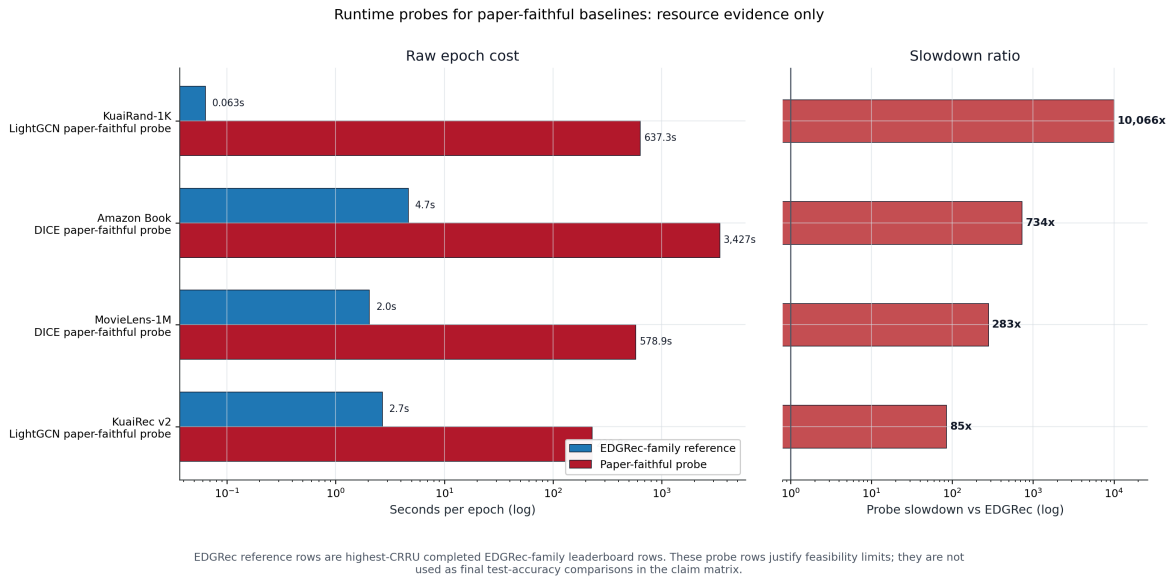


Figure 5.6: Reference DICE feasibility evaluations. The paired log-time bars show the single-epoch DICE feasibility cost beside completed EDGRec resource evaluations and annotate epoch-time ratios. The figure supports a feasibility statement and motivates complete matched DICE evaluations for future ranking comparison.

The DICE evaluations are single-epoch feasibility evaluations, so their NDCG and Recall remain diagnostic. Their role is resource-specific: they show that reference DICE is substantially more expensive than the completed EDGRec resource evaluations under the declared workstation and experimental protocol. These ratios compare logged epoch times and peak VRAM for the thesis experimental pipeline and workstation; they support feasibility interpretation rather than end-to-end deployment speedups or final DICE accuracy comparisons.

Table 5.8: Reference DICE feasibility evaluations against completed EDGRec evaluations.

Dataset	EDGRec resource evaluation	Reference DICE single-epoch feasibility	DICE/EDGRec ratio	Evidence role
Amazon-Book	Epoch: 4.7 s	Epoch: 3426.2 s	$\approx 730\times$	Resource feasibility only; final DICE ranking evidence is unavailable.
	VRAM: 1139 MB	VRAM: 5197 MB	$\approx 4.6\times$	
MovieLens-1M	Epoch: 2.0 s	Epoch: 578.8 s	$\approx 280\times$	Resource feasibility only; final DICE ranking evidence is unavailable.
	VRAM: 594 MB	VRAM: 2899 MB	$\approx 4.9\times$	

The large epoch-time ratios are consistent with the reference DICE regime, which keeps dual full-graph branch propagation, the published multi-negative sampling convention, exact per-user correction logic where required, and discrepancy regularization between branches. EDGRec keeps the interest and conformity motivation but changes the training regime through sampled subgraphs, one or few contrast items in the default configurations, vectorized popularity-conditioned routing, and bounded auxiliary losses. The thesis therefore supports a resource feasibility advantage over the evaluated reference DICE specification; final DICE ranking quality remains outside the present final test evidence.

5.8 Ablation Evidence

The available matched ablation evidence is limited to KuaiRec v2 evaluations from an earlier matched protocol. Table 5.9 reports deltas relative to that matched EDGRec reference configuration rather than to the completed sparse watch-ratio evaluation, and Figure 5.7 visualizes the same component-removal direction across accuracy, list behaviour, and resource fields. Negative ΔNDCG and negative ΔCRRU indicate that the ablation is less favourable than its matched reference configuration; positive $\Delta\text{AveragePopularity}$ means recommendations concentrate more on train-popular items.

Table 5.9: Matched KuaiRec v2 EDGRec ablations under an earlier protocol.

Component removed	Δ NDCG@20	Δ AvgPop@20	Δ Pers.@20	Δ CRRU@20
Popularity head	-0.0149	+0.3044	+0.0616	-0.0007
Independence regularization	-0.0208	+0.1045	+0.0343	-0.0033

The ablation evidence is consistent with keeping both the popularity head and independence regularization as KuaiRec design priors, but only under the matched protocol used for those evaluations. Removing the popularity head lowers NDCG and CRRU while sharply increasing raw AveragePopularity; the Personalization gain shown in Table 5.9 and Figure 5.7 does not recover the utility loss. Removing the independence regularizer also

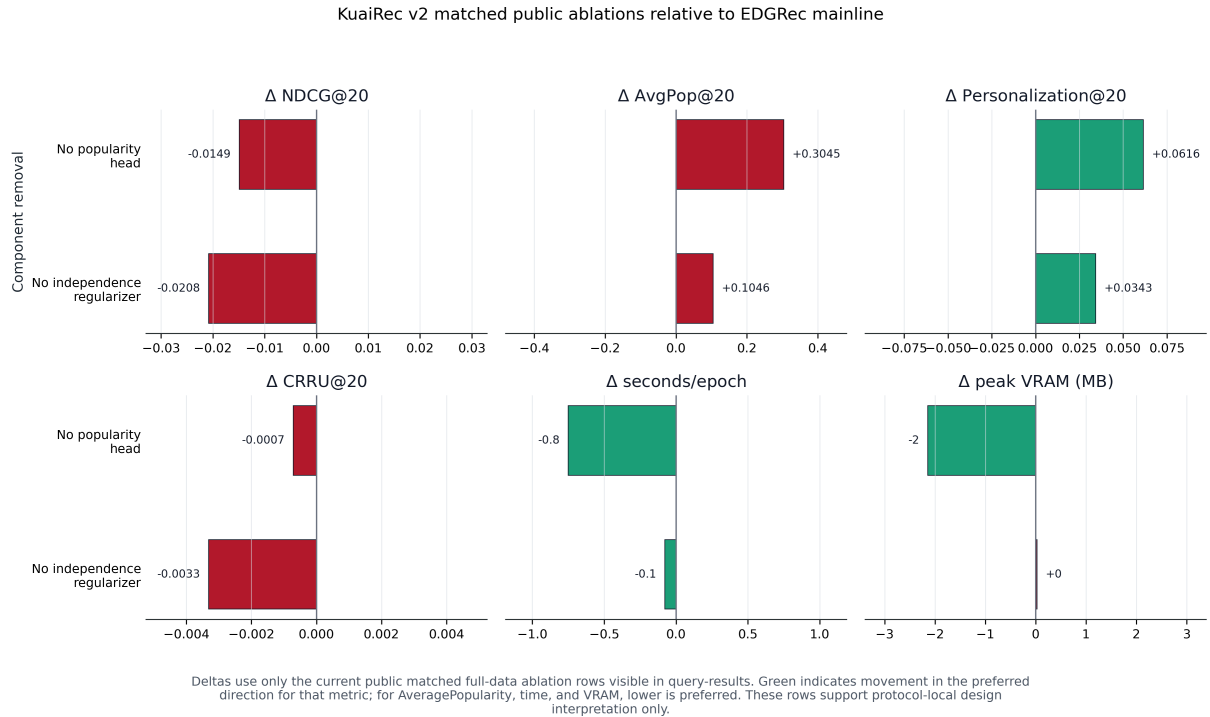


Figure 5.7: KuaiRec v2 matched ablation deltas relative to the matched EDGRec reference configuration. Each panel keeps the metric in its original units; green bars denote movement in the preferred direction for that metric and red bars denote movement away from it. The figure supports a bounded design interpretation for the available KuaiRec ablations; dataset-general component necessity would require matched ablations on the other datasets.

lowers NDCG and CRRU and increases raw AveragePopularity. The conservative interpretation is that both components help keep the branch/context split from behaving as a single popularity-concentrated scorer in this KuaiRec protocol; the evidence does not isolate the same mechanism on the other datasets.

This component evidence is intentionally narrow. It supports a KuaiRec v2 component interpretation for the matched ablation protocol, while the completed sparse watch-ratio result and the other datasets require their own matched component evaluations. Complete ablations for removing the conformity branch, fixed versus learned fusion, negative sampling alone, and safe features versus interaction-only inputs remain the most important missing component comparisons.

Branch, score fusion, and learned-embedding diagnostics are qualitative checks. They support consistency analysis, while the principal component-level evidence remains the matched ablation deltas in Table 5.9 and Figure 5.7. Figure 5.8 summarizes whether score fusion, branch alignment, and branch ranking behaviour are consistent with the intended component roles, while Figure 5.9 visualizes learned interest-embedding neighborhoods. These plots are read alongside the metric tables and ablation deltas; their interpretation is diagnostic rather than evidence of recovered latent causes or causal disentanglement.

EDGRec mechanism diagnostics for reference rows

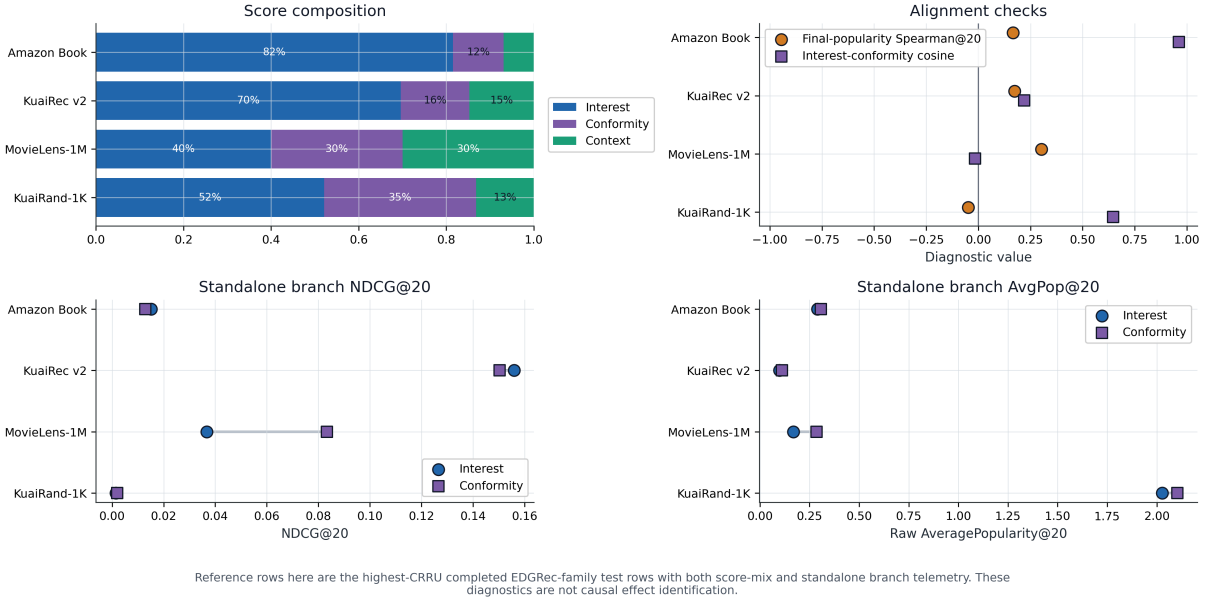


Figure 5.8: EDGRec component diagnostics. The panels summarize score fusion, branch alignment, and branch ranking behaviour for the available diagnostic evaluations. These diagnostics support a consistency check of branch usage, with interpretation remaining metric- and protocol-bound rather than identified latent-factor recovery.

5.9 Dataset-Level Discussion

5.9.1 AmazonBook

AmazonBook is a sparse interaction-only collaborative-filtering benchmark in this thesis. The reference LightGCN evaluation is higher on NDCG, Recall, HitRatio, Personalization, and CRRU, while also having lower raw AveragePopularity and lower peak VRAM. This pattern is consistent with the available evidence: the thesis uses no side-feature or exposure signal for AmazonBook, so EDGRec’s additional branch structure has little information beyond the same sparse collaborative graph while sampled training changes the optimization regime. EDGRec reduces average epoch time from 36.2 s to 4.7 s, so the dataset supports an efficiency trade-off rather than a recommendation-quality improvement.

5.9.2 MovieLens-1M

MovieLens-1M is an explicit-rating dataset converted into an implicit top- K task. It does not provide exposure logs, so the result is interpreted as converted-rating recommendation evidence. EDGRec is close to LightGCN in NDCG@20 and HitRatio@20 and is faster per epoch, but LightGCN remains slightly higher in Recall, CRRU, and Personalization, with lower raw AveragePopularity and lower peak VRAM. The close-ranking pattern is consistent with both methods relying primarily on the same converted collaborative signal; the less

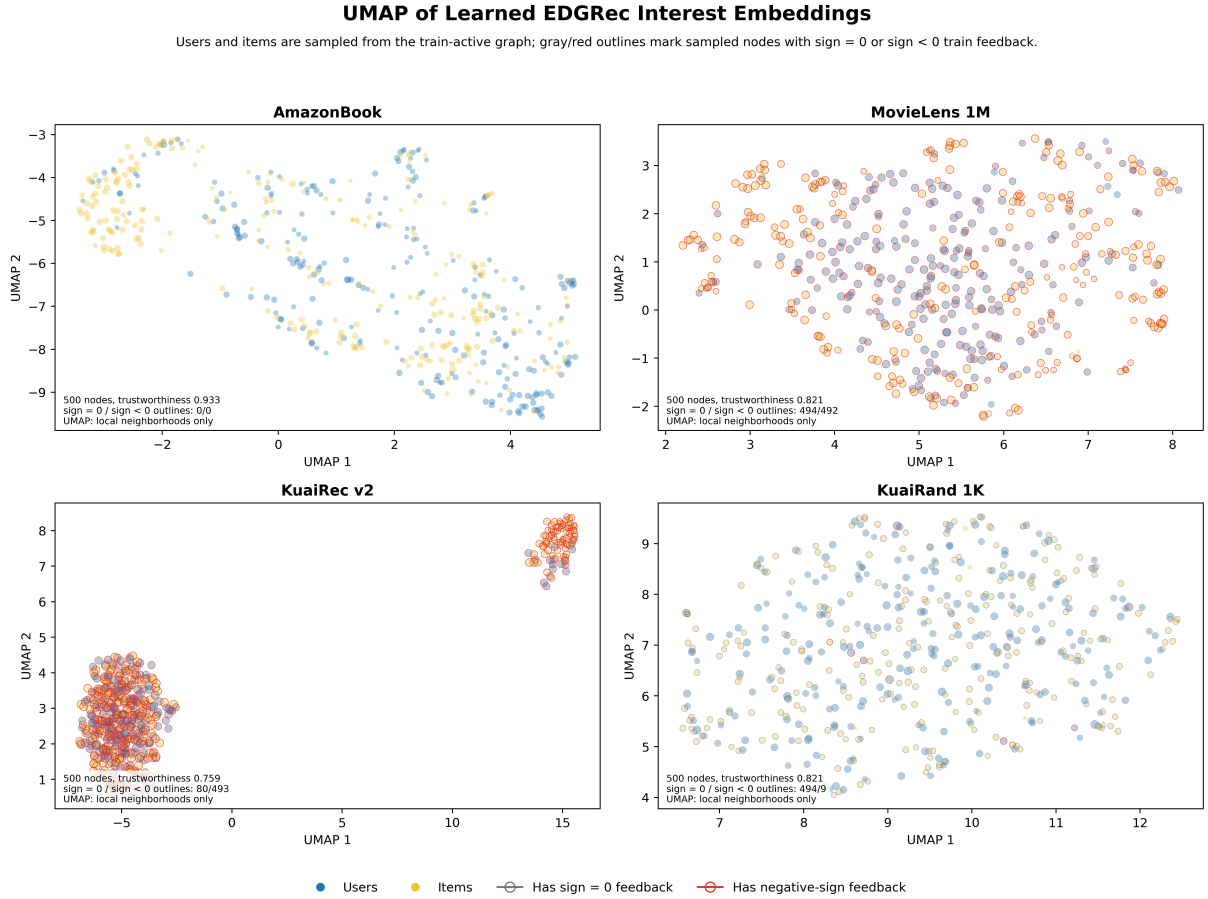


Figure 5.9: UMAP projections of propagated EDGRec interest embeddings from the reference model states [34]. Blue points are users, yellow points are items, semi-transparent markers make denser regions visually darker, and black/red outlines mark sampled nodes that have sign = 0 or sign < 0 training feedback. The panels are qualitative diagnostics of learned embedding geometry and signed-feedback exposure; they must be interpreted with the metric tables, ablations, and branch diagnostics, and they do not establish ranking superiority or causal disentanglement.

favourable list behaviour diagnostics indicate that EDGRec did not improve popularity concentration or inter-user list diversity under this protocol. The conclusion is closely matched top- K ranking with a speed benefit.

5.9.3 KuaiRec v2

KuaiRec v2 provides the most favourable primary EDGRec evidence under the sparse watch-ratio setting. The selected sparse EDGRec evaluation is higher than the reference LightGCN evaluation on ranking metrics and is substantially faster per epoch.

This pattern is consistent with a setting where the watch-ratio protocol contains pronounced exposure and popularity structure, and where EDGRec allocates model capacity to interest, conformity, and item-context components while using sampled graph training. The available ablations and diagnostics support a component-consistency interpretation rather than a single-component attribution.

Numerically higher dense small-matrix evaluations exist in the supporting result set, but they belong to a different evidence role unless they are rerun under the same primary protocol. Figure 5.10 separates these protocol regimes visually so the dense small-matrix evidence is not conflated with the sparse watch-ratio result. The current KuaiRec evidence is favourable for ranking and runtime, with mixed memory and list behaviour diagnostics.

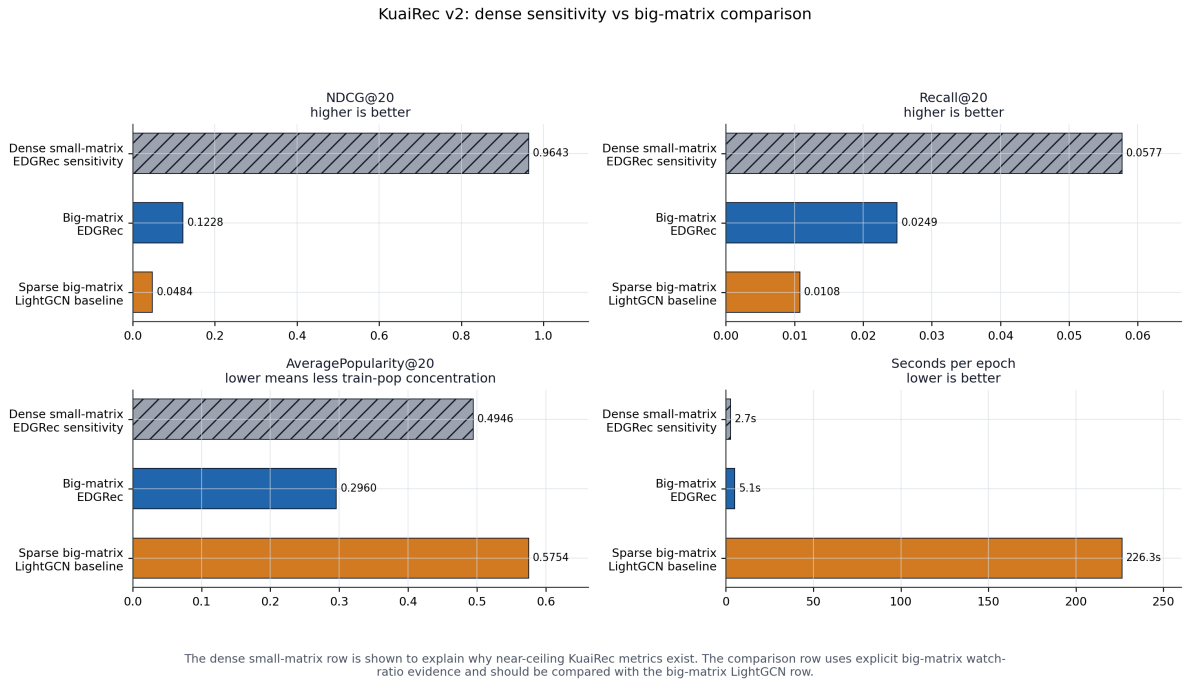


Figure 5.10: KuaiRec protocol-sensitivity companion. The figure separates sparse watch-ratio evidence from dense small-matrix sensitivity evaluations so that the reader does not conflate the sparse primary comparison with a different observation regime.

5.9.4 KuaiRand-1K

KuaiRand-1K is an exposure-aware short-video setting evaluated with the same ranking and resource metric families as the other datasets. Compact EDGRec uses a reduced randomized-exposure item-universe policy and has substantially lower runtime and peak VRAM than the LightGCN comparison that provides the stronger top-position ranking evidence. Because the item-universe policy is part of the protocol, this result is read as compact exposure-aware trade-off evidence rather than as a single matched accuracy claim. Its Recall and Personalization are higher, but NDCG, HitRatio, CRRU, and raw AveragePopularity are less favourable. The large Recall percentage deltas arise from a small LightGCN denominator and should be read together with the absolute Recall values.

The overall pattern is consistent with compact randomized-exposure item universes reducing the scoring and training burden while changing the candidate regime and making precision at the highest ranks unstable under sparse positives. Additional completed LightGCN configurations under different non-reference settings have higher NDCG and HitRatio, and at least one supporting LightGCN evaluation has much lower raw AveragePopularity than the compact EDGRec evaluation. The non-identical item-universe and candidate policies therefore make KuaiRand a protocol-qualified case study in randomized exposure, sparse positives, and efficient graph training rather than a general ranking result.

5.10 Answers to the Research Questions

RQ1. The ranking answer is dataset-specific. EDGRec is less favourable than LightGCN on AmazonBook, close but slightly lower on MovieLens-1M, favourable on the sparse KuaiRec watch-ratio comparison, and mixed on KuaiRand-1K because Recall is higher while NDCG and HitRatio are lower in the displayed comparison. The most favourable ranking evidence for EDGRec is therefore the KuaiRec sparse watch-ratio comparison rather than a uniform improvement across all datasets.

RQ2. The list behaviour answer is also a trade-off. AmazonBook and MovieLens-1M favour LightGCN on raw AveragePopularity and Personalization. The displayed sparse KuaiRec v2 comparison favours EDGRec on raw AveragePopularity but favours LightGCN on Personalization. KuaiRand-1K favours compact EDGRec on Personalization but not on raw AveragePopularity against the LightGCN comparison with stronger top-position ranking evidence. AveragePopularity is therefore read as a train-count concentration diagnostic within the logged protocol rather than as a standalone popularity-bias result.

RQ3. EDGRec gives substantial epoch-time reductions in selected configurations, especially on AmazonBook, KuaiRec v2, and KuaiRand-1K. Peak VRAM is not uniformly

improved: EDGRec uses more peak VRAM than LightGCN on AmazonBook, MovieLens-1M, and the displayed sparse KuaiRec v2 comparison, but less peak VRAM than the supporting LightGCN comparison on compact KuaiRand-1K. Reference DICE evidence has a narrower role because the available DICE evaluations are single-epoch feasibility evaluations; they support resource feasibility discussion for the current pipeline rather than final recommendation-quality comparison.

RQ4. The clearest component evidence comes from matched KuaiRec v2 ablations. In those evaluations, removing the popularity head or the independence regularizer lowers NDCG@20 and CRRU@20 and increases AveragePopularity@20 relative to the matched reference configuration. Feature-subset results support candidate selection and data-engineering interpretation, but they remain validation evidence unless matched complete evaluations with the same feature configuration are available. Other component-level interpretations remain outside the present matched ablation surface.

RQ5. Dataset regime is central to interpretation. AmazonBook favours LightGCN except for EDGRec speed. MovieLens-1M shows close ranking with an EDGRec speed benefit but less favourable popularity and personalization diagnostics. Sparse KuaiRec v2 provides the most favourable EDGRec ranking and resource result. KuaiRand-1K remains a compact exposure-aware regime in which resource efficiency and Recall improve while NDCG, HitRatio, CRRU, and raw AveragePopularity are less favourable, so it is best read as feasibility and sensitivity evidence.

RQ6. CRRU is most informative when its components remain visible. On AmazonBook and MovieLens-1M, CRRU favours LightGCN because ranking and list behaviour advantages outweigh EDGRec’s speed. On sparse KuaiRec v2, CRRU favours EDGRec because ranking and epoch-time gains dominate the memory cost and mixed list behaviour diagnostics under the declared weights. On KuaiRand-1K, CRRU remains lower for compact EDGRec because the Recall and resource gains do not compensate for lower NDCG, lower HitRatio, and higher raw AveragePopularity against the LightGCN comparison with stronger top-position ranking evidence.

Figure 5.3 provides the corresponding component decomposition. Conclusions that depend on CRRU are therefore tied to the declared weights and should be read beside the unaggregated component metrics.

5.11 Limitations of the Empirical Evidence

- The reported tables are primarily single-seed or limited-seed evidence. They identify favourable and unfavourable regimes, but robustness across random initialization,

negative sampling, validation selection, and split variation remains an empirical extension.

- Offline top- K evaluation measures held-out relevance under the declared logged protocol. Online serving utility, user satisfaction, long-term feedback loops, and counterfactual policy value require additional evaluation designs.
- Observational implicit feedback leaves unobserved items as missing outcomes rather than verified dislikes. Sampled negatives are optimization contrasts, and observed positives remain conditional on exposure, interface, platform policy, and preprocessing choices.
- Preprocessing dependence remains substantial: positive-label thresholds, repeat collapse, item-universe policy, feature eligibility, candidate filtering, and train-only popularity denominators can all change graph structure and metric interpretation.
- The completed comparator set is centred on LightGCN as the main graph baseline. Matched final test evaluations for BPR-MF, DICE, and the GCN instantiation of DICE remain important extensions; the current reference DICE evaluations support feasibility discussion.
- Several EDGRec configurations are supporting, validation search, or ablation evaluations rather than selected final-test thesis candidates; they explain component behaviour and candidate selection but do not replace final test evidence.
- Cross-dataset synthesis is qualitative rather than a pooled benchmark score because feedback semantics, exposure evidence, density, side information, and item-universe policies differ across the evaluated regimes.
- Runtime and peak VRAM are hardware-dependent and sensitive to full-graph versus sampled training, fan-out, batch size, item-universe policy, CUDA allocation, accelerator libraries, and evaluation chunking.
- CRRU is a parameterized utility; conclusions that depend on the scalar value should be checked against reasonable alternative weight vectors and the unaggregated component metrics. The present results therefore support the declared resource-aware preference, not a universal ordering of recommender models.
- Causal terminology is used as representation motivation; exposure, popularity, conformity, side-feature, fairness, and treatment-effect estimands require separate identification and evaluation protocols.

5.12 Empirical Summary

The reported evidence supports a dataset- and configuration-specific thesis contribution. EDGRec demonstrates resource-aware benefits in selected regimes and a positive sparse KuaiRec ranking and resource result, while LightGCN remains stronger on AmazonBook and marginally stronger on MovieLens-1M under several component metrics. KuaiRand-1K illustrates the difficulty of efficient recommendation under randomized exposure, compact item universes, and sparse positives. The supported claim is Pareto-oriented rather than universal: EDGRec can serve as an efficient graph recommender with interest and conformity channels and favourable trade-offs in selected regimes when the data protocol, resource measurements, and component metrics remain visible.

Chapter 6

Conclusion

The dissertation studied efficient graph recommendation with interest and conformity separation under observational implicit-feedback data. The central empirical lesson is that such modelling should be evaluated as a dataset- and configuration-specific trade-off rather than as a single accuracy claim. The Efficient Disentangled Graph Recommender (EDGRec) contributes a graph recommender specification informed by causal recommendation together with an evaluation protocol that keeps ranking quality, popularity and personalization diagnostics, runtime, memory, and evidence boundaries visible before any composite utility is interpreted.

6.1 Summary of Contributions

The work makes four linked contributions. First, it gives a protocol-centred formulation of offline graph recommendation in which observed positives, missing outcomes, split masks, train-only graph construction, item-universe policy, negative sampling, feature-column lineage, and leakage risks are stated explicitly. This formulation keeps observational recommendation data aligned with what they measure: logged events under a particular protocol rather than complete preference labels or direct counterfactual outcomes. It also treats feature engineering as a scientific protocol: raw columns must be eligible under the leakage policy, encoded into declared groups, evaluated through validation-only feature subset search, and completed as test evaluations before they can support a primary thesis claim.

Second, it defines EDGRec, an Efficient Disentangled Graph Recommender. EDGRec adapts the interest and conformity separation motivated by the DICE causal-embedding framework to graph recommendation with LightGCN propagation. Its distinction from reference DICE is methodological and operational: EDGRec keeps the branch hypothesis but redesigns the training and evaluation contract around sampled graph training, bounded auxiliary losses, controlled score fusion, optional context and feature signals, and resource-aware configuration choices. This makes the contribution a practical graph-recommendation

framework for evaluating the branch hypothesis under explicit data and hardware constraints, rather than a direct replication of the DICE estimator. Its claims are tied to the observable ranking, list behaviour, ablation, runtime, and memory evidence reported in the experiments.

Third, it provides an evidence-labelled evaluation of EDGRec against completed reference LightGCN baselines, sampled-training comparators, reference DICE feasibility evaluations, representation diagnostics, feature-subset evidence, and matched ablations. The evidence labels separate validation search evaluations, feature subset candidates, feasibility evaluations, final test evaluations, and component diagnostics, allowing resource feasibility and candidate selection to be interpreted separately from final ranking evidence.

Fourth, it formalizes and applies Composite Resource-aware Recommendation Utility (CRRU). CRRU is a configurable post hoc utility over visible component metrics: Normalized Discounted Cumulative Gain (NDCG), Recall, HitRatio, AveragePopularity, Personalization, training epoch time, and peak training VRAM. Its framework is reusable across recommendation settings when the indicators, transformations, and weights are declared. Validation-CRRU provides model-selection evidence, while final-test CRRU provides post-selection evaluation evidence. In both cases, the interpretation is fixed by the dataset, split, candidate policy, hardware convention, and declared weights.

6.2 Main Empirical Findings

The empirical findings are dataset-specific. On AmazonBook, the completed comparison favours LightGCN on ranking quality, list behaviour, CRRU, and peak VRAM, while EDGRec provides an epoch-time advantage. The dataset therefore supports an efficiency trade-off rather than a recommendation-quality gain.

On MovieLens-1M, EDGRec is close to LightGCN in top- K ranking and is faster per epoch, but LightGCN remains slightly higher on Recall, CRRU, and Personalization, with lower raw AveragePopularity and lower peak VRAM under the reported evaluations. This supports EDGRec as an epoch-time-efficient configuration with closely matched ranking performance for the converted rating-to-implicit protocol, while list behaviour and memory remain the main costs in this dataset.

On KuaiRec v2, the sparse watch-ratio setting provides the most favourable result. The reported EDGRec evaluation has higher ranking metrics and shorter epoch time than the reference LightGCN comparison, while Personalization and peak VRAM remain less favourable. The conclusion is favourable for ranking and runtime in this specific sparse short-video protocol, with list behaviour and memory trade-offs kept visible.

On KuaiRand-1K, compact EDGRec gives a protocol-qualified resource and Recall trade-off. Runtime, peak VRAM, Recall, and Personalization are favourable against the LightGCN comparison with stronger top-position ranking evidence, while NDCG, HitRatio, CRRU, and raw AveragePopularity are less favourable. The dataset therefore highlights the difficulty

of combining randomized exposure, sparse positives, compact item-universe choices, and efficient graph training.

6.3 Scientific Limitations

The main scientific limitation is estimand scope. EDGRec is informed by causal recommendation, and the experiments report offline ranking, list behaviour, resource, and diagnostic evidence for the declared protocol. Identified exposure effects, treatment effects, counterfactual policy value, uplift, fairness effects, and ground-truth latent interest and conformity factors are separate scientific targets that require different estimands and additional data or assumptions. Observational implicit feedback contains missing outcomes and policy-dependent exposure; held-out ranking metrics therefore measure offline relevance under the declared logging and split protocol.

Statistical robustness is also limited by the available experiment set. The reported tables are primarily single-seed or limited-seed evidence. They support bounded qualitative conclusions and identify favourable or unfavourable regimes, especially where the main positive evidence is an efficiency or Pareto trade-off rather than a uniform accuracy improvement. Variance across random seeds, hyperparameter-search randomness, and alternative split construction remains an empirical extension. Broader comparative claims would require repeated complete evaluations with confidence intervals or paired statistical testing.

A third limitation is baseline completeness. LightGCN is the main reference full-graph baseline, while reference DICE is available as single-epoch feasibility evidence in the available result set. Completed DICE, the GCN instantiation of DICE, and Bayesian Personalized Ranking Matrix Factorization (BPR-MF) ranking baselines would strengthen the comparative surface. Comparisons against these methods or broader contemporary recommender systems therefore require additional matched evaluations.

A fourth limitation is utility sensitivity. CRRU uses declared weights and transformations to summarize visible component metrics, so conclusions that depend on CRRU should be checked against alternative weight vectors and against the unaggregated ranking, list behaviour, runtime, and memory measurements. It is reported as a transparent secondary summary whose selected weights represent one declared preference vector; alternate vectors may legitimately produce different model orderings.

A fifth limitation is preprocessing dependence. The positive-label threshold, repeat-collapse rule, split construction, item-universe policy, feature eligibility, and candidate filtering jointly define the training graph and the held-out ranking task. The conclusions therefore apply to the declared protocols rather than to the raw datasets independently of preprocessing.

A sixth limitation is component activation. The implementation contains optional feature fusion, temporal-history scoring, DirectAU alignment and uniformity terms, contrastive

losses, and calibrated propensity/IPW paths, but the conclusion only attributes effects to components that are enabled in named configurations with matching evidence. Search-space labels and validation-only candidates are provenance for future experiments; they do not by themselves establish final test component effects.

6.4 Engineering Limitations

The engineering evidence is tied to the experimental pipeline and hardware environment. Runtime and peak VRAM depend on full-graph versus sampled training, fan-out, batch size, negative sampling, evaluation chunking, CUDA memory accounting, and dataset-specific item-universe policy. EDGRec’s speedups are therefore system-level measurements under a declared training regime rather than architecture-intrinsic constants. This forms a central contribution: it shows that graph recommendation with interest and conformity separation can be made practical through sampled training and bounded losses, while keeping speed ratios tied to the reference-baseline contract, item-universe policy, and hardware profile. The positive interpretation is consequently Pareto oriented: efficiency improves the practical validation and replication budget when the associated ranking, list behaviour, and memory evidence remains acceptable for the stated protocol.

The empirical study also leaves several deployment questions outside scope. It evaluates offline top- K ranking rather than online serving latency, retrieval-index construction, incremental updates, calibration, user-facing diversity, or A/B-test performance. Large-scale production use would require additional engineering around candidate generation, monitoring, cold-start handling, and safety constraints.

6.5 Future Work

Future work should first strengthen empirical robustness. The most immediate extensions are repeated-seed complete evaluations, confidence intervals for the main tables, CRRU weight-sensitivity analysis, complete test evaluations for promising feature configurations, feature-quality audit metrics, and matched ablations across all datasets. Reference DICE, its Graph Convolutional Network instantiation, and a standalone Bayesian Personalized Ranking Matrix Factorization (BPR-MF) reference should be completed before positioning EDGRec against the broader recommender literature.

A second direction is causal evaluation beyond the present offline ranking protocol. Future studies would need explicit treatment definitions, validated exposure or propensity information, randomized interventions, or counterfactual evaluation protocols. KuaiRand and KuaiRec provide relevant exposure-related structure, but randomized or fully observed data alone do not remove the need for a precise estimand, positivity assumptions, and protocol-specific outcome definitions.

A third direction is method development. EDGRec could be extended with more sensitive branch-collapse diagnostics, calibrated score fusion, safer side-feature policies, temporal modelling, multimodal item representations, and scalable retrieval integration. These extensions should be evaluated through component metrics and resource measurements before being summarized by any composite utility.

6.6 Final Remarks

The evidence supports a dataset-specific claim: efficient graph recommendation with interest and conformity separation is a useful framework for studying bias-aware and resource-aware recommendation under observational data. EDGRec provides a favourable trade-off between ranking quality and epoch time in selected regimes, most strongly in the sparse KuaiRec v2 watch-ratio setting reported in Chapter 5. The broader conclusion is methodological as well as empirical: careful preprocessing, explicit evidence labels, component-level metrics, and resource-aware reporting strengthen the scientific credibility of recommender-system evaluation, especially when the central contribution is to expand the feasible accuracy–resource frontier rather than to assert uniform accuracy, memory, or causal dominance.

Bibliography

- [1] A. Jadon and A. Patil, “A comprehensive survey of evaluation techniques for recommendation systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.16015>
- [2] D. Jannach and H. Abdollahpouri, “A survey on multi-objective recommender systems,” *Frontiers in Big Data*, vol. Volume 6 - 2023, 2023. [Online]. Available: <https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2023.1157899>
- [3] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, “Bpr: Bayesian personalized ranking from implicit feedback,” in *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, ser. UAI ’09. Arlington, Virginia, USA: AUAI Press, 2009, p. 452–461.
- [4] C. Gao, Y. Zheng, W. Wang, F. Feng, X. He, and Y. Li, “Causal inference in recommender systems: A survey and future directions,” *ACM Trans. Inf. Syst.*, vol. 42, no. 4, Feb. 2024. [Online]. Available: <https://doi.org/10.1145/3639048>
- [5] Y. Zheng, C. Gao, X. Li, X. He, Y. Li, and D. Jin, “Disentangling user interest and conformity for recommendation with causal embedding,” in *Proceedings of the Web Conference 2021*, ser. WWW ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2980–2991. [Online]. Available: <https://doi.org/10.1145/3442381.3449788>
- [6] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, “Lightgcn: Simplifying and powering graph convolution network for recommendation,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 639–648. [Online]. Available: <https://doi.org/10.1145/3397271.3401063>
- [7] S. Greco, A. Ishizaka, M. Tasiou, and G. Torrisi, “On the methodological framework of composite indices: A review of the issues of weighting, aggregation, and robustness,” *Social Indicators Research*, vol. 141, pp. 1–34, 01 2019.
- [8] S. Wu, F. Sun, W. Zhang, X. Xie, and B. Cui, “Graph neural networks in recommender systems: A survey,” *ACM Comput. Surv.*, vol. 55, no. 5, Dec. 2022. [Online]. Available: <https://doi.org/10.1145/3535101>

- [9] C. Gao, Y. Zheng, N. Li, Y. Li, Y. Qin, J. Piao, Y. Quan, J. Chang, D. Jin, X. He, and Y. Li, “A survey of graph neural networks for recommender systems: Challenges, methods, and directions,” *ACM Trans. Recomm. Syst.*, vol. 1, no. 1, Mar. 2023. [Online]. Available: <https://doi.org/10.1145/3568022>
- [10] H. Luo, F. Zhuang, R. Xie, H. Zhu, D. Wang, Z. An, and Y. Xu, “A survey on causal inference for recommendation,” *The Innovation*, vol. 5, no. 2, p. 100590, Mar. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.xinn.2024.100590>
- [11] S. Xu, J. Ji, Y. Li, Y. Ge, J. Tan, and Y. Zhang, “Causal inference for recommendation: Foundations, methods, and applications,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 3, Apr. 2025. [Online]. Available: <https://doi.org/10.1145/3714430>
- [12] F. M. Harper and J. A. Konstan, “The movielens datasets: History and context,” *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, Dec. 2015. [Online]. Available: <https://doi.org/10.1145/2827872>
- [13] C. Wang, Y. Yu, W. Ma, M. Zhang, C. Chen, Y. Liu, and S. Ma, “Towards representation alignment and uniformity in collaborative filtering,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1816–1825. [Online]. Available: <https://doi.org/10.1145/3534678.3539253>
- [14] C. Gao, S. Li, W. Lei, J. Chen, B. Li, P. Jiang, X. He, J. Mao, and T.-S. Chua, “KuaiREC: A fully-observed dataset and insights for evaluating recommender systems,” in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, ser. CIKM ’22. ACM, Oct. 2022, p. 540–550. [Online]. Available: <http://dx.doi.org/10.1145/3511808.3557220>
- [15] C. Gao, S. Li, Y. Zhang, J. Chen, B. Li, W. Lei, P. Jiang, and X. He, “KuaiRand: An unbiased sequential recommendation dataset with randomly exposed videos,” in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, ser. CIKM ’22. ACM, Oct. 2022, p. 3953–3957. [Online]. Available: <http://dx.doi.org/10.1145/3511808.3557624>
- [16] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, “Graph convolutional neural networks for web-scale recommender systems,” *CoRR*, vol. abs/1806.01973, 2018. [Online]. Available: <http://arxiv.org/abs/1806.01973>
- [17] M. Liu, D. Zheng, J. Su, and C. Wu, “Full-graph vs. mini-batch training: Comprehensive analysis from a batch size and fan-out size perspective,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.22678>

- [18] S. Bonner and F. Vasile, “Causal embeddings for recommendation,” *CoRR*, vol. abs/1706.07639, 2017. [Online]. Available: <http://arxiv.org/abs/1706.07639>
- [19] Z. Liu, Y. Fang, and M. Wu, “Estimating propensity for causality-based recommendation without exposure data,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [20] W. Zhao, D. Tang, X. Chen, D. Lv, D. Ou, B. Li, P. Jiang, and K. Gai, “Disentangled causal embedding with contrastive learning for recommender system,” in *Companion Proceedings of the ACM Web Conference 2023*, ser. WWW ’23 Companion. New York, NY, USA: Association for Computing Machinery, 2023, p. 406–410. [Online]. Available: <https://doi.org/10.1145/3543873.3584637>
- [21] L. Zou, X. Liu, J. Jiang, S. Chen, X. Xia, Y. Yang, P. De Meo, and G. Fiumara, “Causal-augmented disentanglement for contrastive recommendation,” *International Journal of Modern Physics C*, 12 2025.
- [22] Y. Zheng, C. Gao, J. Chang, Y. Niu, Y. Song, D. Jin, and Y. Li, “Disentangling long and short-term interests for recommendation,” in *Proceedings of the ACM Web Conference 2022*, ser. WWW ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2256–2267. [Online]. Available: <https://doi.org/10.1145/3485447.3512098>
- [23] C. Wang, Y. Ye, L. Ma, D. Li, and L. Zhuang, “Dual disentanglement of user-item interaction for recommendation with causal embedding,” *Information Processing & Management*, vol. 60, no. 5, p. 103456, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457323001930>
- [24] W. Wang, X. Lin, L. Wang, F. Feng, Y. Ma, and T.-S. Chua, “Causal disentangled recommendation against user preference shifts,” *ACM Trans. Inf. Syst.*, vol. 42, no. 1, Aug. 2023. [Online]. Available: <https://doi.org/10.1145/3593022>
- [25] A.-A. Liu, Y. Zhao, X. Wen, R. Chang, and W. Nie, “Causal disentanglement for tackling popularity bias in sequential recommendation,” *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 1, pp. 426–438, 2026.
- [26] X. Wang, Q. Li, D. Yu, P. Cui, Z. Wang, and G. Xu, “Causal disentanglement for semantic-aware intent learning in recommendation,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 10, pp. 9836–9849, 2023.
- [27] S. Mu, Y. Li, W. X. Zhao, S. Li, and J.-R. Wen, “Knowledge-guided disentangled representation learning for recommender systems,” *ACM Trans. Inf. Syst.*, vol. 40, no. 1, Sep. 2021. [Online]. Available: <https://doi.org/10.1145/3464304>

- [28] Y. Wang, X. Wang, X. Huang, Y. Yu, H. Li, M. Zhang, Z. Guo, and W. Wu, “Intent-aware recommendation via disentangled graph contrastive learning,” in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, ser. IJCAI ’23, 2023. [Online]. Available: <https://doi.org/10.24963/ijcai.2023/260>
- [29] Y. Du, Z. Wang, Z. Sun, Y. Ma, H. Liu, and J. Zhang, “Disentangled multi-interest representation learning for sequential recommendation,” in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 677–688. [Online]. Available: <https://doi.org/10.1145/3637528.3671800>
- [30] A. Chen, A. Chow, A. Davidson, A. DCunha, A. Ghodsi, S. A. Hong, A. Konwinski, C. Mewald, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, A. Singh, F. Xie, M. Zaharia, R. Zang, J. Zheng, and C. Zumar, “Developments in mlflow: A system to accelerate the machine learning lifecycle,” in *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, ser. DEEM ’20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: <https://doi.org/10.1145/3399579.3399867>
- [31] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 2623–2631. [Online]. Available: <https://doi.org/10.1145/3292500.3330701>
- [32] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang, “Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.15136>
- [33] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [34] L. McInnes, J. Healy, N. Saul, and L. Großberger, “Umap: Uniform manifold approximation and projection,” *Journal of Open Source Software*, vol. 3, no. 29, p. 861, 2018. [Online]. Available: <https://doi.org/10.21105/joss.00861>

Appendix A

Supplementary Material

This appendix records supplementary audit material that supports the experimental contract without repeating the conclusion. The sections below collect data-feature lineage and reproducibility checks referenced from Chapters 3–5, including Composite Resource-aware Recommendation Utility (CRRU) inputs and peak training video random-access memory (VRAM) provenance, while keeping the main empirical narrative in the methods, setup, and results chapters.

A.1 Data and Feature-Lineage Audit

Chapter 3 states the feature policy and encoding contract in Tables 3.4 and 3.5. Chapter 4 then fixes how validation-only feature subset search may select candidate configurations for later testing in Table 4.4. Table A.1 keeps those contracts together at the appendix level: it states which dataset views, feature groups, exclusions, and evidence boundaries can support a thesis claim. The table is an audit table rather than a result table; it records whether a source column is a safe descriptor, train-derived context, candidate propensity-calibration field, excluded post-treatment aggregate, or component-diagnostic signal.

Table 5.2 reports the main feature evidence. Figure 4.11 gives the validation search view in the main experimental setup chapter: it is useful for explaining which feature groups moved ValidationCRRU, but it does not replace complete test evidence. Thus, feature availability, validation usefulness, and final test attribution remain three separate audit states.

A.2 Environment and Reproducibility Checklist

Chapter 4 reports the primary hardware environment in Table 4.5 and the runtime reporting contract in Table 4.6. Table A.2 records what must be present for a result to be reproducible and interpretable.

Table A.1: Appendix data-feature lineage and leakage audit summary.

Dataset view	Positive-label and graph source	Feature-lineage status	Allowed thesis use
AmazonBook interaction-only	Provided graph interactions are treated as observed positives; propagation, popularity, and popularity-conditioned masks use positive training edges only.	Side-information sources are outside the primary thesis contract; the feature state is interaction-only.	Sparse collaborative-filtering and resource evidence only. Side-information claims are outside the scope for this dataset.
MovieLens-1M rating-to-implicit	Explicit ratings are converted into implicit positives under the configured threshold and split policy.	Item genre is the eligible item group; user demographics are loaded metadata but are not part of the searched thesis feature configurations. Validation evidence selects no side features.	Converted top- K ranking evidence. Genre availability is documented, but the current search evidence does not support a genre-feature claim.
KuaiRec v2 sparse matrix	The primary thesis view is the sparse watch-ratio protocol with positives defined by the configured watch-ratio threshold.	Safe item descriptors include author and music, video metadata, resolution, category or tag, and upload-time groups. Engagement aggregates stay excluded from primary scoring features.	Main KuaiRec claims use the sparse watch-ratio comparison. Feature configurations are candidate evidence unless matched complete evaluations support the same configuration. Small-matrix evaluations are sensitivity evidence only.
KuaiRand-1K exposure-aware and random-only views	Standard and randomized exposure records are retained according to the configured item-universe policy; compact evaluations may use randomized-exposure items only.	Safe item descriptors include author and music, video metadata, resolution, category or tag, and upload-time groups. Statistic and engagement aggregates stay excluded from ordinary model features; show-count information is reserved for explicit propensity-calibration or IPW ablations, not default scoring.	Exposure-aware evidence. Current feature movements are validation and search evidence, while compact complete evaluations require protocol-qualified interpretation because ranking and resource metrics form a trade-off.
MovieLens-20M and Taobao preprocessing inventory	Raw files were screened and schema-checked, but no trained model or complete test evaluation is reported in the thesis result set.	Feature eligibility was not selected into a model configuration under the thesis reporting contract.	Preprocessing inventory only. These sources do not support ranking, CRRU, runtime, or VRAM claims in Chapters 4–5.

The reported experiments are supported by supplementary experimental materials and result summaries for auditing empirical comparisons. The appendix documents the minimum provenance needed to interpret those materials; Chapter 5 remains the main scientific summary, and supplementary records do not change an evidence category unless a configuration is rerun under the declared protocol.

Table A.2: Environment and reproducibility checklist.

Checklist item	Required record	Reason for interpretation
Hardware environment	CPU, GPU, system memory, storage class, operating-system family, CUDA/accelerator path, and any run-specific environment record.	Runtime and peak VRAM are hardware-dependent. The values in Table 4.5 define the local thesis workstation, not an architecture-intrinsic constant.
Software environment	Python, PyTorch [33], PyTorch Geometric, Optuna, CUDA, and related package versions stored with the run or environment export.	Package versions can affect sparse kernels, CUDA allocation behaviour, AMP behaviour, and metric implementations.
Dataset protocol	Dataset name, preprocessing protocol, split masks, positive-label rule, item-universe policy, feature policy, and candidate filtering.	These fields determine graph size, relevance targets, seen-item exclusion, popularity denominators, and whether side-feature evidence is admissible.
Model and configuration record	Model family, configuration description, search space revision when applicable, effective configuration, random seed policy, stored model-state record, and evidence role.	Validation candidates, final test evaluations, runtime feasibility evaluations, archival records, and diagnostics have different interpretation scopes.
Selection boundary	Validation objective, completed-trial eligibility, selection decision, and the final test evaluation when available.	Test metrics are excluded from model selection. ValidationCRRU can select a configuration, but final-test CRRU is interpreted only after complete test evaluation.
Metric provenance	NDCG, Recall, HitRatio, raw train-count AveragePopularity, Personalization, epoch time, peak VRAM field, and CRRU inputs at each reported cutoff.	CRRU is meaningful only when its visible components remain available and when raw popularity/resource inputs use the declared transformations.
Result provenance	Structured experiment record, result summaries, Optuna study metadata, feature-analysis summaries, model-state metadata, and figure provenance record.	The archived records define the evidence surface; figures summarize those records rather than replacing them.
Feasibility and failure labels	Runtime feasibility status, CPU/GPU fallback, automatic batch-size decisions, out-of-memory records, archival record status, and component-diagnostic labels.	Feasibility evidence and final ranking evidence must remain separated, especially for reference DICE and hardware-scaled baseline variants.

This procedural checklist defines the minimum context needed to rerun, audit, or correctly qualify the empirical comparisons reported in Chapter 5. It preserves the distinction between final test evidence, validation search evidence, runtime feasibility evaluations, and diagnostics without introducing an additional empirical claim. The same checklist is also the provenance boundary for CRRU: the scalar can be recomputed only when the ranking

metrics, raw train-count AveragePopularity, Personalization, epoch duration, peak VRAM, cutoff, and declared weight vector are all available under the same dataset and candidate protocol.